

A COMPLETE COURSE, BUILT FROM YOUR OWN MATERIALS

# Digital Communication Decoded

Everything in the nine PDFs you gave me — the lecture decks, the tutorials and the handwritten notes — rebuilt from zero in plain English. No prior knowledge assumed. Every formula explained word by word, every tutorial problem solved step by step.

---

## Subject

EC301 Digital  
Communication

## Built from

171 pages, 9 PDFs

## Covers

Modules 1–8 below

## Reading time

About 5–6 hours

---

**00** START HERE

# How to use this course

Read it in order. Each module is built on the one before it. Do not skip the worked examples — in this subject the exam is mostly worked examples.

You told me you know nothing about this subject. So this course starts from truly zero. I explain what every letter in every formula means, every single time it first appears. If a sentence in your PDF was written in hard English, I have rewritten it in easy English and kept the exact original meaning.

## The four kinds of boxes

As you read, you will keep meeting the same four boxes. They always mean the same thing:

### FORMULA

A formula you must remember. Written once in symbols, then once in words.

### IN SIMPLE WORDS

The same idea again, with no maths at all. If the formula box confused you, this box will fix it.

### WORKED EXAMPLE

A real question from your tutorial PDFs, solved step by step with every number shown.

### EXAM TIP

Something that is very likely to be asked, or a mistake students usually make.

## What this course covers

Your syllabus (EC301) has five units. But your *materials* only cover part of them, plus some earlier background topics. This course teaches exactly what your materials cover — nothing missing, nothing invented.

| WHAT YOUR PDFS ACTUALLY CONTAIN |                                                                  |                 |
|---------------------------------|------------------------------------------------------------------|-----------------|
| FILE                            | TOPIC                                                            | TAUGHT IN       |
| first.pdf                       | Radio spectrum & where real systems sit                          | Module 1        |
| second.pdf                      | Full transmitter-channel-receiver block diagram                  | Module 1        |
| third.pdf                       | Information Theory (42 slides)                                   | Modules 6, 7, 8 |
| fourth.pdf                      | Handwritten Huffman tree                                         | Module 7        |
| fifth.pdf                       | Tutorial 2 – line codes, raised cosine, TDM                      | Modules 2, 3, 4 |
| sixth.pdf                       | Tutorial 4 – information & entropy sums                          | Module 5        |
| seventh.pdf                     | Tutorial 3 – PCM numericals                                      | Module 2        |
| eight.pdf                       | Channel Coding (26 slides)                                       | Module 8        |
| ninth.pdf                       | Digital data transmission over a bandlimited channel (57 slides) | Modules 3, 4    |

#### HONEST NOTE

Your syllabus also lists **ASK / PSK / FSK carrier modulation, spread spectrum (DSSS, FHSS, CDMA), OFDM, and symbol synchronisation**. There is *nothing* on those topics in the PDFs you gave me, so this course does not teach them. If you get materials for them later, send them and I will add those modules.

**What ninth.pdf added.** It covers **Unit 2 of your syllabus in full** — digital transmission through bandlimited channels, signal design for bandlimited channels, and system design in the presence of channel distortion. Module 3 has therefore been expanded, and a whole new **Module 4** written. Equalization is now covered, so it has been removed from the missing list above.

One more thing: I found three genuine misprints in your slides — one on slide 8 of the Channel Coding deck, and two in ninth.pdf. I teach the correct version each time and flag the misprint clearly, so you are not caught out in the exam.

## 01 FROM SECOND.PDF AND FIRST.PDF

# The whole system, end to end

Before any formula, you need the map. Every other module in this course is a zoom-in on one box of the diagram below.

## 1.1 What problem are we even solving?

Somebody wants to send something — a voice, a photo, a video, a sensor reading — to somebody far away. Between them is a **channel**: air, a copper cable, an optical fibre, or space. The channel is not kind. It weakens the signal, it adds noise, it bends the signal out of shape, and other people's signals crash into yours.

So the goal of a digital communication system is written in one line at the end of your second.pdf:

### THE ONE-LINE GOAL

Send information **reliably** (very few mistakes) and **efficiently** (fast, and without wasting bandwidth or battery) across a noisy channel.

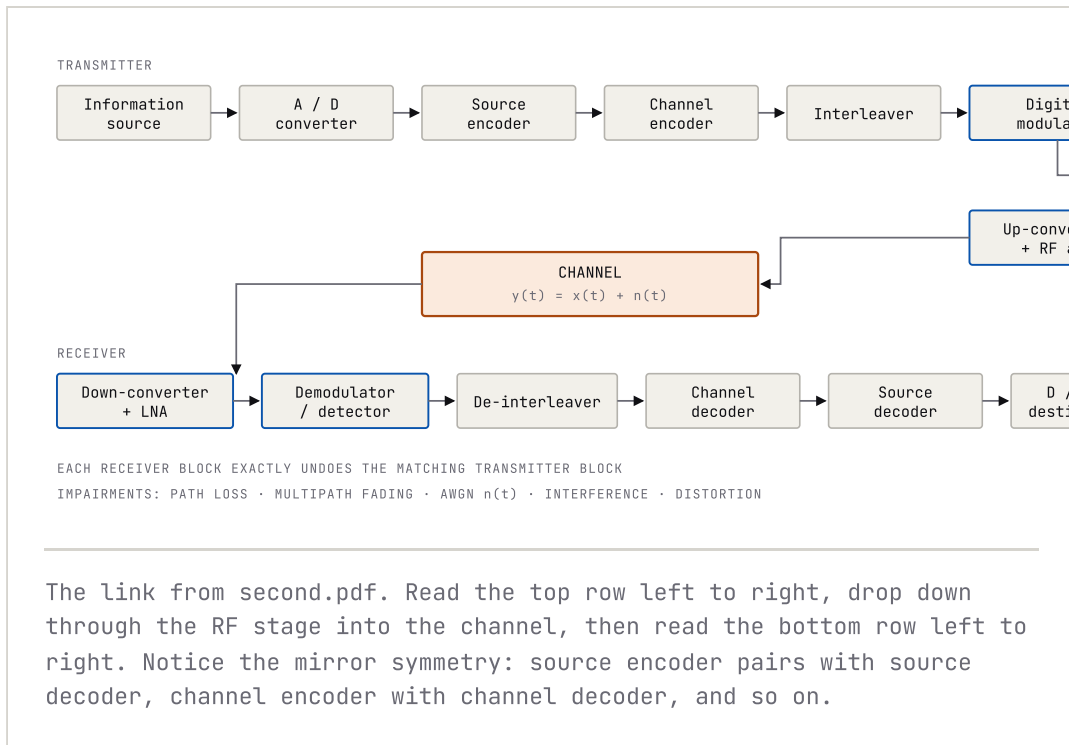
Everything else — entropy, Huffman coding, Hamming codes, Viterbi decoding — is just a tool for hitting those two words: *reliably* and *efficiently*.

### Why go digital at all?

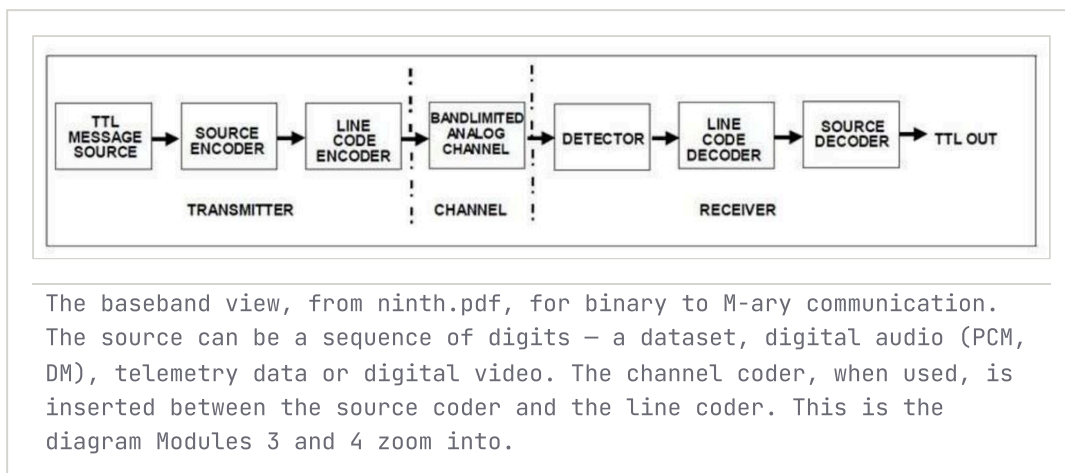
An analog signal that is damaged stays damaged, and every amplifier along the way makes it a bit worse. A digital signal is only ever a 0 or a 1. As long as the noise is not big enough to flip a 0 into a 1, the receiver can rebuild the signal *perfectly*. And because it is only 0s and 1s, we can do clever things to it: compress it, encrypt it, and add error-correcting maths. You cannot do any of that easily with an analog waveform.

## 1.2 The block diagram you must be able to draw

This is the diagram in your second.pdf. It is worth 10–15 marks on its own, so learn to draw it from memory.



Your ninth.pdf draws the same link in a simpler form — the version used when we are studying *baseband* transmission, so the modulator is replaced by a **line code encoder** and the channel coding stage is left out.



## 1.3 What each block actually does

Take them one at a time. This is the vocabulary for the rest of the course.

### Information source

The thing making the message: a microphone, a camera, a keyboard, a temperature sensor.

### A/D converter (analog to digital)

Turns a smooth real-world wave into numbers. Three steps: **sampling** (take readings at regular times), **quantization** (round each reading to the nearest allowed level), **encoding** (write each level as a binary number). This whole block is Module 2.

## Source encoder

**Squeezes** the data. It removes the parts that are repeated or predictable, so fewer bits are needed. This is compression. Think ZIP, JPEG, MP3. This is Module 7.

## Channel encoder (FEC)

**Adds** extra bits back — but not random extra bits. Carefully designed extra bits that let the receiver spot and repair errors caused by noise. This is Module 8.

## Interleaver

Optional. It shuffles the order of the bits before sending. If the channel then wipes out a whole burst of neighbouring bits, un-shuffling at the far end spreads that damage thinly across many codewords, and error correction can handle thin damage far better than one big hole.

## Digital modulator

Turns groups of bits into actual electrical waveforms (called **symbols**) that can travel. Examples from your syllabus: BPSK, QPSK, 16-QAM, 64-QAM, 256-QAM.

## Up-converter and RF amplifier

Shifts the signal up to the radio frequency that the antenna can radiate, and makes it powerful enough to survive the journey.

## Channel

Air, cable, fibre, satellite link. Where all the damage happens.

## LNA and down-converter

LNA = Low Noise Amplifier. The received signal is tiny, so it must be amplified before anything else — and by an amplifier that adds as little noise of its own as possible. Then the signal is shifted back down from radio frequency.

## Demodulator / detector

Looks at the received waveform and decides: was that symbol a 0 or a 1 (or which of the 16 QAM points)? This is a *guess*, and sometimes the guess is wrong. That is where bit errors are born.

## Channel decoder

Uses the extra bits the channel encoder added to find and fix the wrong guesses.

## Source decoder

Un-squeezes the compressed data back to full size.

## D/A converter

Turns the numbers back into a smooth wave for the speaker or screen.

### EXAM TIP

Examiners love this question: “Source coding removes redundancy but channel coding adds redundancy. Is this not a contradiction?”

Answer: No. Source coding removes **useless, uncontrolled** redundancy that was already sitting in the data by accident. Channel coding then adds back a small

amount of **useful, carefully designed** redundancy that is mathematically built to fight noise. You throw out the junk, then add the right tool.

## 1.4 How we score a system

Four numbers decide whether a design is good. These appear in your second.pdf and again in Module 8.

### FORMULA – PERFORMANCE MEASURES

**BER** = (number of bits received wrongly) ÷ (total number of bits sent)

**SER** = (number of symbols received wrongly) ÷ (total symbols sent)

**SNR** = signal power ÷ noise power → often written as  $E_b/N_0$

$SNR_{dB} = 10 \log_{10}(E_b/N_0)$

### IN SIMPLE WORDS

**BER** (Bit Error Rate) is your report card. A BER of  $10^{-6}$  means one bit in a million comes out wrong. Lower is better.

$E_b$  is the energy spent on one bit.  $N_0$  is the noise power in each hertz of bandwidth. So  $E_b/N_0$  asks: *how loud is one bit compared to the hiss?* Louder means fewer mistakes, so higher  $E_b/N_0$  always gives lower BER.

We write it in **dB** because these numbers span a huge range. The formula  $10 \log_{10}(\dots)$  squashes a range of millions into a comfortable range of about 0 to 60.

What can you change to improve BER? Your PDF lists five levers: the modulation scheme, the bandwidth, the  $E_b/N_0$ , the coding technique, and the channel conditions. Only the first four are under your control.

## 1.5 What is AWGN?

You will see “AWGN” on nearly every slide and in every unit heading of your syllabus. Break the four letters apart:

- **A — Additive.** The noise simply *adds* to your signal:  $y(t) = x(t) + n(t)$ . It does not multiply or twist it.
- **W — White.** Like white light contains all colours, this noise contains all frequencies equally. Its power is spread flat across the whole spectrum.
- **G — Gaussian.** The noise voltage at any instant follows the bell curve. Small values are common, huge values are rare.
- **N — Noise.** Unwanted random electrical energy, mostly from the random jiggling of electrons because of heat.

So an AWGN channel is the simplest useful model of a real channel: your signal, plus flat random bell-curve hiss. It is the standard baseline that every textbook uses.

## 1.6 Where in the spectrum does all this live?

Your first.pdf is a map of the electromagnetic spectrum. You do not need to memorise every number, but you should know the order and roughly where the famous systems sit.

| SPECTRUM, FROM LOWEST FREQUENCY TO HIGHEST |                    |                                                          |
|--------------------------------------------|--------------------|----------------------------------------------------------|
| BAND                                       | FREQUENCY          | USED FOR                                                 |
| Radio waves                                | $10^5$ – $10^9$ Hz | AM/FM radio, TV, maritime and aviation VHF               |
| → FM broadcast                             | 88–108 MHz         | Car radio                                                |
| → Mobile 2G/3G/4G                          | 700 MHz–2.6 GHz    | Phone calls, mobile data                                 |
| → 5G sub-6                                 | 3–7 GHz            | Main 5G coverage layer                                   |
| Microwaves                                 | $\sim 10^{11}$ Hz  | Satellite links, radar, backhaul, 5G mmWave (24–100 GHz) |
| Terahertz                                  | 100 GHz–3 THz      | 6G research                                              |
| Infrared                                   | $\sim 10^{13}$ Hz  | TV remotes, optical fibre                                |
| Visible light                              | $\sim 10^{15}$ Hz  | Li-Fi / visible light communication                      |
| Ultraviolet                                | $\sim 10^{17}$ Hz  | Research                                                 |
| X-rays                                     | $\sim 10^{21}$ Hz  | Imaging, research                                        |
| Gamma rays                                 | $\sim 10^{23}$ Hz  | Research                                                 |

#### IN SIMPLE WORDS

Higher frequency gives you more bandwidth, so more data per second. But higher frequency also travels a shorter distance and is blocked more easily by walls, rain and even leaves. That is the eternal trade: **5G mmWave is blisteringly fast but only works across a street; 700 MHz is slower but covers a whole town.**

Your first.pdf also lists where this is going: smart cities, self-driving cars, industrial IoT, remote surgery, AR/VR, holographic calls, drones, low-orbit satellite internet, quantum communication and AI-managed networks. All of them need more bits per second, more reliably — which is exactly what the rest of this course is about.

## 02 FROM SEVENTH.PDF — TUTORIAL 3

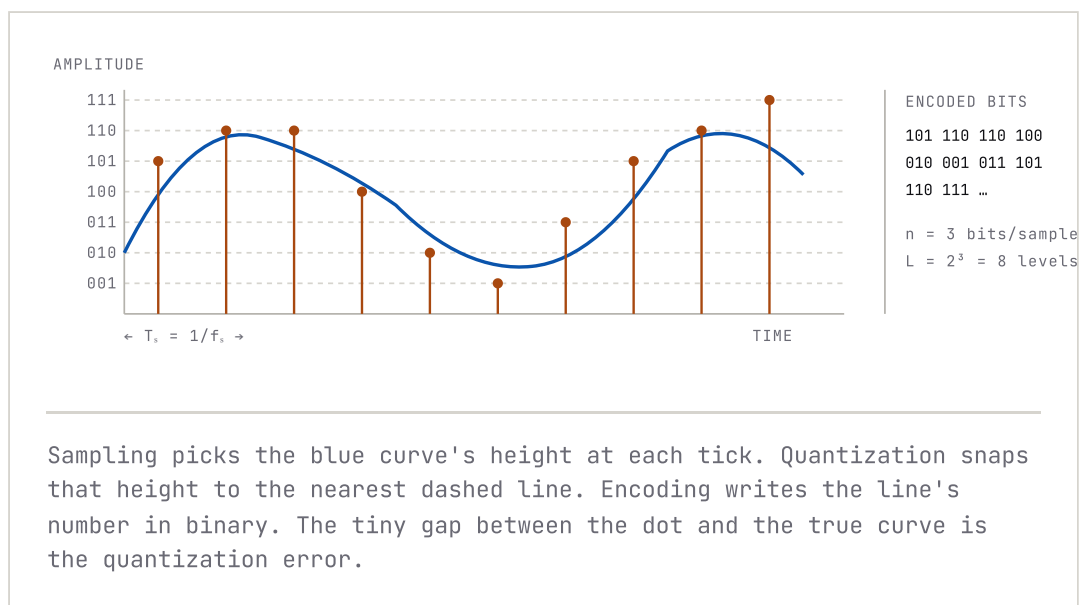
# Sampling, quantization and PCM

This is the A/D block of Module 1, opened up. It is also the single most exam-heavy numerical topic in your materials — a whole tutorial is devoted to it.

## 2.1 The three steps of PCM

**PCM** stands for **Pulse Code Modulation**. It is the standard recipe for turning a smooth analog signal into bits. It has exactly three steps, and you must be able to name them:

1. **Sampling** — measure the signal's height at regular time intervals.
2. **Quantization** — round each measured height to the nearest allowed level.
3. **Encoding** — write each level number as a binary codeword.



## 2.2 Step 1 — Sampling and the Nyquist rule

How often must you measure? If you measure too rarely, you miss the fast wiggles and the rebuilt signal is wrong — a fault called **aliasing**. The answer is the most famous rule in the subject:

FORMULA — NYQUIST SAMPLING CRITERION

$$f_s \geq 2 f_m$$

$f_s$  = sampling frequency (samples per second) ·  $f_m$  =  
highest frequency present in the signal (Hz)

The value  $2 f_m$  is called the **Nyquist rate**. Sampling exactly at it is the minimum; sampling above it is safer.

#### IN SIMPLE WORDS

You must take at least **two** samples in every cycle of the fastest wiggle in your signal. One sample to catch the peak, one to catch the trough. Fewer than two and a fast wave can pretend to be a slow wave, and you can never tell them apart afterwards.

Example: human speech on a telephone is limited to about 4 kHz, so telephones sample at 8 000 times a second. Music goes up to 20 kHz, so CDs sample at 44 100 times a second — slightly above  $2 \times 20$  kHz, to leave room for the filter.

## 2.3 Step 2 — Quantization

A sample's true height could be any number at all: 2.718281828... volts. We cannot store infinite digits, so we allow only **L** fixed levels and round to the nearest one. The rounding error we accept is called **quantization noise**.

#### FORMULA — LEVELS, BITS AND STEP SIZE

$$n = \log_2 L \quad (\text{bits per sample}) \quad \text{or equivalently} \quad L = 2^n$$

$$\Delta = V_{\text{peak-to-peak}} / 2^n \quad (\text{step size, volts})$$

$$P_q = \Delta^2 / 12 \quad (\text{quantization noise power})$$

$$\epsilon_{\max} = \Delta / 2 \quad (\text{largest possible rounding error})$$

#### IN SIMPLE WORDS

$\Delta$  (say “delta”) is the height of one step on the ladder. Take the full voltage range of the signal and cut it into  $2^n$  equal slices — each slice is  $\Delta$  tall.

Because you always round to the nearest level, the worst you can ever be wrong is **half a step**, which is why  $\epsilon_{\max} = \Delta/2$ .

The  $\Delta^2/12$  comes from statistics: if the error is equally likely to be anywhere between  $-\Delta/2$  and  $+\Delta/2$ , its average power works out to exactly  $\Delta^2/12$ . Just memorise it — the derivation is not asked.

### Signal-to-quantization-noise ratio

This is the number that says how good your quantizer is. There are two forms and you need both.

#### FORMULA – SQNR, BOTH FORMS

General:  $SNR = V_{rms}^2 \div (\Delta^2/12)$  = signal power  $\div$  quantization noise power

Full-scale sine wave only:  $SQNR_{dB} \approx 6.02 n + 1.76$

#### EXAM TIP – WHICH FORM TO USE

Use **6.02n + 1.76** *only* when the question says the signal is a sine wave filling the whole range (“full-scale sinusoidal”). That is Tutorial 3 Question 1.

Use  $V_{rms}^2 \div (\Delta^2/12)$  whenever the question gives you an actual  $V_{rms}$  value. That is Tutorial 3 Question 2. Using the wrong one is the most common mistake in this topic.

Remember the headline: **every extra bit buys you about 6 dB**. Double the number of levels, and quantization noise drops by 6 dB.

## 2.4 Step 3 — Encoding, bit rate and bandwidth

#### FORMULA – PCM BIT RATE AND TRANSMISSION BANDWIDTH

$R_b = n \times f_s$  (bits per second)

Polar NRZ:  $B_T \approx R_b / 2$

RZ:  $B_T \approx R_b$

#### IN SIMPLE WORDS

Bit rate is simply *how many samples per second* multiplied by *how many bits each sample costs*. Nothing clever.

The bandwidth rules say: an RZ pulse is only half a bit wide, so it wiggles twice as fast, so it needs twice the bandwidth of NRZ. You will meet RZ and NRZ properly in Module 3.

## 2.5 Worked example — Tutorial 3, Question 1

### WORKED EXAMPLE – TV SIGNAL

**Question.** A television signal having a bandwidth of 4.2 MHz is transmitted using a binary PCM system. The number of quantization levels is 512. Determine (1) code word length, (2) bit rate, (3) transmission bandwidth, (4) signal-to-quantization-noise ratio.

1 **Write down what you are given**

$$f_m = 4.2 \text{ MHz} \quad \cdot \quad L = 512$$

2 **Find the sampling frequency (Nyquist)**

$$f_s = 2 f_m = 2 \times 4.2 \text{ MHz}$$

$$f_s = 8.4 \text{ MSamples/sec}$$

3 **Find the code word length**

Ask: 2 to what power gives 512? Since  $512 = 2^9$ :

$$n = \log_2 L = \log_2 512 = 9$$

$$n = 9 \text{ bits/sample}$$

4 **Find the bit rate**

$$R_b = n \times f_s = 9 \times 8.4 \times 10^6$$

$$R_b = 75.6 \text{ Mb/s}$$

### 5 Find the transmission bandwidth — give both cases

Polar NRZ:  $B_T \approx R_b/2 = 75.6/2$

$$B_T \text{ (NRZ)} \approx 37.8 \text{ MHz}$$

RZ:  $B_T \approx R_b$

$$B_T \text{ (RZ)} \approx 75.6 \text{ MHz}$$

### 6 Find the SQNR

No  $V_{\text{rms}}$  is given, and we assume a full-scale sine wave, so use the 6.02n rule:

$$\text{SQNR}_{\text{dB}} = 6.02 \text{ n} + 1.76 = 6.02(9) + 1.76 = 54.18 + 1.76$$

$$\text{SQNR} \approx 55.94 \text{ dB} \approx 56 \text{ dB}$$

## 2.6 Worked example — Tutorial 3, Question 2

This one runs *backwards*. The channel speed is fixed, and you must work out how many bits per sample you are allowed to afford.

### WORKED EXAMPLE — FITTING PCM INTO A 50 KBIT/S CHANNEL

**Question.** A signal of bandwidth 3.5 kHz is sampled, quantized and coded by a PCM system. The coded signal is transmitted over a channel supporting a transmission rate of 50 kbits/sec. Determine the maximum signal-to-noise ratio obtainable. The input signal has  $V_{\text{peak-to-peak}} = 4 \text{ V}$  and  $V_{\text{rms}} = 0.2 \text{ V}$ .

#### 1 Write down what you are given

$$\begin{aligned} f_m &= 3.5 \text{ kHz} \quad \cdot \quad R_b = 50 \text{ kbit/s} \quad \cdot \quad V_{\text{pp}} = 4 \text{ V} \quad \cdot \quad V_{\text{rms}} \\ &= 0.2 \text{ V} \end{aligned}$$

#### 2 Find the minimum sampling rate

$$f_s = 2 f_m = 2 \times 3.5 \text{ kHz}$$

$$f_s = 7 \text{ kSamples/sec}$$

### 3 Find how many bits per sample the channel can afford

Rearrange  $R_b = n f_s$  to get  $n$ :

$$n_{\max} = R_b / f_s = 50\,000 / 7\,000 = 7.142$$

Bits must be a whole number, and we must not exceed the channel rate, so round down:

$$n = 7 \text{ bits/sample}$$

### 4 Find the step size

$$\Delta = V_{pp} / 2^n = 4 / 2^7 = 4 / 128$$

$$\Delta = 0.03125 \text{ V}$$

### 5 Find SNR as a plain ratio

Here  $V_{\text{rms}}$  is given, so use the general form:

$$\begin{aligned} \text{SNR} &= V_{\text{rms}}^2 \div (\Delta^2/12) = (0.2)^2 \div ((0.03125)^2/12) \\ &= 0.04 \div (0.0009766/12) = 0.04 \div 0.00008138 \end{aligned}$$

$$\text{SNR} = 491.52$$

### 6 Convert to decibels

$$\text{SNR}_{\text{dB}} = 10 \log_{10}(491.52)$$

$$\text{SNR}_{\max} = 26.91 \text{ dB}$$

**Why is this only 27 dB when Question 1 gave 56 dB?** Because here the signal is quiet. A full-scale sine wave with  $V_{pp} = 4 \text{ V}$  would have  $V_{\text{rms}} = 2/\sqrt{2} \approx 1.414 \text{ V}$ , but this signal's  $V_{\text{rms}}$  is only  $0.2 \text{ V}$  — about seven times smaller. A quiet signal sits closer to the quantization noise floor, so the ratio is worse. This is exactly why real systems use companding ( $\mu$ -law and A-law).

## 2.7 Time Division Multiplexing, and the full-pipeline example

**TDM** means many signals share one wire by taking turns. Signal 1 sends a sample, then signal 2 sends a sample, then signal 3, and so on; after the last one it goes back to signal 1.

If you have  $N$  signals each sampled at  $f_s$ , the shared line must carry  $N \times f_s$  samples per second.

#### FORMULA – TDM AGGREGATE BIT RATE

$$R_b = N \times f_s \times n$$

$N$  = number of multiplexed channels ·  $f_s$  = samples/sec per channel ·  $n$  = bits/sample

The next example is the biggest in your materials. It chains together sampling, quantization, TDM and raised-cosine bandwidth (which is Module 3). Do it now, and revisit it after Module 3.

#### WORKED EXAMPLE – TUTORIAL 2, EXAMPLE 5 (TELEMETRY SYSTEM)

**Question.** In a certain telemetry system, eight message signals having 2 kHz bandwidth each are time-division multiplexed using a binary PCM technique. The error in sampling amplitude cannot be greater than 1 percent of the peak amplitude. Determine the minimum transmission bandwidth required if raised-cosine pulses with roll-off factor  $\alpha = 0.2$  are used. The sampling rate must be at least 25 percent above the Nyquist rate.

##### 1 Sampling rate per channel

$$\text{Nyquist rate} = 2B = 2 \times 2 \text{ kHz} = 4 \text{ kHz}$$

“At least 25% above Nyquist” means multiply by 1.25:

$$f_s = 1.25 \times 4 \text{ kHz}$$

$$f_s = 5 \text{ kHz per channel}$$

##### 2 Turn the 1% error rule into a number of levels

The largest rounding error is half a step, and it must stay under 1% of the peak amplitude  $A$ :

$$\Delta/2 \leq 0.01 A$$

For a uniform quantizer covering the range  $-A$  to  $+A$ , the full range is  $2A$ , so  $\Delta = 2A/L$ . Substituting:

$$2A / 2L \leq 0.01 A \Rightarrow A/L \leq 0.01 A \Rightarrow L \geq 100$$

### 3 Turn levels into bits

We need at least 100 levels, and levels must be a power of two.  $2^6 = 64$  is too few;  $2^7 = 128$  is enough:

$$n = \lceil \log_2 100 \rceil = 7 \quad (\text{that is, choose } L = 128 = 2^7)$$

$$n = 7 \text{ bits/sample}$$

### 4 Aggregate bit rate for all 8 channels

$$\text{Total samples/sec} = 8 \times 5\,000 = 40\,000$$

$$R_b = 40\,000 \times 7 = 280\,000 \text{ bits/s}$$

$$R_b = 280 \text{ kb/s}$$

### 5 Minimum bandwidth with raised-cosine pulses

The raised-cosine bandwidth rule (proved in Module 3) is:

$$B_T = (R_b/2)(1 + \alpha) = (280\text{k} / 2) \times (1 + 0.2) = 140\text{k} \times 1.2$$

$$B_T = 168 \text{ kHz}$$

**Note on notation.** Your PDF solves this twice, once using  $L$  and once using the symbol  $q$  for the number of levels, and writes the error rule as  $\epsilon_{\max} = \Delta/2 = m_p/q \leq 0.01 m_p$ , where  $m_p$  is the peak amplitude. Both routes give  $q \geq 100 \rightarrow q = 128 = 2^7 \rightarrow 7 \text{ bits}$ . Same answer, different letters. Do not let the change of symbol confuse you.

### EXAM TIP – THE STANDARD PCM ATTACK PLAN

Almost every PCM numerical is the same five moves in the same order. Write them down before you start:

1.  $f_s$  from  $f_m$  (watch for “25% above Nyquist” style twists)
2.  $n$  from  $L$ , or  $L$  from an accuracy requirement, or  $n$  from  $R_b/f_s$
3.  $\Delta$  from  $V_{pp}/2^n$
4.  $R_b = n f_s$  (multiply by  $N$  if it is TDM)
5. SNR — either  $6.02n + 1.76$ , or  $V_{rms}^2/(\Delta^2/12)$

Rounding rule: when  $n$  comes from a **channel rate limit**, round **down** (you cannot exceed the channel). When  $n$  comes from an **accuracy requirement**, round **up** (you must be at least that accurate).



## 03 FROM NINTH.PDF AND FIFTH.PDF — TUTORIAL 2

## Line codes and baseband signalling

You have bits. Now you must turn them into actual voltages on a wire. The way you do that is called a line code, and the choice matters more than you would guess.

### 3.1 Data element versus signal element

Two words get used constantly from here on, and they are *not* the same thing. Your ninth.pdf opens with them, so we will too.

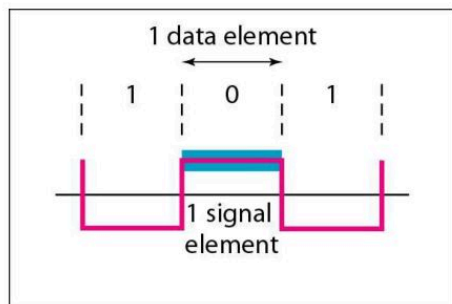
#### THE TWO DEFINITIONS

**Data element** — the smallest entity that can represent a piece of information. That is a **bit**.

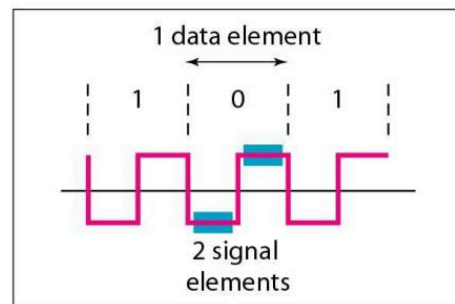
**Signal element** — the shortest unit (timewise) of a **digital signal**.

Or, in the words of your slide: **data elements are what we need to send; signal elements are what we can send** practically, in a form the receiver can synchronise to, identify and interpret.

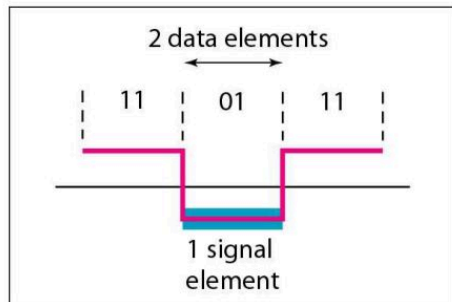
The link between them is a ratio called **r**: how many data elements ride on one signal element. It does not have to be 1.



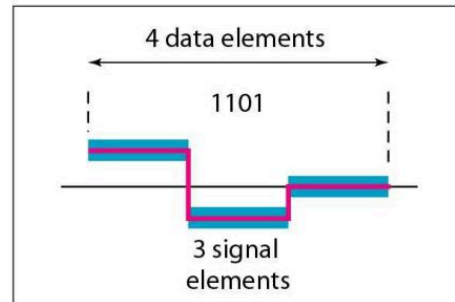
a. One data element per one signal element ( $r = 1$ )



b. One data element per two signal elements ( $r = \frac{1}{2}$ )



c. Two data elements per one signal element ( $r = 2$ )



d. Four data elements per three signal elements ( $r = \frac{4}{3}$ )

The four cases from your slide. (a) one bit per signal element,  $r = 1$ . (b) one bit takes two signal elements,  $r = \frac{1}{2}$ . (c) two bits share one signal element,  $r = 2$ . (d) four bits spread over three signal elements,  $r = \frac{4}{3}$ .

#### IN SIMPLE WORDS

Think of signal elements as **envelopes** and data elements as **letters**.

$r = 1$  — one letter per envelope. Simple.

$r = 2$  — two letters stuffed in one envelope. Fewer envelopes needed, so less bandwidth — but the envelope must be able to show four different things (00, 01, 10, 11), so the levels sit closer together and noise hurts more.

$r = \frac{1}{2}$  — one letter split across two envelopes. Wasteful of bandwidth, but the extra envelope gives you a guaranteed transition, which helps the receiver's clock. Manchester coding is exactly this.

**Bigger  $r$  is more bandwidth-efficient. Smaller  $r$  is more robust.** That trade-off runs through this whole module.

## 3.2 Data rate versus signal rate

Now put the two elements into *per second* form, and you get the two rates that students confuse most often.

### FORMULA – THE TWO RATES AND THE LINK BETWEEN THEM

**Data rate N** = number of data elements (bits) sent in 1 s – unit **bps**. Also called the **bit rate**.

**Signal rate S** = number of signal elements sent in 1 s – unit **baud**. Also called the **pulse rate**, **modulation rate** or **baud rate**.

$$S = c \times N \times (1/r) \quad \text{baud}$$

S = signal rate · c = the **case factor** · N = data rate (bps) · r = data elements per signal element

### IN SIMPLE WORDS – WHAT IS THAT MYSTERIOUS C?

The signal rate is not one fixed number, because it depends on the *data pattern*. Take NRZ-I: it only makes a transition when a 1 arrives. Send all zeros and the signal never changes — the worst case. Send alternating bits and it changes constantly — the best case.

So **c is a fudge factor between 0 and 1 that says which case you are in**. For an average, typical data stream we take  $c = \frac{1}{2}$ . That is the value your slides use, and the value the exam expects unless it says otherwise.

### WORKED EXAMPLE – NINTH.PDF, EXAMPLE 1

**Question.** A signal is carrying data in which one data element is encoded as one signal element ( $r = 1$ ). If the bit rate is 100 kbps, what is the average value of the baud rate if  $c$  is between 0 and 1?

#### 1 Take the average case factor

$c$  lies between 0 and 1, so the average value is

$$c = \frac{1}{2}$$

#### 2 Substitute into $S = c \times N \times (1/r)$

$$S = \frac{1}{2} \times 100\,000 \times (1/1) = 50\,000$$

$$S = 50 \text{ kbaud}$$

Read the meaning: 100 000 bits per second travel using only 50 000 signal changes per second, *on average*. Baud and bps are not the same number, and here they differ by a factor of two purely because of the averaging.

#### WORKED EXAMPLE – NINTH.PDF, EXAMPLE 2

**Question.** The maximum data rate of a channel is  $N_{\max} = 2 \times B \times \log_2 L$  (the Nyquist formula). Does this agree with the previous formula for  $N_{\max}$ ?

##### 1 Work out how many bits one level carries

A signal with  $L$  levels can carry  $\log_2 L$  bits per level. If each level corresponds to one signal element, then  $r = \log_2 L$ .

##### 2 Rearrange $S = cN(1/r)$ for $N$ , and use the average case $c = 1/2$

The most signal elements a bandwidth  $B$  can carry is  $S = B$ , so

$$N_{\max} = (1/c) \times B \times r = 2 \times B \times \log_2 L$$

**Yes – the two formulas agree exactly**

**Why this matters.** It shows that the general  $S = cN/r$  relationship and the famous Nyquist data-rate formula are the same statement wearing different clothes. The  $1/c = 2$  is where the “2” in  $2B \log_2 L$  comes from.

## 3.3 Why do we need line coding at all?

You could just put 0 V for a zero and 5 V for a one and be done. Your ninth.pdf spends four slides explaining why that naive choice breaks. Learn these four reasons — they are a standard exam question.

### Reason 1 — Baseline wandering

#### IN SIMPLE WORDS

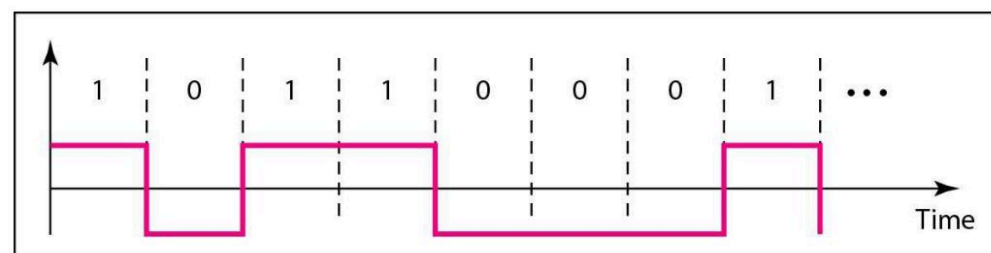
To decide whether an incoming voltage means 0 or 1, the receiver needs something to compare it against. So it keeps a **running average of the received signal power**. That average is called the **baseline**. Every incoming bit is judged against it.

Now send a long string of 1s. The running average creeps upward. Send a long string of 0s and it sags downward. This drift is called **baseline wandering**, and once the baseline has moved, the receiver starts judging bits against the wrong reference and makes mistakes.

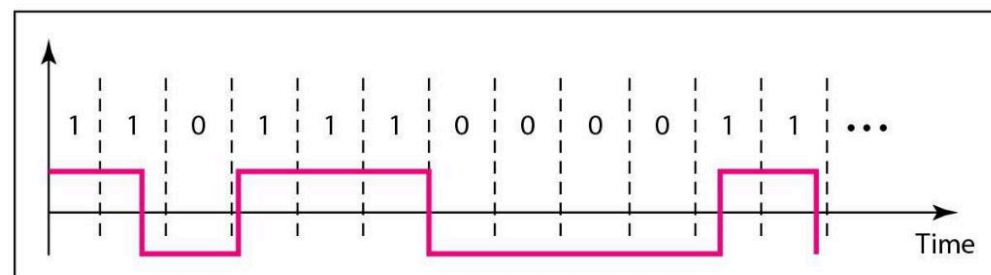
**A good line coding scheme must prevent baseline wandering.**

## Reason 2 — Lack of synchronization

If the receiver's clock does not tick at exactly the same rate as the sender's, it samples in the wrong places — and reads bits that were never sent.



a. Sent



b. Received

The effect of lack of synchronization, from your slide. (a) what was sent: 1 0 1 1 0 0 0 1. (b) what the receiver read, because its clock runs fast: it squeezes in extra samples and invents extra bits. The waveform is identical — only the sampling instants are wrong.

### WORKED EXAMPLE — NINTH.PDF, EXAMPLE 3

**Question.** In a digital transmission, the receiver clock is 0.1 percent faster than the sender clock. How many extra bits per second does the receiver receive if the data rate is 1 kbps? How many if the data rate is 1 Mbps?

#### 1 At 1 kbps

0.1% of 1000 is 1. So while 1000 bits are sent, the receiver clocks in 1001.

1000 bits sent → 1001 bits received

**1 extra bit per second**

## 2 At 1 Mbps

0.1% of 1 000 000 is 1000.

1 000 000 bits sent → 1 001 000 bits received

**1000 extra bits per second**

**The lesson.** The clock error is the same 0.1% in both cases, but the damage scales with speed. At 1 kbps you gain one junk bit a second; at 1 Mbps you gain a thousand. **The faster you go, the more precisely the clocks must agree** — which is exactly why high-speed links need self-synchronising line codes.

## Reason 3 — DC components

### IN SIMPLE WORDS

When the voltage in a digital signal stays constant for a while, Fourier analysis says the spectrum picks up **very low frequencies**. The frequencies right around zero are the **DC (direct-current) components**.

That is a problem, because many systems simply cannot pass low frequencies — particularly anything that uses **transformer coupling**, since a transformer only responds to *changing* current. Your slide gives the classic example: **a telephone line cannot pass frequencies below 200 Hz**. Any DC content you put into it is thrown away, and the signal arrives distorted.

So a good line code should have **no DC component** — its average voltage over time should be zero.

## Reason 4 — Self-synchronization

### IN SIMPLE WORDS

To interpret the signal correctly, **the receiver's bit intervals must line up exactly with the sender's**. If the receiver clock runs fast or slow, the intervals drift apart and bits get misread — as Example 3 just showed.

The fix is **self-synchronization**: build the timing information *into the data itself*. Your slide states the condition precisely — this works **if there are transitions in the signal that alert the receiver to the beginning, middle or end of the pulse**.

That single sentence is why Manchester coding exists. It guarantees a transition in the middle of *every* bit, so the receiver's clock is re-set every single bit period and can never drift.

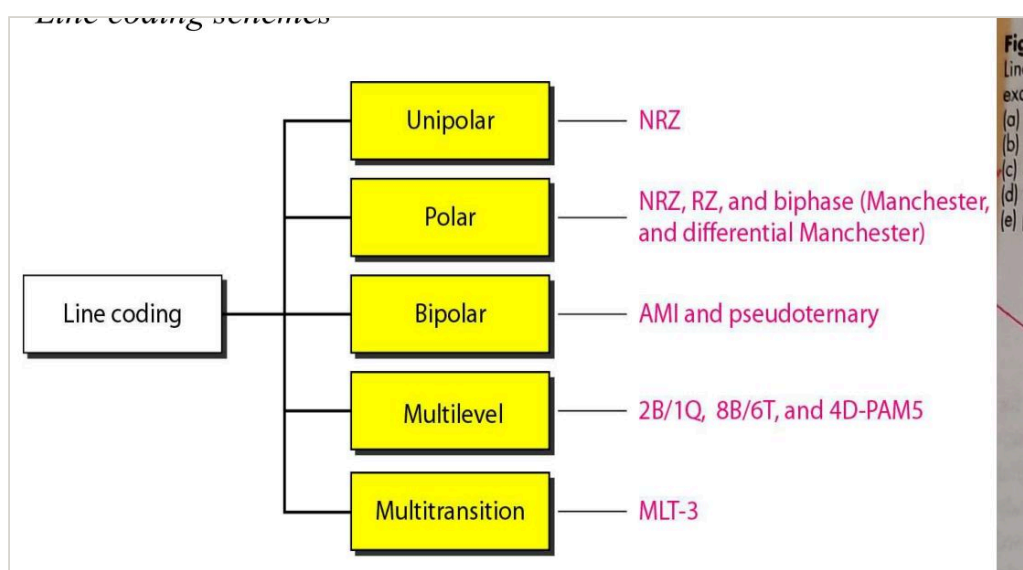
#### EXAM TIP – THE DESIRABLE PROPERTIES OF A LINE CODE

Tutorial 2 Example 3 asks exactly this. The answer is the four reasons above, turned into requirements:

1. **No DC component** (so it survives transformers and telephone lines)
2. **No baseline wandering** (so the decision threshold stays put)
3. **Self-synchronizing** — plenty of transitions for clock recovery
4. **Small bandwidth** (so it is cheap in spectrum)
5. **Built-in error detection** (a bonus that AMI gives free)
6. **Immunity to noise, and low complexity/cost**

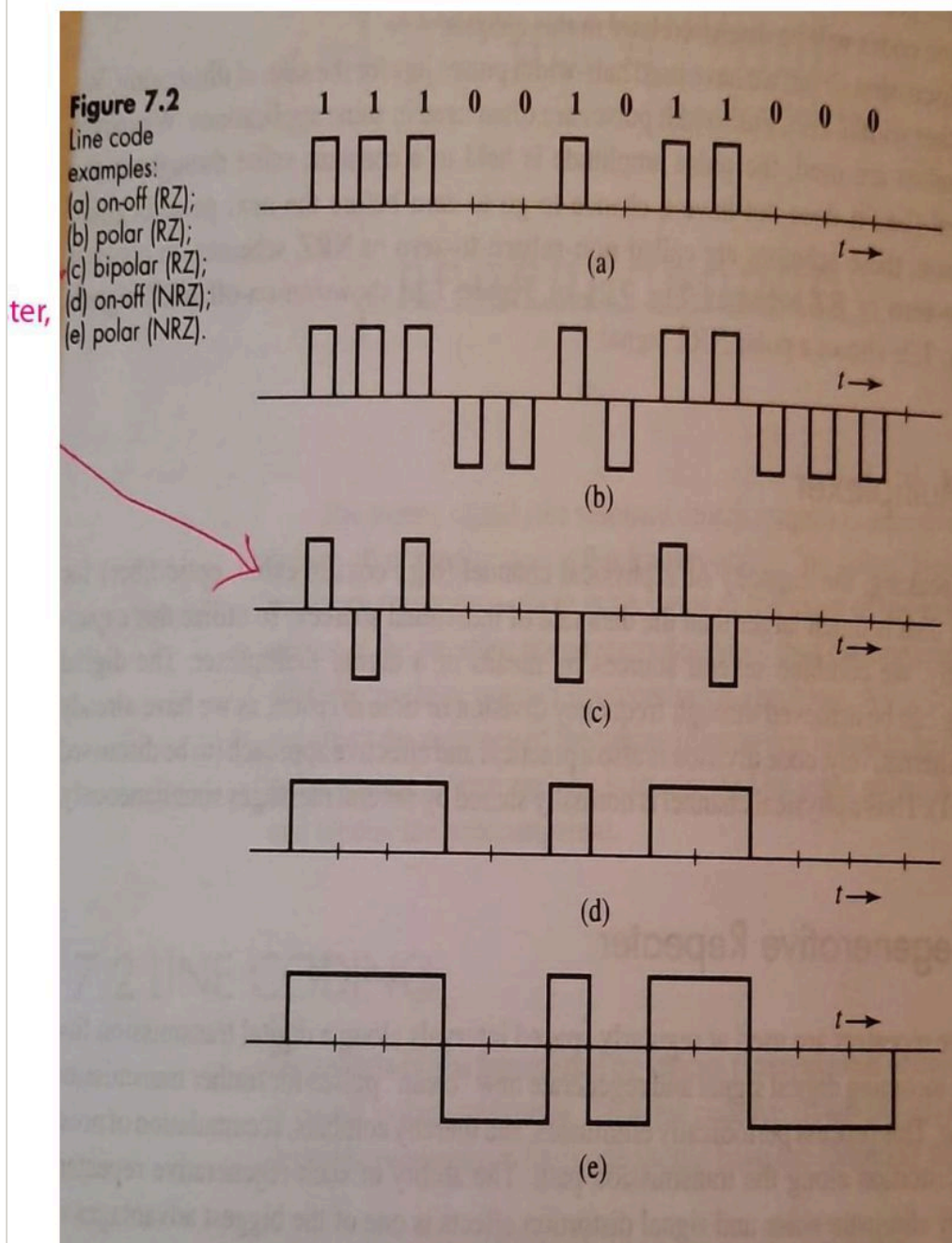
## 3.4 The family tree of line codes

Your slide organises every scheme into five families. Learn this tree — it is a very common short-answer question.



The line coding family tree from your slide.

| THE FIVE FAMILIES |                                                                           |                                                     |
|-------------------|---------------------------------------------------------------------------|-----------------------------------------------------|
| FAMILY            | SCHEMES                                                                   | LEVELS USED                                         |
| Unipolar          | NRZ                                                                       | 0 and +V (one polarity)                             |
| Polar             | NRZ (NRZ-L, NRZ-I), RZ, and biphase (Manchester, differential Manchester) | -V and +V                                           |
| Bipolar           | AMI and pseudoternary                                                     | -V, 0 and +V                                        |
| Multilevel        | 2B/1Q, 8B/6T, 4D-PAM5                                                     | many                                                |
| Multitransition   | MLT-3                                                                     | three, but the <i>transitions</i> carry the meaning |



The textbook figure reproduced on your slide, for the bit stream 1 1 1 0 0 1 0 1 1 0 0 0. (a) on-off (RZ), (b) polar (RZ), (c) bipolar (RZ), (d) on-off (NRZ), (e) polar (NRZ). "On-off" is another name for unipolar.

### 3.5 The line codes you must be able to draw

These are the exact rules from your Tutorial 2 and your ninth.pdf. Learn them as rules first, then draw.

| LINE CODE RULES         |                                                            |                                                                       |
|-------------------------|------------------------------------------------------------|-----------------------------------------------------------------------|
| CODE                    | LOGIC 1                                                    | LOGIC 0                                                               |
| Unipolar NRZ            | +A for the full bit                                        | 0 for the full bit                                                    |
| Unipolar RZ             | +A for half the bit,<br>then back to 0                     | 0 for the full bit                                                    |
| Polar NRZ-L             | the level says the bit: +A for one value, -A for the other | the opposite level                                                    |
| Polar NRZ-I             | <i>invert</i> the level at the start of the bit            | do <i>not</i> invert – hold the previous level                        |
| Polar RZ                | +A/2 for half the bit, then 0                              | -A/2 for half the bit, then 0                                         |
| Bipolar AMI             | alternates +A, -A, +A, -A ...                              | 0 (no signal)                                                         |
| Pseudoternary           | 0 (no signal)                                              | alternates +A, -A, +A ...                                             |
| Manchester              | transition in the <i>middle</i> , low→high                 | transition in the middle, high→low                                    |
| Differential Manchester | <i>no</i> transition at the start of the bit               | transition at the start of the bit – plus a mid-bit transition always |

#### IN SIMPLE WORDS – DECODING THE NAMES

**NRZ** = Non-Return to Zero. The voltage holds for the whole bit and does not come back to 0 in the middle.

**RZ** = Return to Zero. The voltage does its job for the first half of the bit, then returns to 0. This guarantees a transition in every 1-bit, which helps the clock — but the pulse is half as wide, so it needs twice the bandwidth.

**Unipolar** = one polarity (0 and +A). **Polar** = two polarities (-A and +A). **Bipolar** = three levels (-A, 0, +A).

**AMI** = Alternate Mark Inversion. “Mark” is old telegraph language for a 1. Every 1 flips the sign of the one before it: +A, -A, +A ... Zeros are skipped in this counting.

**Pseudoternary** is AMI with the roles of 0 and 1 swapped — nothing more.

**Manchester** — the rule is not about the level, it is about the *direction of the jump in the middle*. Every bit has a mid-bit transition, so the clock can never get lost.

**EXAM TIP — “L” VERSUS “I”, THE DIFFERENCE THAT MATTERS**

**NRZ-L is absolute; NRZ-I is differential.**

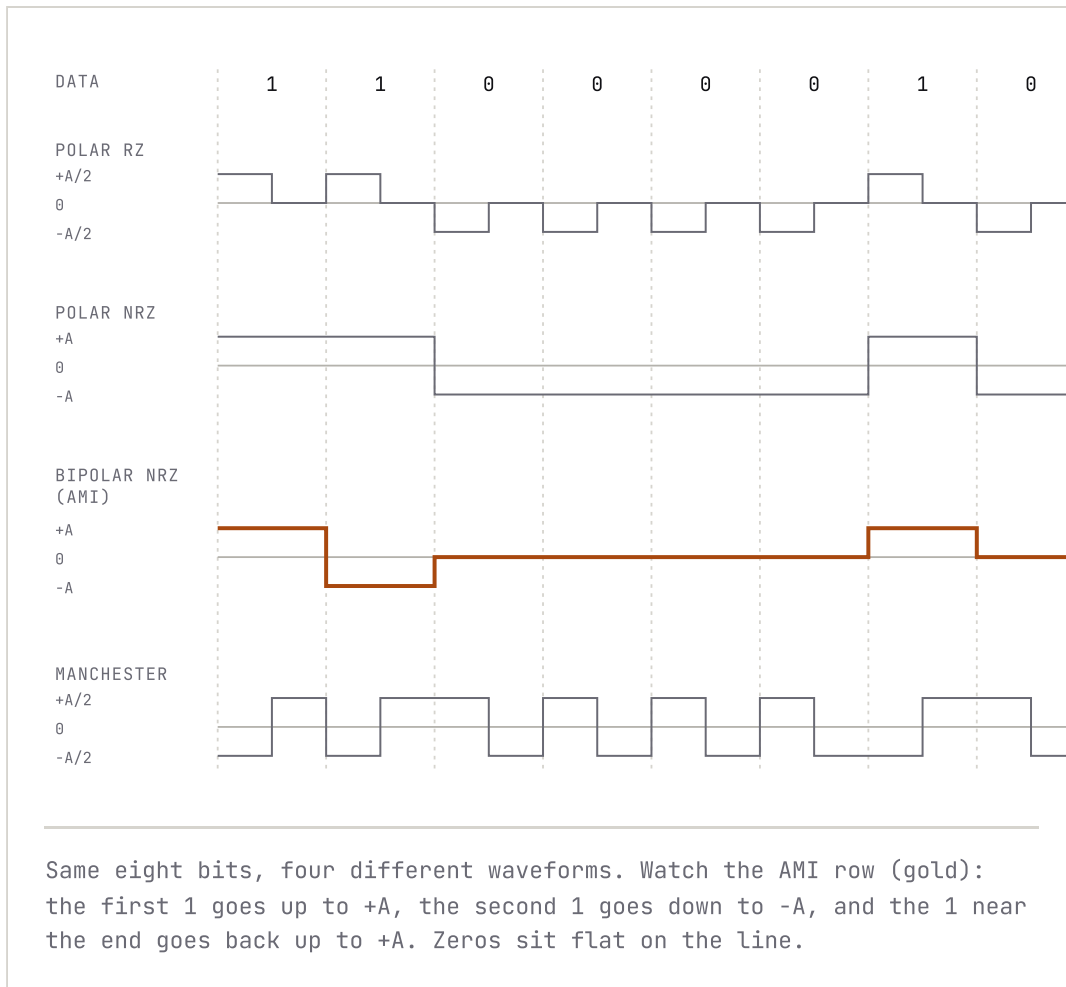
In **NRZ-L**, the voltage *level* tells you the bit directly. Simple — but if the two wires get swapped somewhere in the cable, every single bit comes out inverted and the whole message is wrong.

In **NRZ-I**, the bit is carried by *whether the level changed*, not by the level itself. Swap the wires and the *changes* are still in the same places, so the data still decodes correctly. That immunity to polarity reversal is why differential schemes are used in practice.

Differential Manchester works the same way, and adds the guaranteed mid-bit transition on top — so it is both polarity-proof *and* self-clocking. That is why it was chosen for Token Ring.

## 3.6 Tutorial 2, Example 1 — drawing the waveforms

**Question.** Encode the data stream 1 1 0 0 0 0 1 0 into RZ, NRZ, AMI and Manchester codes.

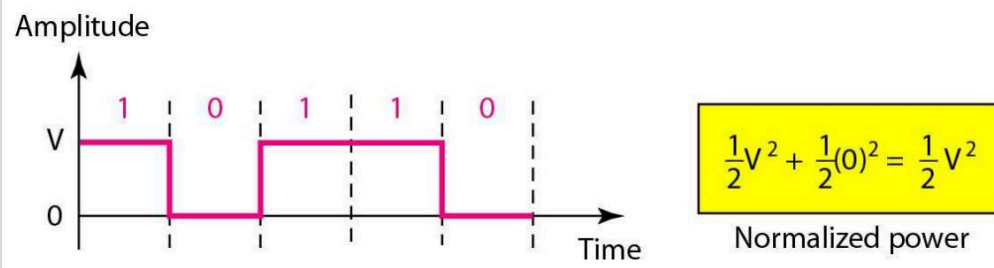


#### EXAM TIP – HOW TO DRAW THESE WITHOUT MISTAKES

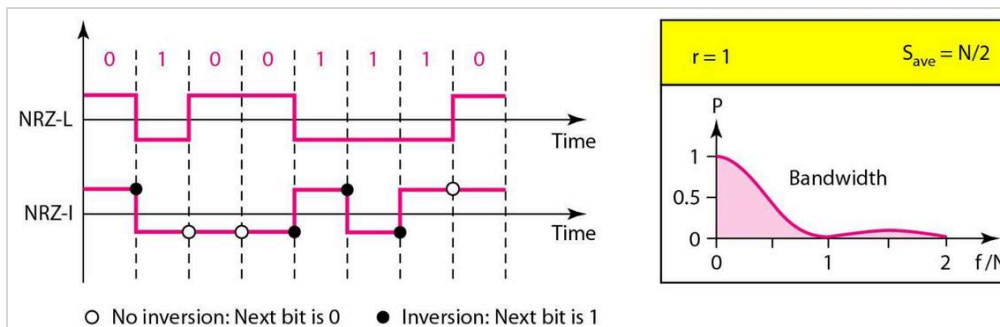
1. Draw the vertical dashed lines for the bit boundaries *first*, and write the bits above them.
2. For RZ codes, lightly mark the half-bit point too — that is where the pulse drops back to zero.
3. For AMI, go through the bits and pencil  $+ - + -$  above the 1s only, before you draw anything.
4. For Manchester, remember it is about the *middle jump direction*, not the level. Some textbooks use the opposite convention, so always state which one you are using.

## 3.7 Bandwidth and power of each scheme

Your ninth.pdf gives each scheme a spectrum box with two numbers: the ratio  $r$ , and the average signal rate  $S_{ave}$ . Together with the power spectrum plot, they tell you what the scheme costs.



Unipolar NRZ. Normalized power =  $\frac{1}{2}V^2 + \frac{1}{2}(0)^2 = \frac{1}{2}V^2$ . Half the time you transmit V and half the time nothing, so on average you burn half the power – but that non-zero average is a large DC component, which is unipolar NRZ's fatal flaw.



Polar NRZ-L and NRZ-I. Both have  $r = 1$  and  $S_{ave} = N/2$ . The spectrum on the right peaks at  $f = 0$ , which is the tell-tale sign of a strong DC component. Filled dots on the NRZ-I trace mark an inversion (next bit is 1); open dots mark no inversion (next bit is 0).

#### WORKED EXAMPLE – NINTH.PDF, EXAMPLE 4

**Question.** A system is using NRZ-I to transfer data. What are the average signal rate and minimum bandwidth?

##### 1 Average signal rate

For NRZ, the spectrum box gives

$$S_{ave} = N/2$$

##### 2 Minimum bandwidth

The minimum bandwidth needed equals the signal rate:

$$B_{min} = S$$

### 3 Put the numbers in ( $N = 1 \text{ Mbps}$ )

$$S = 1\,000\,000 / 2 = 500\,000$$

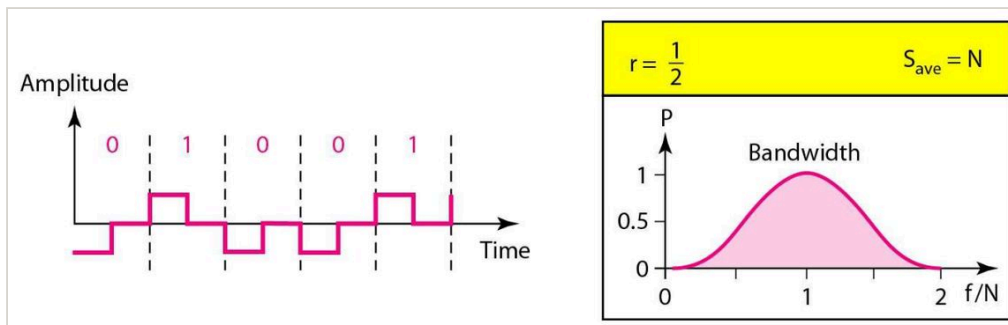
$$S = 500 \text{ kbaud} \quad \cdot \quad B_{\min} = 500 \text{ kHz}$$

#### CORRECTION – A SLIP IN NINTH.PDF, EXAMPLE 4

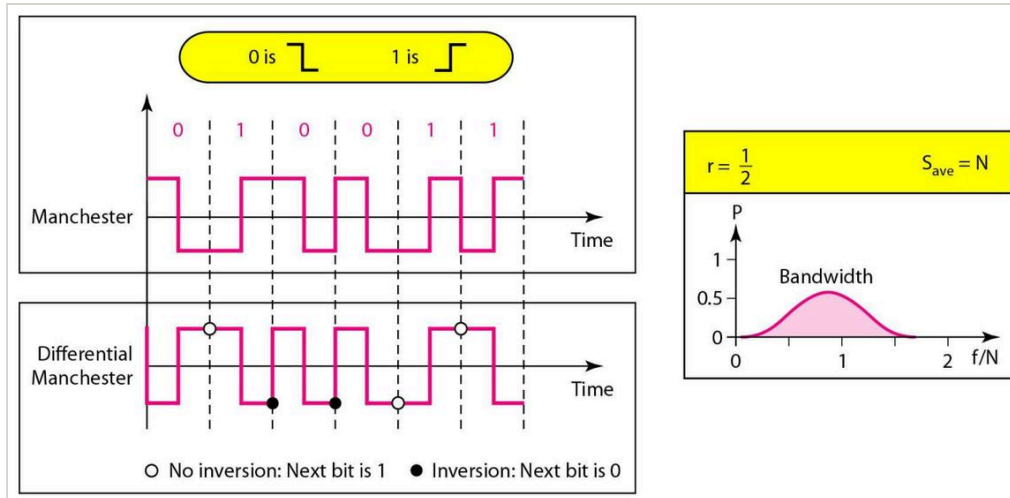
The slide states the data rate as **10 Mbps** but then prints the answer as **500 kbaud** and **500 kHz**. Those do not match: if  $N$  were really 10 Mbps, then  $N/2$  would be **5 Mbaud** and  $B_{\min}$  would be **5 MHz**.

The printed answers correspond to  **$N = 1 \text{ Mbps}$** , which is the value in the original textbook this example comes from. So the **method** on the slide is right; only the stated data rate is a typo.

**In the exam:** use whatever  $N$  the question gives you, and apply  $S = N/2$  then  $B_{\min} = S$ . If you are asked this exact question with 10 Mbps, the correct answers are 5 Mbaud and 5 MHz.

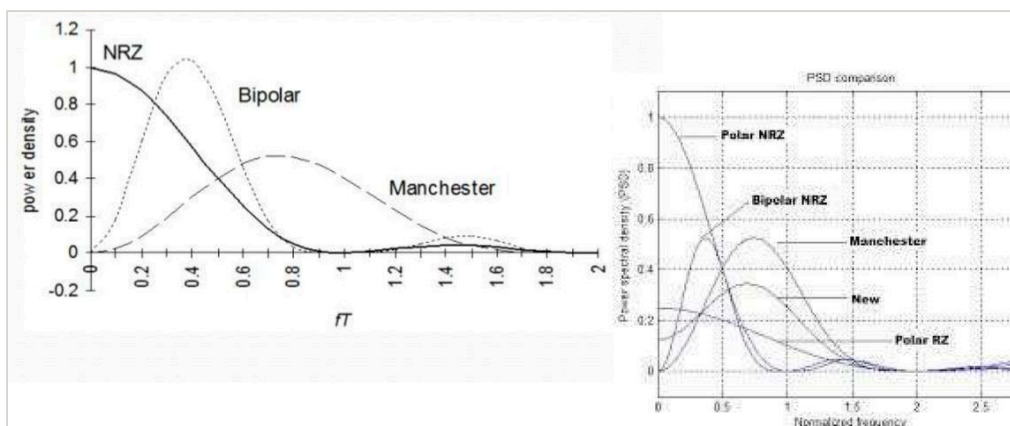


Polar RZ. Here  $r = \frac{1}{2}$  and  $S_{\text{ave}} = N$  – twice the signal rate of NRZ, so twice the bandwidth. In exchange, the spectrum is *zero at  $f = 0$* : no DC component, and a guaranteed transition in every bit.



Manchester and differential Manchester. Both have  $r = \frac{1}{2}$  and  $S_{ave} = N$ . Note the two conventions side by side: in Manchester the mid-bit jump direction carries the bit; in differential Manchester the mid-bit jump is always there for timing, and the bit is carried by whether there is also a jump at the start of the bit.

## Comparing the spectra



Power spectral density of the main line codes, on a normalized frequency scale ( $fT$ ). This one picture answers most "compare the line codes" questions.

### IN SIMPLE WORDS – HOW TO READ THAT PSD PLOT

**NRZ** starts at its *maximum* at  $f = 0$  and falls away. Big value at zero frequency = **big DC component**. Its first null is at  $fT = 1$ , so it is narrow — bandwidth-efficient but DC-heavy.

**Bipolar (AMI)** is *zero* at  $f = 0$  and humps up around  $fT \approx 0.5$ . Zero at DC is exactly what we wanted, and its bandwidth is no worse than NRZ. This is the best of both, which is why AMI was the workhorse of telephone trunk lines.

**Manchester** is also zero at DC but its hump sits further out, near  $f_T \approx 0.75$ , and stretches wider. So Manchester buys perfect clock recovery and no DC at the price of **roughly double the bandwidth**.

#### LINE CODE COMPARISON – THE SUMMARY TO MEMORISE

| SCHEME              | R             | $S_{AVE}$ | DC?        | SELF-CLOCKING? | BANDWIDTH |
|---------------------|---------------|-----------|------------|----------------|-----------|
| Unipolar NRZ        | 1             | $N/2$     | Yes, worst | No             | Narrow    |
| Polar NRZ-L / NRZ-I | 1             | $N/2$     | Yes        | No             | Narrow    |
| Polar RZ            | $\frac{1}{2}$ | N         | No         | Yes            | Double    |
| Bipolar AMI         | 1             | $N/2$     | No         | Only on 1s     | Narrow    |
| Manchester          | $\frac{1}{2}$ | N         | No         | Yes, always    | Double    |
| Diff. Manchester    | $\frac{1}{2}$ | N         | No         | Yes, always    | Double    |

#### EXAM TIP – AMI'S FREE ERROR DETECTION, AND ITS ONE WEAKNESS

Because AMI's 1s must strictly alternate  $+A, -A, +A, -A$ , the receiver can spot an error for free: **if two consecutive 1s have the same sign, something went wrong**. This is called a **bipolar violation**. No extra bits were spent to get it.

But AMI has one weakness. Zeros are sent as *nothing at all*, so a long run of zeros means a long silence — and the receiver's clock drifts. That is precisely the problem **scrambling** (section 3.8) was invented to solve.

## 3.8 Bit rate versus pulse rate, revisited

Section 3.2 gave the general relationship. Tutorial 2 asks you to apply it to two waveforms, so let us do it.

#### FORMULA – BIT RATE FROM PULSE RATE

**Pulse rate** =  $1 / (\text{duration of one pulse})$  pulses/sec,  
also called baud

**Bit rate** = **Pulse rate**  $\times \log_2 L$  bits/sec

$L$  = number of different voltage levels the pulse can take

#### IN SIMPLE WORDS

A **pulse** (or symbol, or signal element) is one blob of signal. The **pulse rate** counts how many blobs go past per second.

How many bits does one blob carry? That depends on how many different shapes a blob can have. Two levels  $\rightarrow$  1 bit ( $\log_2 2 = 1$ ). Four levels  $\rightarrow$  2 bits. Sixteen levels  $\rightarrow$  4 bits.

So **bit rate is always  $\geq$  pulse rate**. They are equal only for binary signalling. This is exactly why modern systems use 256-QAM: one symbol carries 8 bits instead of 1.

#### WORKED EXAMPLE – TUTORIAL 2, EXAMPLE 2

**Question.** For the signals shown in figures 13.5(a) and (b), determine the bit rate and pulse rate. The bit duration  $T_b = 0.5$  ms.

**Figure (a)** — two levels (0 or A), one pulse lasts exactly one bit.

##### 1 Pulse rate

$$\text{Pulse rate} = 1 / T_b = 1 / (0.5 \times 10^{-3})$$

**2000 pulses/sec**

##### 2 Bit rate

Two levels, so  $L = 2$  and  $\log_2 2 = 1$  :

$$\text{Bit rate} = 2000 \times 1$$

**2000 bps**

**Figure (b)** — multilevel ( $-A, -A/2, 0, +A/2, +A$ ), one pulse lasts *two* bit periods.

##### 1 Pulse duration and pulse rate

$$\text{pulse duration} = 2 T_b = 1 \text{ ms}$$

$$\text{Pulse rate} = 1 / (1 \times 10^{-3})$$

**1000 pulses/sec**

## 2 Bit rate

Here  $L = 4$ , so  $\log_2 4 = 2$  :

$$\text{Bit rate} = 1000 \times 2$$

**2000 bps**

**The lesson.** Both signals carry the *same* 2000 bits per second. But (b) does it with half as many pulses per second, by making each pulse carry 2 bits instead of 1. Half the pulse rate means roughly half the bandwidth. That is the whole idea behind multilevel signalling — and the price is that the levels sit closer together, so noise flips them more easily.

## 3.9 Scrambling

Section 3.7 left AMI with one unsolved problem: a long run of zeros means a long silence, and the clock drifts. Your ninth.pdf gives the fix.

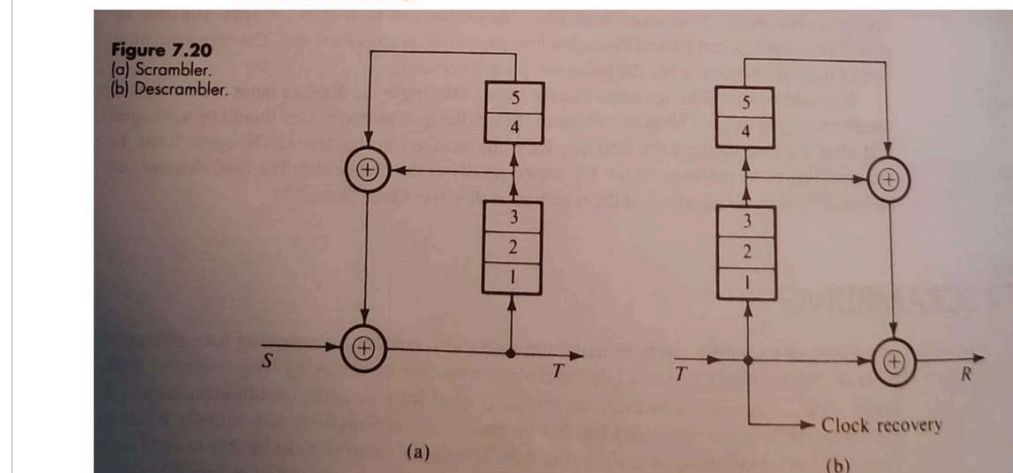
### THE PROBLEM AND THE IDEA

Your slide states it in three lines:

- A long sequence of 0s upsets the synchronization.
- If we can find a way to avoid a long sequence of 0s in the original stream, we can use bipolar AMI and others for long distances.
- One solution is called **scrambling**.

**Scrambling shuffles the data into a random-looking stream that has no long runs of zeros — without adding a single extra bit.** That last part is what makes it different from line coding and from channel coding.

ation is called **scrambling**.



(a) The scrambler and (b) the descrambler. Both are the same 5-stage shift register with XOR taps at stages 3 and 5. Notice the descrambler is the exact mirror of the scrambler – that is what makes the process reversible. Its output also drives clock recovery.

#### FORMULA – SCRAMBLER AND DESCRAMBLER EQUATIONS

Scrambler:  $S \oplus D^3T \oplus D^5T = T$

Rearranged:  $S = T \oplus (D^3 \oplus D^5)T = [1 \oplus (D^3 \oplus D^5)]T$   
 $= (1 \oplus F)T$

Descrambler:  $T \oplus FT = T \oplus (D^3 \oplus D^5)T = S$

$S$  = input data ·  $T$  = scrambled output ·  $\oplus$  = XOR ·  $D^n$  = “delayed by  $n$  bits” ·  $F = D^3 \oplus D^5$

#### IN SIMPLE WORDS

$D^n$  just means “the same stream, but  $n$  bits late”. So  $D^3T$  is the transmitted stream delayed by 3 bits — which is exactly what tapping stage 3 of the shift register gives you.

The scrambler XORs the incoming data with two delayed copies of its *own output*. That feedback is what makes the result look random.

And the beauty of XOR is that it undoes itself:  $x \oplus y \oplus y = x$ . So running the identical circuit at the far end recovers  $S$  exactly. **No key, no extra bits, no negotiation.**

## WORKED EXAMPLE – NINTH.PDF, EXAMPLE 7.2

**Question.** The data stream 101010100000111 is fed to the scrambler. Find the scrambler output T, assuming the initial contents of the registers are zero.

1 **Follow the feedback round and round**

Initially  $T = S$ . That S enters the register and comes back as  $(D^3 \oplus D^5)S = FS$ . That new sequence enters again and returns as  $F^2S$ , and so on. So

$$T = S \oplus FS \oplus F^2S \oplus F^3S \oplus \dots$$

$$= (1 \oplus F \oplus F^2 \oplus F^3 \oplus \dots) S$$

2 **Work out  $F^2$ , using the XOR cancellation trick**

$$F^2 = (D^3 \oplus D^5)(D^3 \oplus D^5) = D^6 \oplus D^{10} \oplus D^8 \oplus D^8$$

Modulo-2 addition of any sequence with itself is zero, so  $D^8 \oplus D^8 = 0$  :

$$F^2 = D^6 \oplus D^{10}$$

3 **And  $F^3$**

$$F^3 = (D^6 \oplus D^{10})(D^3 \oplus D^5) = D^9 \oplus D^{11} \oplus D^{13} \oplus D^{15}$$

4 **Collect all the delay terms**

$$T = (1 \oplus D^3 \oplus D^5 \oplus D^6 \oplus D^9 \oplus D^{10} \oplus D^{11} \oplus D^{12} \oplus D^{13} \oplus D^{15} \dots) S$$

### 5 Write out each delayed copy and XOR the columns

Because  $D^n S$  is simply  $S$  shifted right by  $n$  places:

|            |                                |
|------------|--------------------------------|
| $S$        | 101010100000111                |
| $D^3 S$    | 000101010100000111             |
| $D^5 S$    | 00000101010100000111           |
| $D^6 S$    | 000000101010100000111          |
| $D^9 S$    | 000000000101010100000111       |
| $D^{10} S$ | 0000000000101010100000111      |
| $D^{11} S$ | 00000000000101010100000111     |
| $D^{12} S$ | 000000000000101010100000111    |
| $D^{13} S$ | 0000000000000101010100000111   |
| $D^{15} S$ | 000000000000000101010100000111 |
| $T$        | 101110001101001                |

$$T = 101110001101001$$

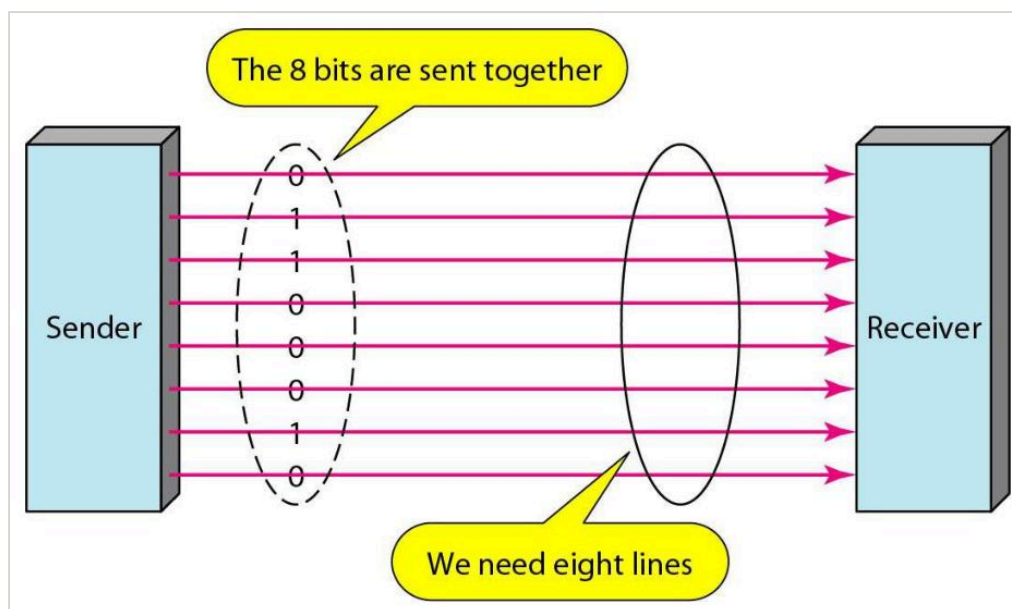
**Did it work?** Look at the two streams. The input 101010100000111 contains a run of **five zeros**. The output 101110001101001 has a longest run of only **three**. The clock now gets a transition often enough to stay locked — and not one extra bit was transmitted.

## 3.10 How data is actually sent: transmission modes

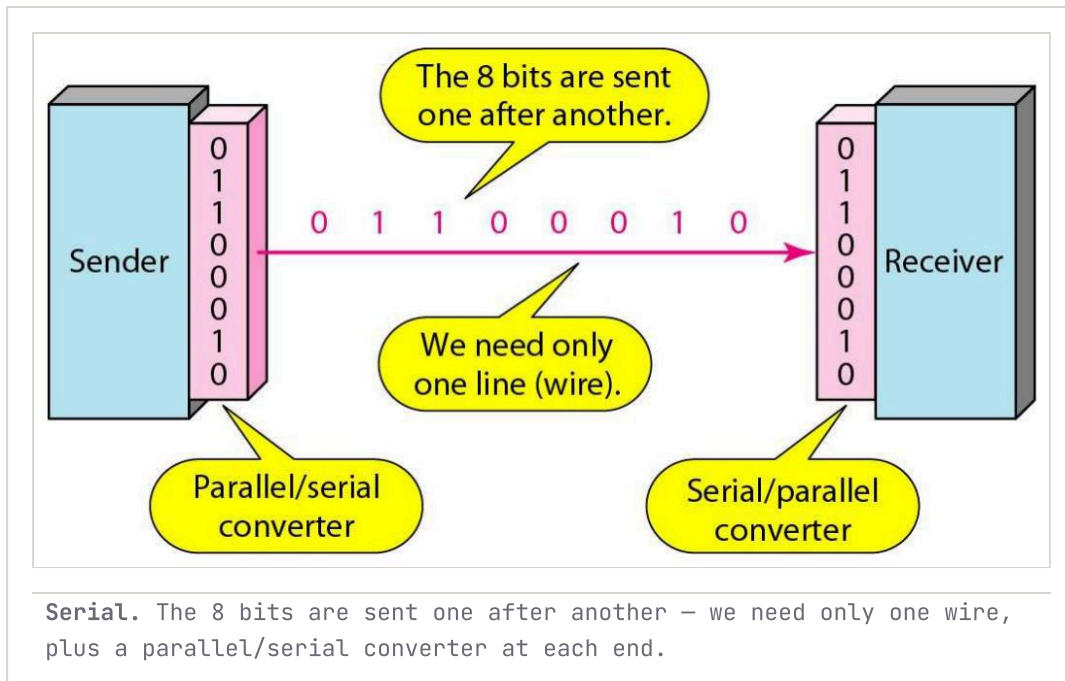
Your ninth.pdf closes this part with the practical question of how bits leave the machine.

## THE TRANSMISSION MODES TREE

| MODE            | SUB-MODE     | WHAT IT MEANS                                                                    |
|-----------------|--------------|----------------------------------------------------------------------------------|
| <b>Parallel</b> | –            | All bits of a group travel at once, on separate wires.                           |
| <b>Serial</b>   | Asynchronous | Each character framed with start and stop bits; gaps allowed between characters. |
|                 | Synchronous  | Continuous stream of frames, no gaps, no start/stop bits.                        |
|                 | Isochronous  | Fixed, guaranteed timing between frames<br>– needed for live audio and video.    |



**Parallel.** The 8 bits are sent together – but we need eight lines.



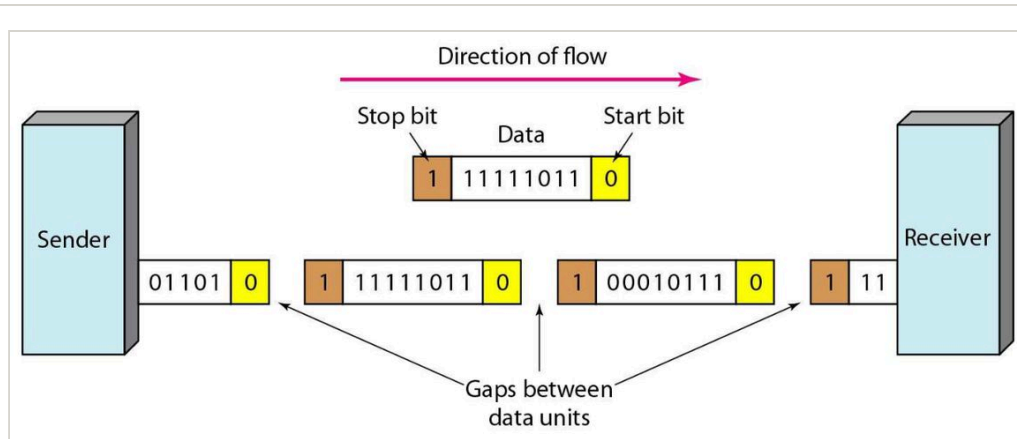
#### IN SIMPLE WORDS – WHY SERIAL WON

Parallel looks eight times faster, and over a few centimetres it is. But over any real distance it fails, for two reasons.

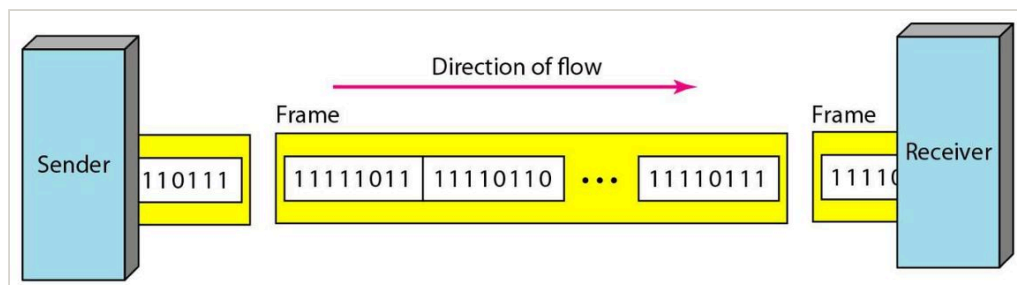
**Cost:** eight wires cost eight times as much as one.

**Skew:** no two wires are exactly the same length or have exactly the same delay. Over a long cable the eight bits that left together arrive at slightly different times, and the further and faster you go the worse it gets. A single wire has nothing to skew against.

That is why old parallel printer ports and IDE ribbon cables were replaced by USB, SATA and PCIe — all serial. Parallel survives only inside a chip or across a circuit board, where distances are tiny.



**Asynchronous.** Each character is wrapped in a **start bit (0)** and a **stop bit (1)**, and the gaps between data units can be any length. The start bit tells the receiver's clock exactly when to begin counting, so the two clocks only need to agree for the length of one character.



**Synchronous.** Bits flow as one continuous stream of frames – no start bits, no stop bits, no gaps. Nothing is wasted on framing every character, so it is far more efficient, but the receiver must hold its clock in step for the whole frame. This is where self-synchronising line codes and scrambling earn their keep.

#### EXAM TIP – ASYNCHRONOUS VS SYNCHRONOUS IN ONE LINE EACH

**Asynchronous:** cheap, simple, forgiving of clock error, but wastes 2 bits of overhead on every 8 bits of data (20% loss). Used for keyboards, old modems, simple serial ports.

**Synchronous:** no per-character overhead so much faster, but needs proper clock recovery. Used for everything high-speed.

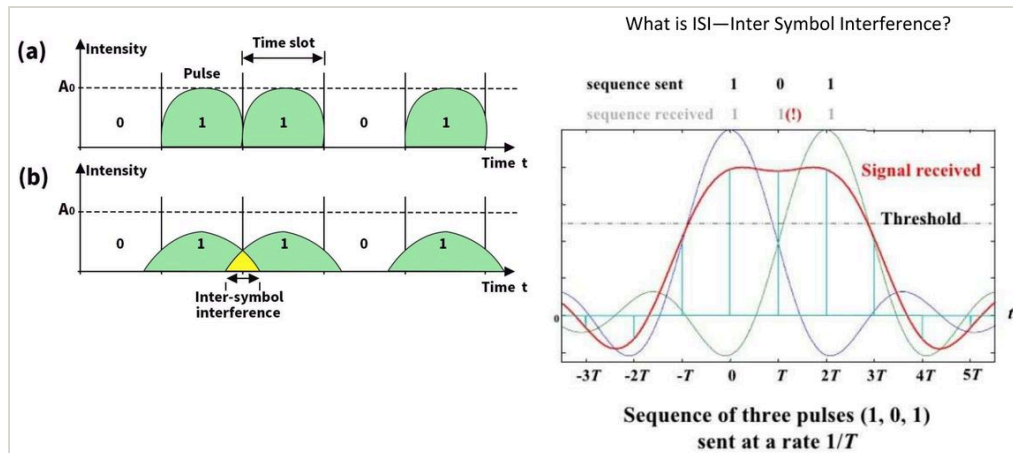
**Isochronous:** synchronous *plus* a guarantee about timing — frames arrive at a fixed rate with bounded delay. Needed for real-time audio and video, where arriving late is as bad as not arriving.

## Bandlimited channels, ISI and equalization

This is Unit 2 of your syllabus, in full. Every channel has limited bandwidth, and that single fact causes one specific disease — intersymbol interference. This module is the diagnosis, the prevention, and the cure.

### 4.1 What goes wrong on a bandlimited channel

Module 3 gave you neat rectangular pulses. Real channels do not pass rectangles. A rectangle contains frequencies stretching out to infinity, and a channel with bandwidth  $B$  chops all of them off above  $B$ . What comes out the far end is a **smeared, spread-out** version of what went in.



Left: (a) pulses that fit inside their own time slots decode cleanly; (b) pulses that spread wider than their slot overlap, and the shaded overlap is intersymbol interference. Right: the consequence. The sequence sent was 1 0 1, but the tails of the two 1s add up under the middle slot and push the sample above the threshold, so the receiver reads 1 1 1. The middle bit is wrong.

#### IN SIMPLE WORDS - WHAT ISI IS

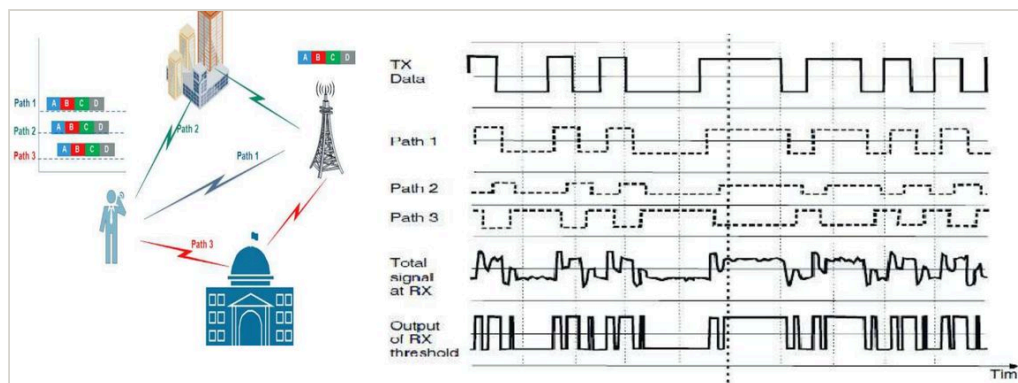
**ISI = Intersymbol Interference.** The tail of pulse number 1 is still hanging around when pulse number 2 is being read, so they add together and the receiver decides the wrong value.

Picture shouting a word in a large empty hall. The echo of your first word is still bouncing around when you say the second, and the listener hears a muddle. ISI is exactly that, in electrical form.

And notice something important from the figure: **ISI is not noise**. No random hiss was added. The signal interfered with *itself*, purely because the channel was too narrow. You cannot fix it by shouting louder.

## The second source of ISI: multipath

In a wireless channel there is a second, quite different cause of the same disease.



Multipath. The same packet reaches the tower by a direct path and by two reflected paths, each arriving at a slightly different time. The receiver adds them all up ("Total signal at RX"), and the bits it recovers ("Output of RX threshold") no longer match the transmitted data.

### IN SIMPLE WORDS

Your signal takes several routes to the receiver at once — one straight, others bouncing off buildings. The bounced copies travel further, so they arrive **late**. A late copy of symbol 1 lands on top of symbol 2. That is ISI again, arriving by a different door.

This is why ISI is the central problem of mobile communication, and why the whole of OFDM (Unit 5 of your syllabus) exists to defeat it.

## 4.2 The ISI equation

Now the mathematics. Your slide splits the received signal into exactly the pieces we care about.

FORMULA — THE RECEIVED SIGNAL, SPLIT INTO THREE PARTS

$$y(t) = \mu \sum_k a_k p(t - kT_b) + n(t)$$

Sampling at the  $i$ -th instant  $t_i$ :

$$y(t_i) = \mu a_i + \mu \sum_{k \neq i} a_k p[(i - k)T_b] + n(t_i)$$

$a_k$  = the  $k$ -th transmitted bit ·  $p(t)$  = the pulse shape ·  $T_b$   
 = bit period ·  $\mu$  = a scale factor ·  $n(t)$  = noise

#### IN SIMPLE WORDS — READ THE THREE TERMS

**Term 1,  $\mu a_i$**  — the contribution of the bit you actually wanted. This is the signal.

**Term 2, the sum with  $k \neq i$**  — the **ISI**. Your slide calls it “the residual effect of all other transmitted bits”. Every *other* bit contributes a little bit of its tail at this sampling instant. Note the condition  $k \neq i$  : we deliberately exclude the wanted bit and sum up everyone else.

**Term 3,  $n(t_i)$**  — the noise.

So the receiver's job is to read term 1 while terms 2 and 3 try to fool it. And here is the design goal: **we want to make term 2 exactly zero**. That is what the rest of this module achieves.

#### EXAM TIP — A LINE WORTH QUOTING

Your slide ends with a sentence examiners love:

*“When the signal-to-noise ratio is high, as is the case in a telephone system, the operation of the system is largely limited **by ISI rather than noise**.”*

In other words: on a good, quiet wire, noise is not your enemy — your own pulse tails are. Turning up the power does not help at all, because the ISI grows in exactly the same proportion as the signal.

## 4.3 Nyquist's three criteria

Harry Nyquist asked the key question: *the pulses are going to spread and overlap no matter what — so can we still read them correctly?* His answer is the foundation of this whole subject.

#### NYQUIST'S INSIGHT, FROM YOUR SLIDE

*“Pulse amplitudes can be detected correctly despite pulse spreading or overlapping, **if there is no ISI at the decision-making instants**.”*

Read that carefully, because it is cleverer than it first looks. Nyquist does **not** say “stop the pulses overlapping”. He accepts that they will overlap. He only insists that at the exact instants when the receiver looks, the interference happens to be **zero**.

Think of a room full of people talking. You do not need silence — you only need everyone else to happen to pause at the exact moment your friend says each word.

#### THE THREE CRITERIA

| CRITERION | CONDITION                                            | WHAT IT PROTECTS                                                   |
|-----------|------------------------------------------------------|--------------------------------------------------------------------|
| 1st       | At sampling points, no ISI                           | The pulse <b>centres</b> — so the amplitude you read is correct    |
| 2nd       | At threshold, no ISI                                 | The pulse <b>edges</b> (zero crossings) — so timing recovery works |
| 3rd       | Areas within the symbol period are zero, then no ISI | The <b>area</b> under the pulse — used by integrating receivers    |

## 4.4 The first Nyquist criterion — time domain

### FORMULA — THE FIRST NYQUIST CRITERION IN TIME

Let  $1/T$  be the sample rate. The pulse  $p(t)$  must satisfy

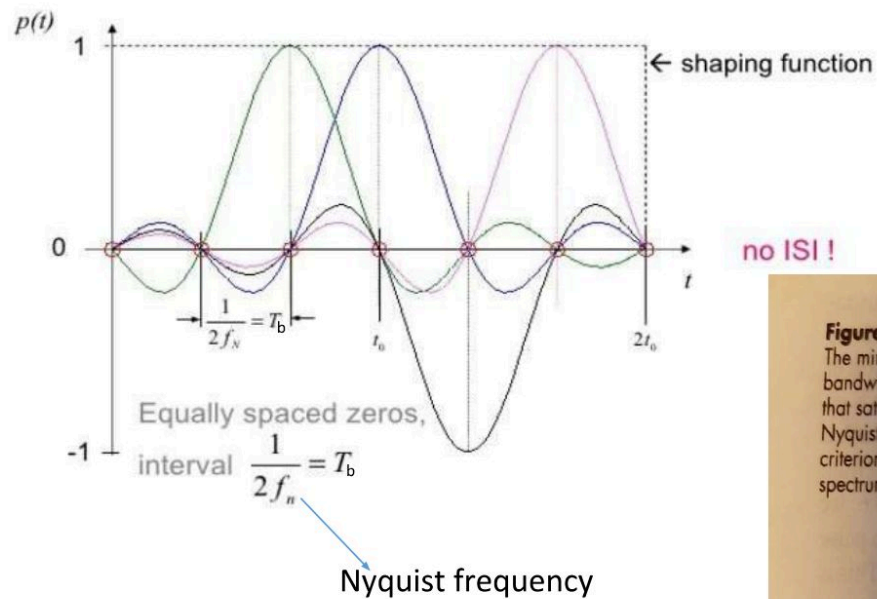
$$p(nT) = 1 \quad \text{when } n = 0$$

$$p(nT) = 0 \quad \text{when } n \neq 0$$

### IN SIMPLE WORDS

The pulse must be **tall (value 1) at its own sampling instant, and exactly zero at every other sampling instant**. Between those instants it can do whatever it likes — wander, ring, overshoot. Nobody is looking then.

$p(t)$ : impulse response of a transmission system (infinite length)

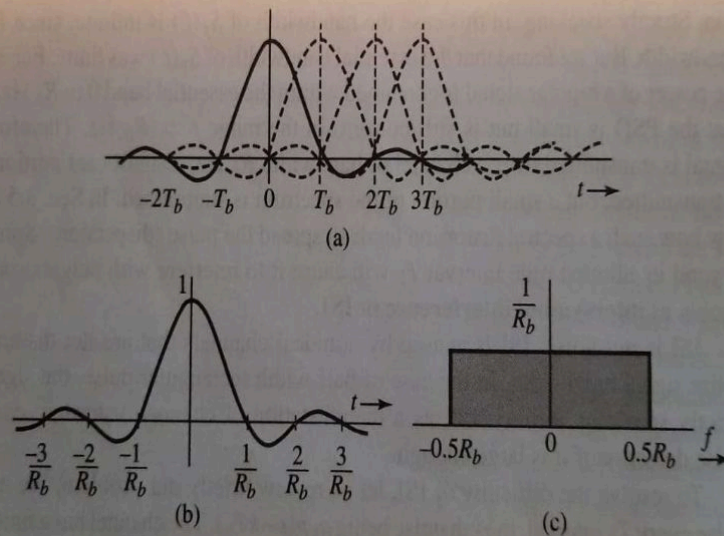


The first Nyquist criterion in the time domain.  $p(t)$  is the **shaping function** – the impulse response of the transmission system. Each coloured pulse peaks at its own instant and passes through zero at every other one, so the total interference at each sampling point is nil: **no ISI!** The zeros are **equally spaced** at intervals of  $1/(2f_N) = T_b$ , where  $f_N$  is the **Nyquist frequency**.

no ISI !

**Figure 7.11**

The minimum bandwidth pulse that satisfies Nyquist's first criterion and its spectrum.



The same idea from B. P. Lathi's book, reproduced on your slide: the minimum-bandwidth pulse satisfying Nyquist's first criterion, and its spectrum. (a) the shifted pulses overlapping; (b) one pulse alone, with zeros at  $1/R_b, 2/R_b, 3/R_b, \dots$ ; (c) its spectrum – a plain rectangle of height  $1/R_b$  stretching from  $-0.5R_b$  to  $+0.5R_b$ .

## 4.5 The first Nyquist criterion — frequency domain

The time-domain condition is easy to state but hard to check. Nyquist's real contribution was showing it has an exactly equivalent form in frequency, which *is* easy to check.

### FORMULA – THE FIRST NYQUIST CRITERION IN FREQUENCY

The **necessary and sufficient** condition for  $p(nT) = 1, 0, 0, 0, \dots$  is that its Fourier transform  $P(f)$  satisfies

$$\sum_m = -\infty^{\infty} P(f + m/T) = T$$

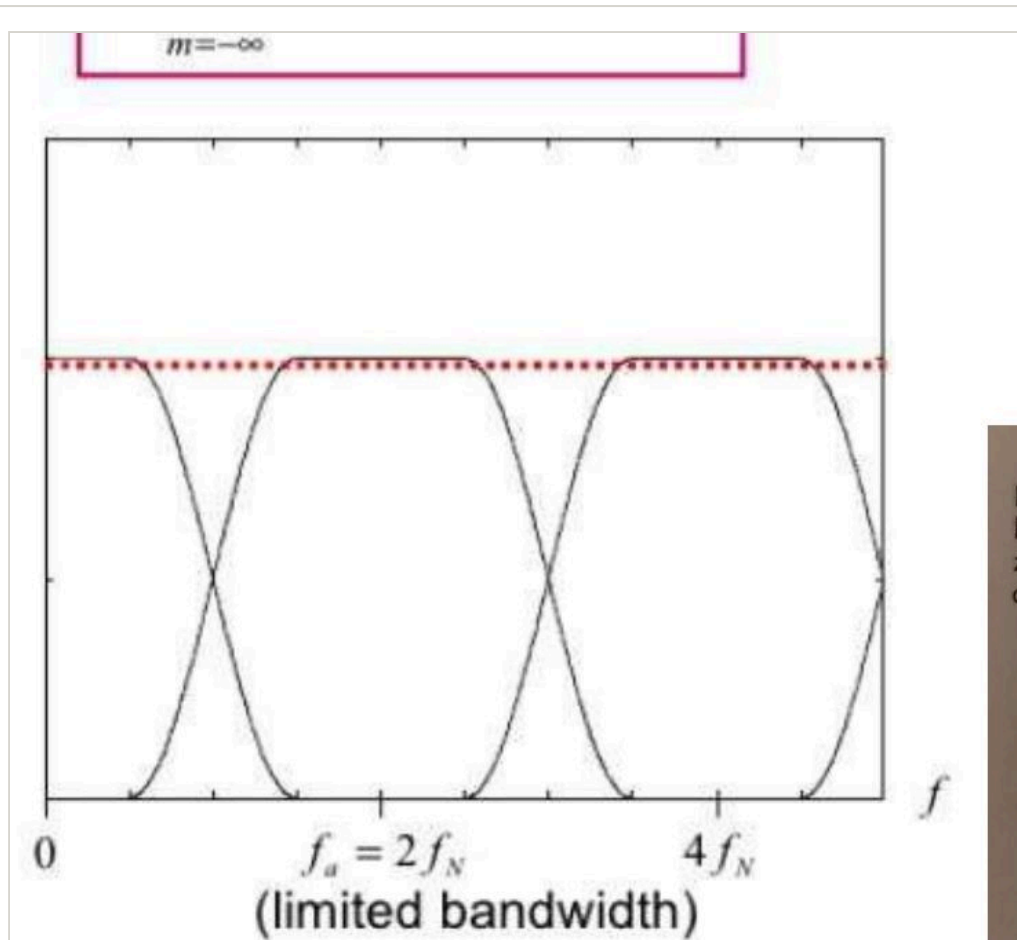
### IN SIMPLE WORDS – THE FOLDING TEST

This looks frightening. It is actually a recipe you could do with scissors:

1. Draw the pulse's spectrum  $P(f)$ .
2. Make copies of it, shifted left and right by the symbol rate  $1/T, 2/T$ , and so on.
3. Stack all the copies on top of each other and **add them up**.

4. If the total comes out as a **flat horizontal line** (a constant,  $T$ ), the pulse has zero ISI. If the total is bumpy, it does not.

The shifted copies must overlap in exactly such a way that wherever one is falling, the neighbour is rising by the same amount — so the sum stays level. That property is called having **odd symmetry about the band edge**, and it is the whole secret.



The folding test in a picture. The individual shifted copies rise and fall, but their sum (the red dotted line) is perfectly flat — so this pulse satisfies the first Nyquist criterion.

## 4.6 Sample rate versus bandwidth — the three cases

Now the practical question: given a channel of bandwidth  $W$ , how fast can I signal? Your slide gives three cases, and the answer is different in each.

THE THREE CASES, WHERE  $W$  IS THE BANDWIDTH OF  $P(f)$

| CASE       | WHAT HAPPENS                                                                                              |
|------------|-----------------------------------------------------------------------------------------------------------|
| $1/T > 2W$ | No function can satisfy the Nyquist condition. Zero-ISI signalling is impossible.                         |
| $1/T = 2W$ | Exactly one solution: the <b>rectangular</b> spectrum (a sinc pulse in time). The minimum-bandwidth case. |
| $1/T < 2W$ | Many solutions exist. "A typical one is the <b>raised cosine</b> function."                               |

#### FORMULA - THE $1/T = 2W$ SOLUTION

$$p(t) = \sin(\pi t/T) / (\pi t) = \text{sinc}(\pi t/T)$$

$$P(f) = T \text{ for } |f| < W, \quad 0 \text{ otherwise} \quad (\text{a perfect rectangle})$$

- Slightly notation different from the book. But it is the same

$$p_{rc0}(t) = \frac{\sin(\pi \frac{t}{T})}{\pi \frac{t}{T}} \cdot \frac{\cos(r\pi \frac{t}{T})}{1 - (2r \frac{t}{T})^2}$$

$$r : \text{rolloff factor} \quad 0 \leq r \leq 1$$

$$P_{rc0}(j2\pi f) = \begin{cases} 1 & f \leq (1-r) \frac{1}{2T} \\ \frac{1}{2} \left[ 1 + \cos\left(\frac{\pi}{2r} \left(\frac{\pi f}{T} + r - 1\right)\right) \right] & \text{if } \frac{1}{2T}(1-r) \leq f \leq \frac{1}{2T}(1+r) \\ 0 & f \geq \frac{1}{2T}(1+r) \end{cases}$$

The spectrum  $P(f)$  with its shifted copies at  $\pm 1/T$ . Whether the copies leave a gap, touch exactly, or overlap is what separates the three cases above.

#### IN SIMPLE WORDS - WHY THE PERFECT CASE IS USELESS

Case 2 sounds ideal: minimum possible bandwidth, exactly one right answer. But the sinc pulse has two fatal practical problems.

**It needs a brick-wall filter.** A spectrum that is flat up to  $W$  and then drops instantly to zero cannot be built with real components. Real filters roll off gradually.

**Its tails die away far too slowly.** The sinc decays only as  $1/t$ , so a pulse from twenty symbols ago is still contributing. If your sampling clock is off by even a hair, all those slowly-dying tails no longer cancel, and the ISI is catastrophic.

So we accept case 3 — use a bit more bandwidth than the theoretical minimum, in exchange for a filter that can actually be built and tails that die quickly. That compromise pulse is the **raised cosine**.

## 4.7 The raised cosine filter

### FORMULA — THE RAISED COSINE PULSE

**Time domain** (a sinc multiplied by a cosine correction):

$$p_{rc}(t) = \left[ \sin(\pi t/T) / (\pi t/T) \right] \times \left[ \cos(r\pi t/T) / (1 - (2rt/T)^2) \right]$$

**Frequency domain**, with  $r$  the roll-off factor,  $0 \leq r \leq 1$ :

$$P(f) = 1 \quad \text{for } f \leq (1-r)/(2T)$$

$$P(f) = \frac{1}{2} [1 + \cos((\pi/2r)(\pi f/T + r - 1))] \quad \text{for } (1-r)/(2T) \leq f \leq (1+r)/(2T)$$

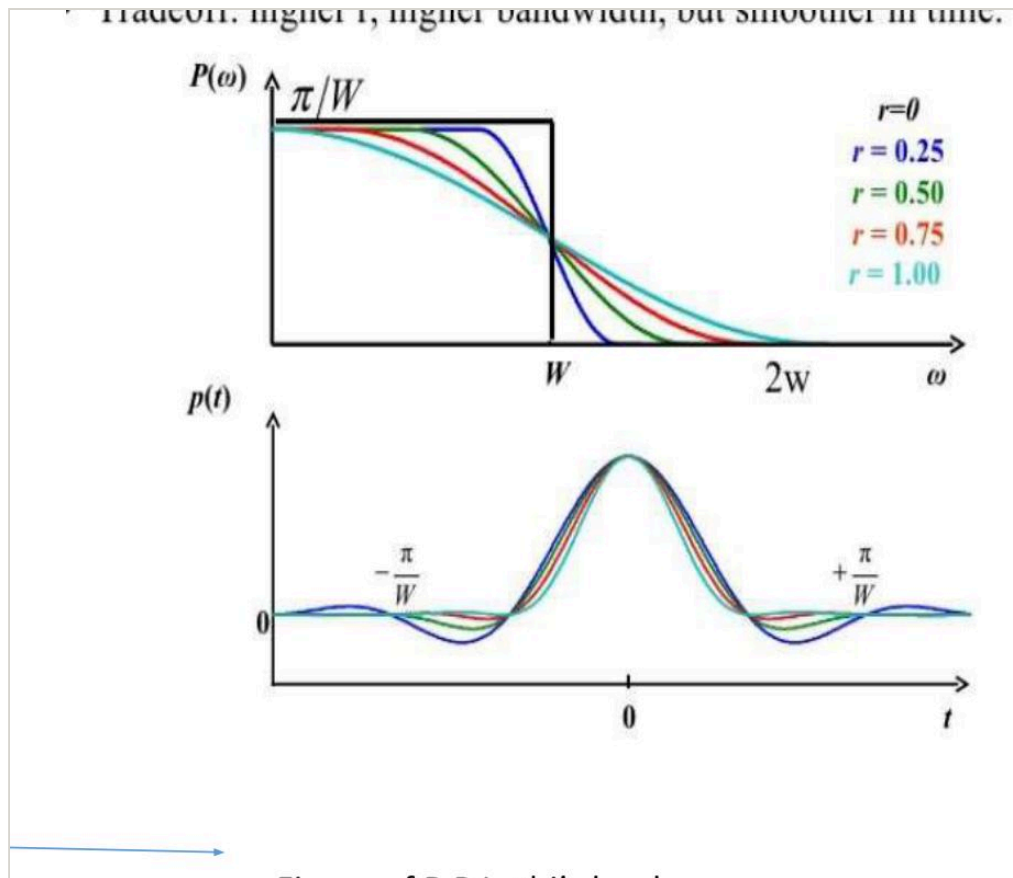
$$P(f) = 0 \quad \text{for } f \geq (1+r)/(2T)$$

### IN SIMPLE WORDS

Do not be scared by the three-part definition. It is just describing a shape with three regions: **flat on top, a smooth cosine-shaped slope down the sides, then zero**. That is literally why it is called a “raised cosine” — a cosine curve lifted up so it sits above zero.

The time-domain formula is the sinc pulse from case 2, **multiplied by an extra factor** that squashes the tails down quickly. You keep the zero-crossings that give you zero ISI, but you lose the slowly-dying tails that made the sinc so fragile.

**Note on symbols:** your ninth.pdf writes the roll-off factor as  $r$ . Your Tutorial 2 writes the same thing as  $\alpha$ . They are identical — do not let the change of letter confuse you.



Raised cosine shaping, for  $r = 0, 0.25, 0.50, 0.75$  and  $1.00$ . Your slide's summary: "Tradeoff: higher  $r$ , higher bandwidth, but smoother in time." Top: as  $r$  rises the spectrum spreads from  $W$  out towards  $2W$ . Bottom: as  $r$  rises the time-domain side lobes shrink, so the pulse is far more forgiving of timing error.

#### FORMULA - RAISED COSINE BANDWIDTH (THE EXAM VERSION)

$$f_B = (1 + \alpha) / (2 T_b)$$

Equivalently, since  $R_b = 1/T_b$ :  $B_T = (R_b/2)(1 + \alpha)$

#### IN SIMPLE WORDS

$R_b/2$  is the absolute theoretical minimum — the **Nyquist bandwidth**. You can never do better.

The factor  $(1 + \alpha)$  is the **penalty** you pay above that minimum:

- $\alpha = 0 \rightarrow$  bandwidth  $= R_b/2$ . The ideal brick wall. Impossible to build, fragile.
- $\alpha = 1 \rightarrow$  bandwidth  $= R_b$ . Double the minimum. Wasteful, but very forgiving.
- $\alpha = 0.2$  to  $0.5 \rightarrow$  the practical compromise everybody actually uses.

#### WORKED EXAMPLE – TUTORIAL 2, EXAMPLE 4

**Question.** A communication channel of bandwidth 75 kHz is required to transmit binary data at a rate of 0.1 Mb/s using raised-cosine pulses. Determine the roll-off factor  $\alpha$ .

##### 1 Find the bit duration

$$T_b = 1 / R_b = 1 / (0.1 \times 10^6) = 10^{-5} \text{ s}$$

##### 2 Write down the given bandwidth

$$f_B = 75 \text{ kHz} = 75 \times 10^3 \text{ Hz}$$

##### 3 Rearrange for $(1 + \alpha)$

From  $f_B = (1 + \alpha) / (2T_b)$ , multiply both sides by  $2T_b$ :

$$1 + \alpha = 2 f_B T_b = 2 \times (75 \times 10^3) \times 10^{-5} = 1.5$$

##### 4 Solve for $\alpha$

$$\alpha = 1.5 - 1$$

$$\alpha = 0.5$$

**Sanity check.** The Nyquist minimum for 0.1 Mb/s would be 50 kHz. We were given 75 kHz, which is 1.5 times the minimum — so  $\alpha = 0.5$ . The answer agrees with common sense.

#### EXAM TIP – AND DON'T FORGET TUTORIAL 2 EXAMPLE 5

The three raised-cosine question types are all the same equation rearranged:

- Given  $R_b$  and  $\alpha \rightarrow$  find  $B_T$ . (*Tutorial 2 Example 5, solved in Module 2*)
- Given  $R_b$  and  $B_T \rightarrow$  find  $\alpha$ . (*Example 4 above*)
- Given  $B_T$  and  $\alpha \rightarrow$  find the maximum  $R_b$ .

The full telemetry problem — eight TDM channels, 1% accuracy,  $\alpha = 0.2$ , answer 168 kHz — is worked step by step in **Module 2, section 2.7**. It chains sampling,

quantization, TDM and this bandwidth formula into one question, so it is the single best revision problem in your materials.

## 4.8 Bandwidth efficiency

Now we can answer the practical question: how many bits per second do I get per hertz of spectrum?

### FORMULA – TRANSMISSION (BANDWIDTH) EFFICIENCY

$$\beta_{rc} = \text{data rate} / \text{bandwidth} = (1/T) / [(1+r)/2T] = 2/(1+r) \quad \text{bit/s per Hz}$$

Rearranged for design:

$$R_b = 2 B_t / (1 + r) \quad \text{and} \quad B_t = R_b(1 + r) / 2$$

$$\text{Range:} \quad 1 \text{ bit/s/Hz} \leq 2/(1+r) < 2 \text{ bit/s/Hz}$$

### IN SIMPLE WORDS – THE TWO ENDS OF THE RANGE

$r = 0$  gives  $2/(1+0) = 2$  bits/s/Hz. The theoretical best for binary signalling. Requires the impossible brick-wall filter.

$r = 1$  gives  $2/(1+1) = 1$  bit/s/Hz. The worst efficiency, but the easiest filter to build — and, as your slide notes, this is the case that also satisfies the **2nd Nyquist criterion**.

So the whole design space for a raised-cosine system lives between **1 and 2 bits per second per hertz**, and choosing  $r$  is choosing where on that line you sit.

## 4.9 The second Nyquist criterion

The first criterion protected the pulse *centres*. The second protects the pulse *edges*.

### FORMULA – THE SECOND NYQUIST CRITERION

Values at the pulse edge are distortionless:

$$p(t) = 0.5 \quad \text{when} \quad t = -T/2 \quad \text{or} \quad T/2$$

$$p(t) = 0 \text{ when } t = (2k-1)T/2, \text{ for } k \neq 0, 1$$

In frequency, over  $-1/T \leq f \leq 1/T$ :

$$P_r(f) = \text{Re} \left[ \sum_n (-1)^n P(f + n/T) \right] = T \cos(fT/2)$$

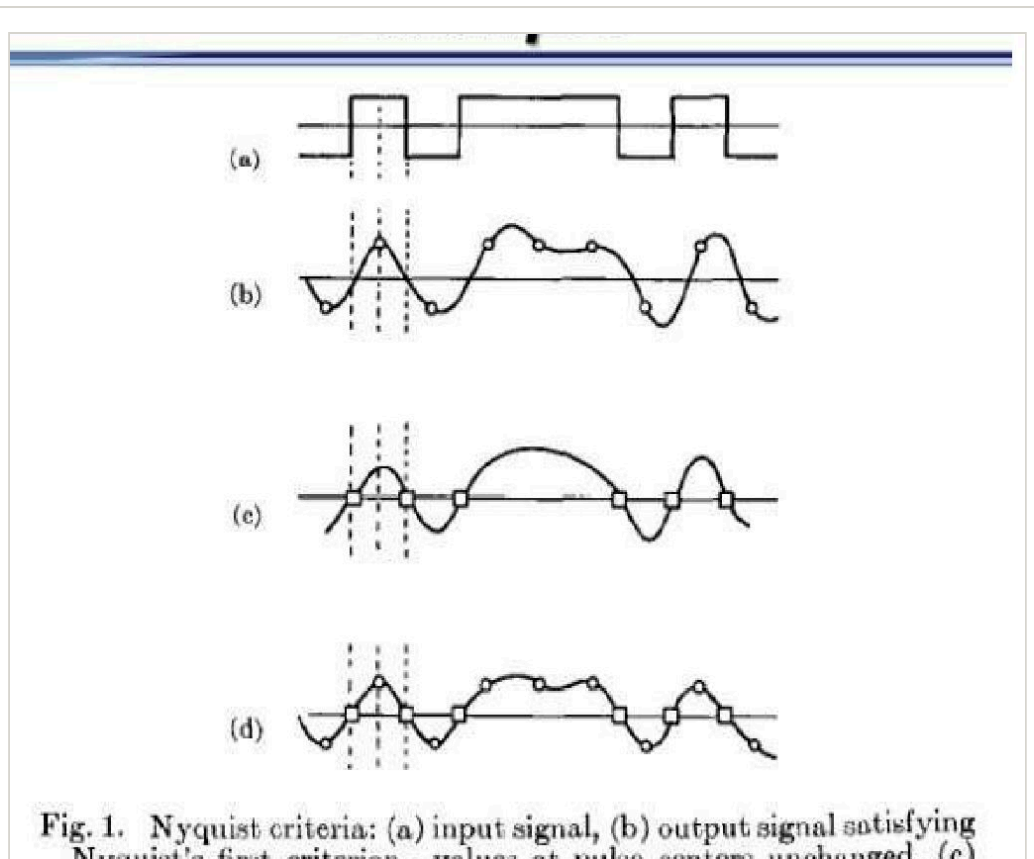
$$P_i(f) = \text{Im} \left[ \sum_n (-1)^n P(f + n/T) \right] = 0$$

#### IN SIMPLE WORDS — WHY ANYONE CARES ABOUT THE EDGES

The receiver does not only need to know *what* the value is — it needs to know *when* to look. It works out the timing from the **zero crossings** of the received waveform, because those are the sharpest, most identifiable features.

If ISI moves those crossings around, the recovered clock jitters, and a jittery clock samples in the wrong place — which undoes all the work the first criterion did.

So: **1st criterion** → **correct amplitudes**. **2nd criterion** → **correct timing**. A system that satisfies both is clean in value *and* in time.



The four cases side by side. (a) the input signal. (b) output satisfying Nyquist's **first** criterion — values at pulse *centres* unchanged (circles). (c) output satisfying the **second** criterion — values at pulse *edges* unchanged (squares). (d) output satisfying **both** — centres and edges both unchanged.

## 4.10 The eye diagram

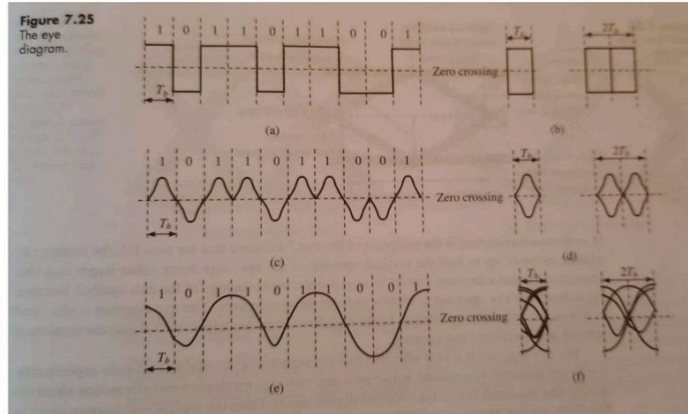
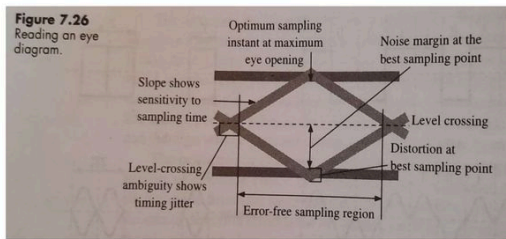
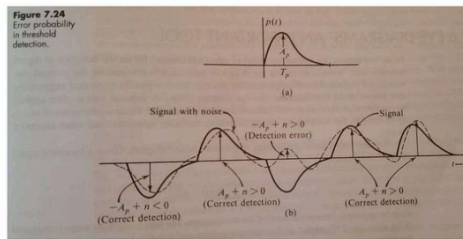
Everything above is theory. The eye diagram is how an engineer sees ISI on an oscilloscope in about two seconds.

### IN SIMPLE WORDS — HOW AN EYE DIAGRAM IS MADE

Take the received waveform. Chop it into pieces exactly one symbol period long. Now lay every piece on top of every other piece, all starting at the same point.

If the channel is clean, every piece follows nearly the same path, so the overlaid picture has a big empty gap in the middle — it looks like an **eye**. If there is ISI, the pieces take many different paths, the traces spread out, and the eye **closes**.

On a real oscilloscope you do not chop anything: you just trigger the scope at the symbol rate and let the traces pile up by themselves.



Three figures from your slide. **Top left (7.24)**: where errors come from – noise added to the clean signal is usually harmless, but occasionally pushes a sample across the threshold, giving a detection error. **Bottom (7.25)**: how the eye is built – rectangular pulses fold into a rectangular eye, rounded pulses into a diamond eye, smeared pulses into a narrow crossed eye. **Top right (7.26)**: how to read one.

#### THE SIX LABELS ON AN EYE DIAGRAM – LEARN ALL OF THEM

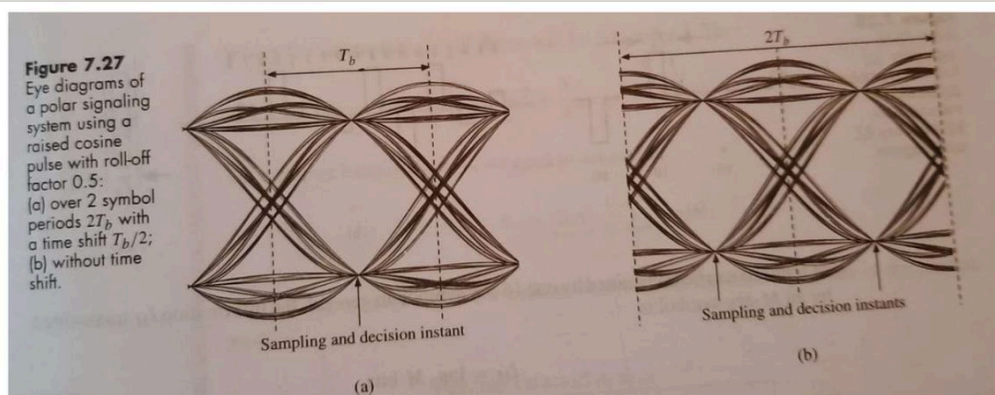
- 1. Optimum sampling instant** – at the point of maximum eye opening. Sample here.
- 2. Noise margin** – the *vertical* opening at the best sampling point. How much noise you can take before an error.
- 3. Slope** – shows sensitivity to sampling time. Steep sides mean a small timing error hurts a lot.
- 4. Level-crossing ambiguity** – the spread of the zero crossings. This is the **timing jitter**.
- 5. Distortion at the best sampling point** – how much the trace has thickened, i.e. the ISI.

**6. Error-free sampling region** – the *horizontal* width of the eye. How much timing error you can tolerate.

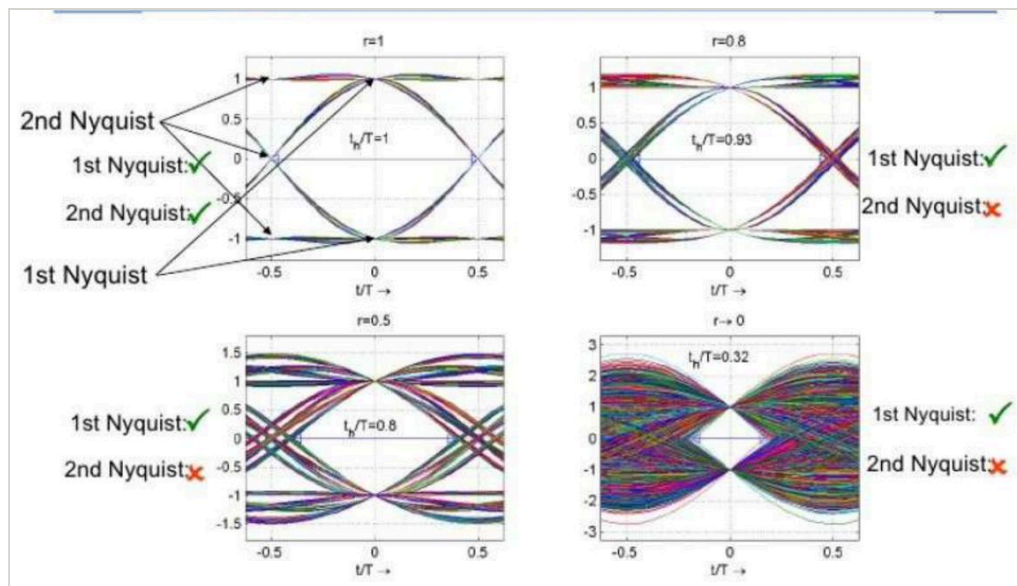
#### IN SIMPLE WORDS – THE ONE SENTENCE TO REMEMBER

**A wide-open eye means a healthy link. A closing eye means trouble — vertically it is losing noise margin, horizontally it is losing timing margin.**

An eye that has closed completely means the ISI alone is enough to cause errors, with no noise required at all.



Real eye diagrams for a polar signalling system using a raised cosine pulse with roll-off 0.5, drawn (a) over two symbol periods with a time shift, and (b) without the shift. The sampling and decision instants sit at the widest openings. Your slide notes eye diagrams work for **both** bit transmission and symbol transmission.



The payoff figure: the same system at four roll-off factors. This single image explains why nobody uses  $r = 0$  in practice.

#### READING THE EYE PATTERN FIGURE – WHAT EACH ROLL-OFF GIVES YOU

| R               | 1ST NYQUIST | 2ND NYQUIST | $T_H/T$ | WHAT THE EYE LOOKS LIKE                |
|-----------------|-------------|-------------|---------|----------------------------------------|
| 1               | Yes         | Yes         | 1.00    | Wide open and clean                    |
| 0.8             | Yes         | No          | 0.93    | Still very good                        |
| 0.5             | Yes         | No          | 0.80    | More cluttered, overshoot to $\pm 1.5$ |
| $\rightarrow 0$ | Yes         | No          | 0.32    | Nearly closed, overshoot to $\pm 3$    |

#### IN SIMPLE WORDS – WHAT $T_H/T$ IS TELLING YOU

$t_h/T$  is the fraction of the symbol period in which you can safely sample. It is the horizontal width of the eye, as a fraction of one symbol.

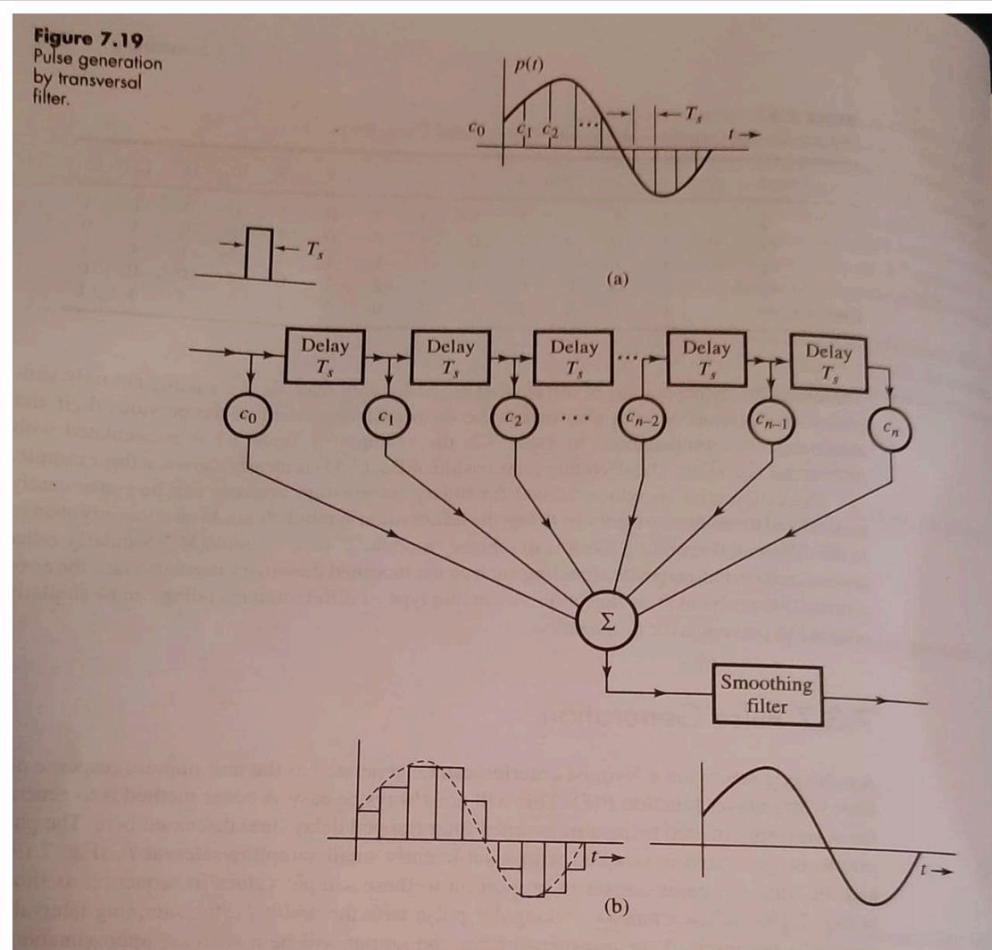
At  $r = 1$  it is 1.00 — you could sample anywhere in the symbol and still be right. At  $r \rightarrow 0$  it collapses to 0.32 — you must hit a window less than a third of a symbol wide, or you make errors.

And look at the overshoot column: at  $r \rightarrow 0$  the waveform swings to  $\pm 3$  instead of  $\pm 1$ . That means your amplifier must handle three times the voltage, so it costs power as well as timing accuracy.

So the “free” bandwidth saving of a small roll-off is not free at all. You pay for it in timing precision and in transmitter power. **That is why practical systems sit around  $r = 0.2$  to  $0.5$ .**

## 4.11 How the pulse is actually generated

We have decided what shape we want. How does a circuit build it?



Pulse generation by a **transversal filter**. (a) the pulse you want, sampled at intervals  $T_s$  – those samples become the tap weights  $c_0, c_1, c_2 \dots$  (b) the circuit: the input passes down a chain of delays, each tap is multiplied by its stored coefficient, everything is summed, and a smoothing filter turns the staircase into the smooth curve.

### IN SIMPLE WORDS

A **transversal filter** is just a row of delay boxes with a multiplier hanging off each one, all feeding an adder. Nothing more.

The clever part is the choice of multipliers: **the tap weights are the samples of the pulse you want to build**. So if you want a raised cosine, you sample a raised cosine,

store those numbers as the coefficients, and the circuit reproduces the shape every time an input pulse arrives.

Remember this structure — the zero-forcing equalizer in section 4.13 is the exact same circuit, used backwards.

## 4.12 Regenerative repeaters

Signals fade over distance. Before we can talk about fixing ISI, we need to talk about what sits along the route.

### FORMULA – CHANNEL ATTENUATION

$$r(t) = a s(t) + n(t), \quad 0 \leq a \leq 1$$

If the transmitted energy is  $E_b$ , the received energy is  $a^2 E_b$ , so the received SNR is  $a^2 E_b / N_0$

### IN SIMPLE WORDS

$a$  is the fraction of your signal that survives the journey. Since power goes as voltage squared, an  $a$  of 0.1 means you keep only  $0.01$  — one percent — of your power.

Meanwhile the noise  $n(t)$  is added *after* the attenuation, at full strength. So the signal shrinks and the noise does not, and your SNR collapses. As your slide puts it: **this makes the system more vulnerable to the additive noise.**

### ANALOG REPEATER

An **amplifier**. It boosts the signal strength along the path.

But it boosts the **noise too** — your slide calls this **noise propagation**. Every hop adds a little more noise on top of the noise already there, and it accumulates forever. After twenty hops the signal is unusable.

### REGENERATIVE REPEATER

It **detects and regenerates** a clean, noise-free signal.

It decides “that was a 1”, throws the noisy waveform away, and builds a **brand new perfect 1**. The noise does not accumulate at all. Used on wire line, fibre and wireless

systems.

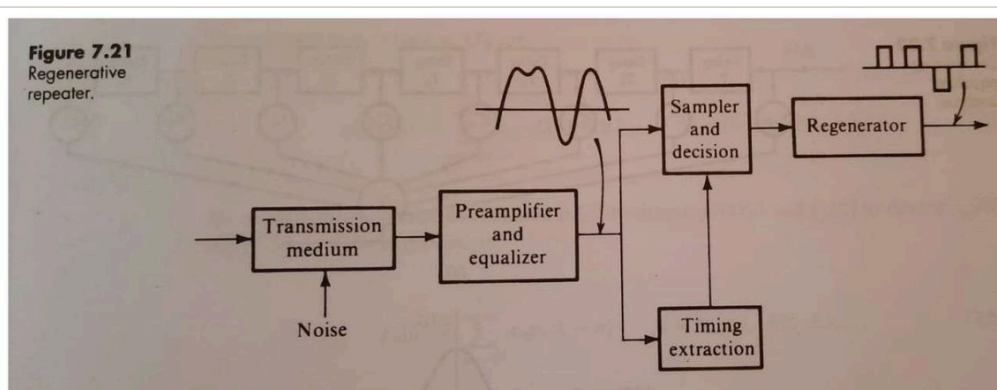
#### IN SIMPLE WORDS – THE ONE BIG ADVANTAGE, AND THE ONE CATCH

**The advantage:** this is the deepest reason digital beats analog. An analog chain accumulates noise at every hop and can never recover. A digital chain *resets to perfect* at every repeater. You can cross an ocean with no degradation at all.

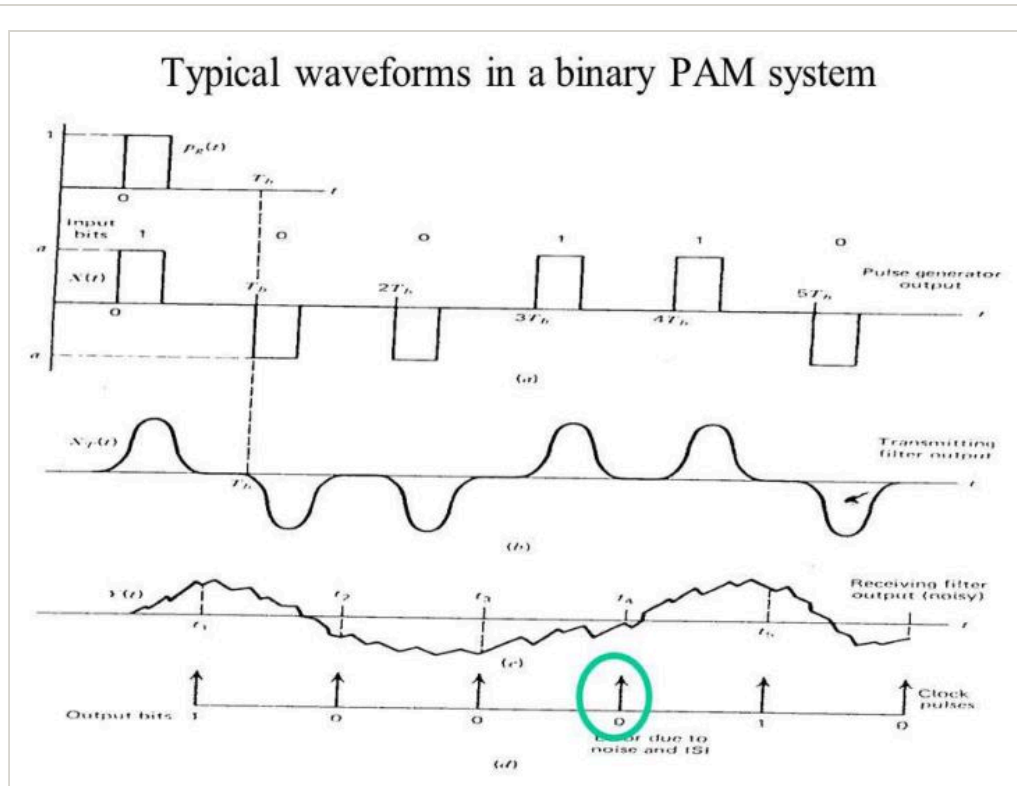
**The catch,** stated on your slide: “if an error occurs in the detector, this error is **propagated forward.**” Once a repeater decides wrongly, it regenerates that wrong bit *perfectly*, and every repeater after it faithfully passes on the mistake. Nothing downstream can tell it was ever wrong.

#### THE THREE FUNCTIONS OF A REGENERATIVE REPEATER – MEMORISE THESE

1. **Reshaping** incoming pulses by means of an **equalizer**.
2. **Extracting the timing** information required to sample incoming pulses at optimum instants.
3. **Making symbol detection decisions** based on the pulse samples.



The regenerative repeater. Signal and noise enter the transmission medium; the **preamplifier and equalizer** reshape the pulse; the result feeds both the **sampler and decision** block and the **timing extraction** block, which supplies the clock; the **regenerator** then builds fresh, clean rectangular pulses. Those three blocks are exactly the three functions listed above.



Typical waveforms in a binary PAM system: (a) the input bits and pulse generator output, (b) the smooth transmitting filter output, (c) the noisy receiving filter output with the sampling instants marked, (d) the clock pulses and recovered bits. Note the bit circled in green – that is an **error due to noise and ISI**, and it is exactly the kind of mistake a repeater will now propagate forever.

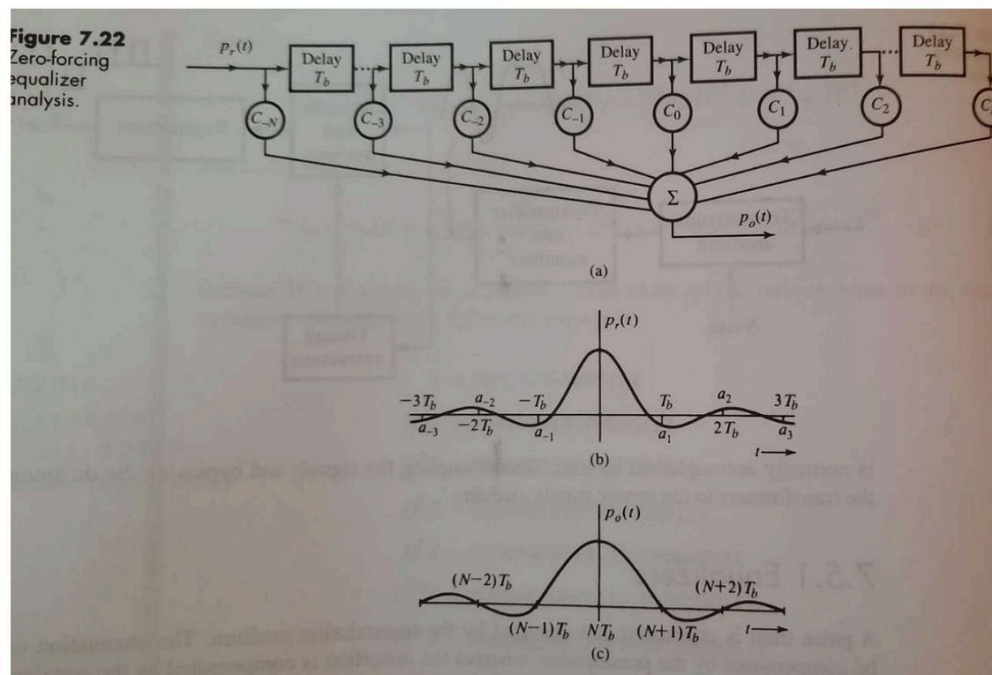
## 4.13 The zero-forcing equalizer

Function 1 of the repeater was “reshaping by means of an equalizer”. This is that equalizer.

### IN SIMPLE WORDS – THE IDEA IN ONE LINE

The channel smeared the pulse. An **equalizer** is a filter that applies the *opposite* smear, so the two cancel and a clean pulse comes out.

Physically it is the same tapped-delay-line from section 4.11, but now the tap weights are chosen to **undo** the channel rather than to build a pulse.



Zero-forcing equalizer analysis. (a) the circuit – the received pulse  $p_r(t)$  runs down a delay line with taps  $C_{-N} \dots C_0 \dots C_N$  ( $2N+1$  in all), summed to give the output  $p_o(t)$ . (b) the received pulse: it peaks correctly but has non-zero values  $a_{-3} \dots a_3$  at the neighbouring sampling instants – those are the ISI. (c) the equalized output: now exactly zero at the neighbouring instants.

#### FORMULA – THE EQUALIZER OUTPUT AND THE DESIGN CONDITION

Output at the  $k$ -th sampling instant:

$$p_o[k] = \sum_{n=-N}^N c_n p_r[k - n] \quad k = 0, \pm 1, \pm 2, \dots$$

The first Nyquist criterion demands:

$$p_o[k] = 1 \text{ for } k = 0, \quad p_o[k] = 0 \text{ for } k = \pm 1, \pm 2, \dots, \pm N$$

#### IN SIMPLE WORDS – AND WHERE THE NAME COMES FROM

Your slide is refreshingly honest about the limitation. Substituting the Nyquist condition gives **an infinite set of simultaneous equations in only  $2N+1$  unknowns**. You cannot possibly solve them all.

So we compromise: we **force the  $2N$  nearest neighbours to zero** and simply accept whatever happens further out. That is literally where the name comes from — you are **forcing zeros** at the sampling instants you can reach.

A 3-tap equalizer kills the ISI from the one symbol either side. A 5-tap kills two either side. More taps means better cancellation and more cost.

#### FORMULA – THE MATRIX EQUATION TO SOLVE

$$\mathbf{P}_0 = \mathbf{P}_r \mathbf{c} \quad \rightarrow \quad \mathbf{c} = \mathbf{P}_r^{-1} \mathbf{P}_0$$

$\mathbf{P}_0$  = target vector  $[0 \dots 0 \ 1 \ 0 \dots 0]^T$  with the 1 in the middle

$\mathbf{P}_r$  = matrix of received-pulse samples, each row shifted one place from the last

$\mathbf{c}$  = the tap weights  $[c_{-N} \dots c_0 \dots c_N]^T$  you are solving for

#### WORKED EXAMPLE – NINTH.PDF, EXAMPLE 7.3 (DESIGN A 3-TAP EQUALIZER)

**Question.** For the received pulse  $p_r(t)$ , let

$$p_r[0] = 1$$

$$p_r[1] = -0.3 \quad p_r[2] = 0.1$$

$$p_r[-1] = -0.2 \quad p_r[-2] = 0.05$$

Design a three-tap ( $N = 1$ ) equalizer.

##### 1 Understand what you have been given

The received pulse should have been 1 at its own instant and 0 everywhere else. Instead it is  $-0.3$  one symbol later,  $-0.2$  one symbol earlier, and so on. **Those non-zero values are the ISI** we must cancel.

##### 2 Set up the matrix

$N = 1$  means three taps:  $c_{-1}, c_0, c_1$ . Each row of the matrix shifts the index by one:

row 1 uses  $p_r[0], p_r[-1], p_r[-2]$

row 2 uses  $p_r[1], p_r[0], p_r[-1]$

row 3 uses  $p_r[2], p_r[1], p_r[0]$

### 3 Write it out

$$\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 & -0.2 & 0.05 \\ -0.3 & 1 & -0.2 \\ 0.1 & -0.3 & 1 \end{bmatrix} \begin{bmatrix} c_{-1} \\ c_0 \\ c_1 \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 & -0.2 & 0.05 \\ -0.3 & 1 & -0.2 \\ 0.1 & -0.3 & 1 \end{bmatrix} \begin{bmatrix} c_{-1} \\ c_0 \\ c_1 \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 & -0.2 & 0.05 \\ -0.3 & 1 & -0.2 \\ 0.1 & -0.3 & 1 \end{bmatrix} \begin{bmatrix} c_{-1} \\ c_0 \\ c_1 \end{bmatrix}$$

The left-hand vector is the target: we want the output to be **0** one symbol early, **1** at the centre, and **0** one symbol late.

### 4 Solve the three simultaneous equations

$$c_{-1} - 0.2 c_0 + 0.05 c_1 = 0$$

$$-0.3 c_{-1} + c_0 - 0.2 c_1 = 1$$

$$0.1 c_{-1} - 0.3 c_0 + c_1 = 0$$

$$c_{-1} = 0.210 \quad \cdot \quad c_0 = 1.13 \quad \cdot \quad c_1 = 0.318$$

### 5 Confirm what that buys you

As your slide states, this tap setting assures that

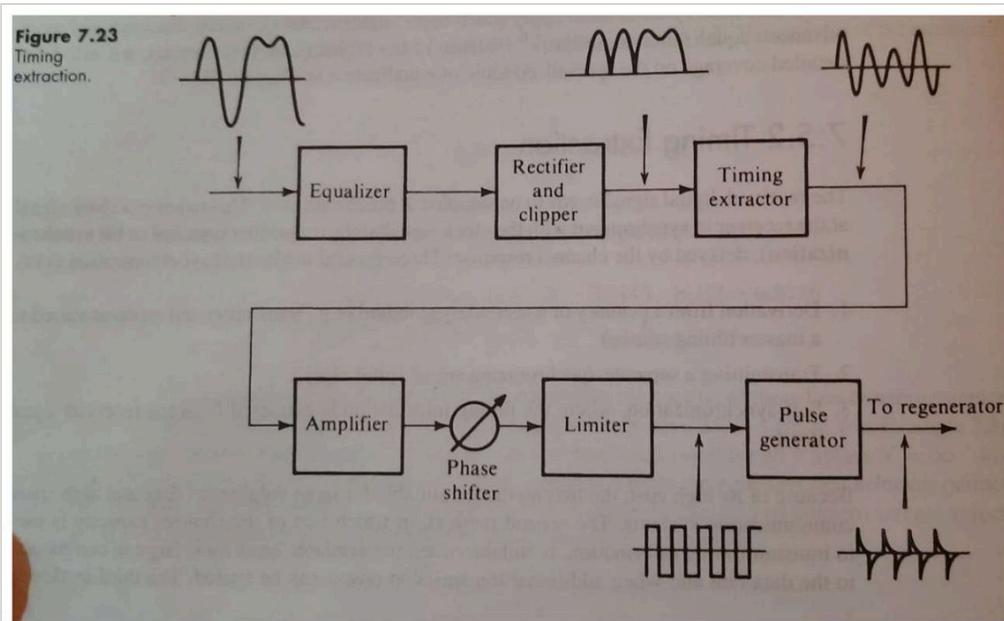
$$p_o[0] = 1 \quad \text{and} \quad p_o[-1] = p_o[1] = 0$$

The ISI from the symbol either side has been cancelled exactly.

**Read the answer.** The centre tap  $c_0 = 1.13$  is the biggest, which makes sense — most of the output is the pulse itself. The side taps are small and positive, adding just enough of the delayed and advanced copies to cancel the negative tails at  $\pm 1$ . Note also that we only forced **two** zeros; the ISI at  $\pm 2$  is left untouched, which is the price of using only three taps.

## 4.14 Timing extraction

Function 2 of the repeater. The receiver needs a clock, and nobody sent it one — so it must be manufactured from the data itself.



The timing extraction chain. Watch the little waveform sketches above the blocks: the messy bipolar signal becomes all-positive humps after the rectifier, a clean sine wave after the timing extractor, and finally a train of narrow clock pulses for the regenerator.

#### IN SIMPLE WORDS — HOW YOU MAKE A CLOCK OUT OF NOTHING

The received data already contains a weak hidden rhythm at the symbol rate — but it is buried, and in a bipolar signal the positive and negative swings tend to cancel it out. Each block digs it out:

1. **Equalizer** — cleans up the pulse shape first.
2. **Rectifier and clipper** — flips all the negative parts positive. Now nothing cancels, and the symbol-rate rhythm becomes strong.
3. **Timing extractor** — a narrow resonant filter tuned to the symbol rate. It rings like a tuning fork and outputs a clean sine wave at exactly that frequency.
4. **Amplifier and phase shifter** — boost it, then slide it sideways so the clock edge lands at the *optimum sampling instant* (the middle of the eye).
5. **Limiter** — squares the smooth sine into a sharp rectangular clock.
6. **Pulse generator** — turns each edge into a narrow pulse to trigger the regenerator.

This is why **self-synchronising line codes matter**. If the data has no transitions, there is no rhythm to extract, and this entire chain has nothing to work with.

## 4.15 Duobinary signalling

One last idea, and it is a clever reversal of everything so far. Also called **correlative coding** or **partial response signalling**.

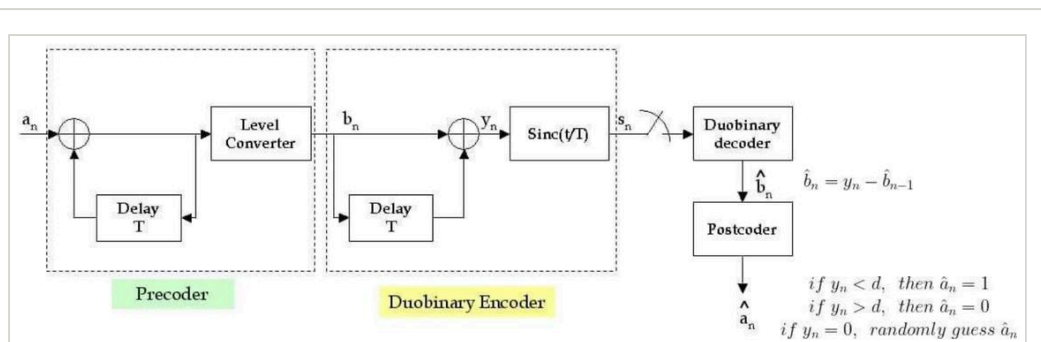
### THE IDEA – IF YOU CANNOT BEAT ISI, CONTROL IT

So far the goal has been **zero** ISI:  $p(nT) = 1$  for  $n = 0$ ,  $0$  otherwise. But we saw in section 4.6 that hitting the ideal Nyquist rate requires an impossible brick-wall filter.

Duobinary takes the opposite approach. Instead of fighting ISI, it **deliberately introduces a known, controlled amount of it**, then subtracts it back out at the receiver. Your slide puts it exactly:

*“In partial response signaling, the requirement of the zero ISI condition is **relaxed** as a **controlled** amount of ISI is introduced in the transmitted signal and is **counteracted** at the receiver side.”*

The relaxed condition is  $p(nT) = 1$  for  $n = 0$  and  $n = 1$ ,  $0$  otherwise — the ISI is **limited to two adjacent samples**. Because that interference is **deterministic** (you know exactly how much and from where), its effect can be removed at detection.



The duobinary system. The **precoder** (XOR with a one-symbol delay, then a level converter) prevents error propagation. The **duobinary encoder** adds each symbol to the previous one and shapes it with a sinc filter. At the far end the **duobinary decoder** and **postcoder** undo both steps.

### FORMULA – THE DUOBINARY ENCODING PROCESS

1.  $a_n$  = binary input bit,  $a_n \in \{0, 1\}$
2.  $b_n$  = NRZ polar output of the level converter:

$$b_n = -d \text{ if } a_n = 0; \quad b_n = +d \text{ if } a_n = 1$$

3.  $y_n = b_n + b_{n-1} =$

$$+2d \text{ if } a_n = a_{n-1} = 1$$

$$0 \text{ if } a_n \neq a_{n-1}$$

$$-2d \text{ if } a_n = a_{n-1} = 0$$

$$\text{Decoding: } \hat{b}_n = y_n - \hat{b}_{n-1}$$

#### IN SIMPLE WORDS

Step 2 turns your 0s and 1s into  $-d$  and  $+d$  volts. Step 3 is the key: **each transmitted value is the sum of the current symbol and the previous one**. That sum is the “controlled ISI” — you have deliberately let one neighbour leak in.

The result is a **three-level** signal:  $+2d$ ,  $0$  or  $-2d$ . And notice what those three levels tell you:

- **$+2d$**  → both this bit and the last one were 1
- **$-2d$**  → both were 0
- **$0$**  → they were different

Your slide makes the contrast precisely: the samples  **$b_n$  are uncorrelated** (just  $+d$  or  $-d$ ), but the samples  **$y_n$  are correlated** — which is why the technique is called **correlative coding**. The encoding deliberately correlates the present input with the previous one.

#### Encoding Process:

1)  $a_n$  = binary input bit;  $a_n \in \{0,1\}$ .

2)  $b_n$  = NRZ polar output of Level converter in the precoder and is given by,

$$b_n = \begin{cases} -d, & \text{if } a_n = 0 \\ +d, & \text{if } a_n = 1 \end{cases}$$

3)  $y_n$  can be represented as  $y_n = b_n + b_{n-1} = \begin{cases} 2d, & \text{if } a_n = a_{n-1} = 1 \\ 0, & \text{if } a_n \neq a_{n-1} \\ -2d, & \text{if } a_n = a_{n-1} = 0 \end{cases}$

No  
eit  
oth  
the  
dep  
du  
and

Advanta

The encoding equations as printed on your slide.

#### THE PAYOFF, AND THE ROLE OF THE PRECODER

**The payoff** is the box on your slide: “**we can send 2B bitrate over Bandwidth B.**”

Duobinary reaches the ideal Nyquist rate of 2 bits/s/Hz — the thing that  $r = 0$

promised but could never deliver — and it does it with a **realizable** filter. That is the whole point of the technique.

**The precoder** exists to solve one problem. Plain decoding uses  $\hat{b}_n = y_n - \hat{b}_{n-1}$ , which depends on the *previous decision*. So one wrong decision would poison every decision after it — error propagation. The precoder's XOR-and-delay at the transmitter arranges things so the receiver can decide each bit on its own, breaking that chain.

#### CORRECTION – THE POSTCODER RULE ON THE LAST SLIDE

The final slide prints the postcoder decision rule as:

```
if  $y_n < d$ , then  $\hat{a}_n = 1$  · if  $y_n > d$ , then  $\hat{a}_n = 0$  ·  
if  $y_n = 0$ , randomly guess
```

As written this is inconsistent, because a value can be both “< d” and negative, and the rule gives no sensible answer for  $y_n = -2d$ .

**The rule that actually works** is a test on the *magnitude*, which follows directly from the three levels above:

```
if  $|y_n|$  is near  $2d \rightarrow$  this bit is the same as the  
previous one
```

```
if  $|y_n|$  is near  $0 \rightarrow$  this bit is different from the  
previous one
```

Learn the three-level meaning (+2d / 0 / -2d) rather than memorising the printed inequalities — then you can always reconstruct the decision rule correctly.

#### EXAM TIP – THE STORY OF THIS WHOLE MODULE IN FIVE LINES

1. A bandlimited channel smears pulses, so they overlap  $\rightarrow$  **ISI**.
2. Nyquist: overlapping is fine, as long as the interference is **zero at the sampling instants**.
3. The perfect ( $r = 0$ ) pulse achieves this but cannot be built  $\rightarrow$  use a **raised cosine** with  $r \approx 0.2-0.5$ .
4. Check your work on an **eye diagram**; fix what the channel still ruins with a **zero-forcing equalizer**.
5. Or invert the whole idea: allow a **known** amount of ISI and subtract it later  $\rightarrow$  **duobinary**, which reaches 2 bits/s/Hz for real.



## 05 FROM THIRD.PDF AND SIXTH.PDF — TUTORIAL 4

# Measuring information

Here the subject becomes beautiful. Claude Shannon asked a question nobody had asked before: can you put a *number* on how much information a message carries? The answer built the digital world.

## 4.1 The two questions information theory answers

Your third.pdf opens with them. They are worth memorising word for word, because they explain why the rest of the course exists:

1. **What is the ultimate limit to data compression?** How small can I squeeze this file? → The answer is **entropy**, and it leads to Module 7.
2. **What is the ultimate limit of reliable communication over a noisy channel?** How fast can I send through this hiss without errors? → The answer is **channel capacity**, and it leads to Modules 6 and 8.

Information theory is a branch of probability theory. It does not care whether your channel is copper or fibre or air — it is pure mathematics about sources and channels. That is why its answers are *ultimate* limits that no future technology can beat.

### IN SIMPLE WORDS — THE TWO BUILDING BLOCKS

Your slides model everything with two idealised pieces:

**The Binary Source (BS)** — a machine that spits out 0s and 1s at a rate of  $R_b$  symbols per second.

**The Binary Symmetric Channel (BSC)** — a pipe that carries one symbol per time unit, but is unreliable: with probability  $p$  the bit comes out flipped. “Symmetric” means a 0 turning into a 1 is just as likely as a 1 turning into a 0. Normally  $0 \leq p \leq 1/2$ .

## 4.2 What *is* information?

This is the idea that trips everyone up at first, so read it twice. Your slide says it plainly:

### THE KEY IDEA

Information depends **only on the probability of the event**. It does **not** depend on the content, the meaning, or the importance of the message.

A message about the **least likely** event contains the **maximum** information.

Compare two messages sent to you in Surat in June:

- “It rained today.” — It is the monsoon. You expected it. This tells you almost nothing. **Low information.**
- “It snowed today.” — Astonishing. You would call everyone you know. **Enormous information.**

Both sentences are five words long. Both cost the same to send. But one *surprised* you and the other did not. Information, in Shannon's sense, is a measurement of **surprise**.

### The four axioms

Before writing a formula, Shannon listed what any sensible measure of information must obey. Your slide gives four. If you are asked to “justify the logarithmic measure of information”, this list is the answer:

| AXIOMS OF AN INFORMATION MEASURE |                                            |                                                                                                                            |
|----------------------------------|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| #                                | AXIOM                                      | WHY IT MUST BE TRUE                                                                                                        |
| 1                                | $I(p) \geq 0$                              | You can never learn a negative amount. At worst you learn nothing.                                                         |
| 2                                | $I(1) = 0$                                 | An event that was certain to happen tells you nothing when it happens.                                                     |
| 3                                | $I(p_1 p_2) = I(p_1) + I(p_2)$             | Two independent pieces of news should <i>add up</i> . Learning two unrelated facts gives you the sum of their information. |
| 4                                | $I(p)$ is continuous and decreasing in $p$ | A slightly less likely event carries slightly more information — no sudden jumps, and rarer always means more.             |

Only one kind of function satisfies all four at once: the **logarithm of  $1/p$** . Axiom 3 is the decisive one, because the logarithm is the function that turns multiplication into addition.

## 4.3 The information content formula

FORMULA — INFORMATION CONTENT OF ONE SYMBOL

$$I(x_i) = \log_b(1 / P(x_i)) = -\log_b P(x_i)$$

$P(x_i)$  = probability that symbol  $x_i$  happens ·  $b$  = the base of the logarithm

#### IN SIMPLE WORDS

Take the probability. Flip it upside down (that turns “rare” into “big”). Take the log of that (which stops the numbers exploding, and makes independent events add). That is all the formula does.

The minus sign in the second version is *not* making the answer negative. Probabilities are always less than 1, so their logarithm is always negative — and the minus sign flips it back to positive. Both versions give the same positive answer. Use whichever is easier.

### The three units

The base  $b$  only changes the *unit* you measure information in, exactly like measuring a length in metres or feet.

| UNITS OF INFORMATION |                                   |                                                                             |
|----------------------|-----------------------------------|-----------------------------------------------------------------------------|
| BASE $b$             | UNIT                              | WHEN USED                                                                   |
| 2                    | <b>bit</b>                        | The standard everywhere in this subject. Assume this unless told otherwise. |
| $e$                  | <b>nat</b>                        | Natural log. Convenient in theoretical proofs.                              |
| 10                   | <b>hartley</b> (or <b>decit</b> ) | Rare. Named after Ralph Hartley.                                            |

#### EXAM TIP – COMPUTING $\log_2$ ON A NORMAL CALCULATOR

Most calculators have no  $\log_2$  button. Use the change-of-base rule, which is on your slide:

$$\log_2 a = \ln a / \ln 2 = \log_{10} a / \log_{10} 2$$

Since  $\log_{10} 2 = 0.3010$ , a fast shortcut is  **$\log_2 a = \log_{10} a \div 0.3010$** , or equivalently  **$\times 3.322$** .

Worth memorising outright:  $\log_2 2 = 1$ ,  $\log_2 4 = 2$ ,  $\log_2 8 = 3$ ,  $\log_2 16 = 4$ ,  $\log_2 32 = 5$ ,  $\log_2 64 = 6$ ,  $\log_2 128 = 7$ ,  $\log_2 256 = 8$ ,  $\log_2 512 = 9$ ,  $\log_2 1024 = 10$ . Also  $\log_2 3 = 1.585$ ,  $\log_2 5 = 2.322$ ,  $\log_2 6 = 2.585$ ,  $\log_2 10 = 3.322$ .

### Four properties of $I(x_i)$

1.  $I(x_i) = 0$  when  $P(x_i) = 1$  — a certain event carries no information.
2.  $I(x_i) \geq 0$  — never negative.
3.  $I(x_i) > I(x_j)$  if  $P(x_i) < P(x_j)$  — the rarer event carries more.
4.  $I(x_i, x_j) = I(x_i) + I(x_j)$  if the two are independent — information adds.

#### WORKED EXAMPLE — TUTORIAL 4, EXAMPLE 1

**Question.** An event  $E$  occurs with probability  $p(E) = 0.2$ . Compute  $I(E)$  in bits, nats and hartleys.

##### 1 In bits (base 2)

$$I_2(E) = -\log_2(0.2) = \log_2(1/0.2) = \log_2 5$$

**2.3219 bits**

##### 2 In nats (base e)

$$I_e(E) = -\ln(0.2) = \ln 5$$

**1.6094 nats**

##### 3 In hartleys (base 10)

$$I_{10}(E) = -\log_{10}(0.2) = \log_{10} 5$$

**0.6990 hartleys**

Notice the trick used in all three lines:  $-\log(0.2)$  is the same as  $\log(1/0.2) = \log(5)$ . Flipping the fraction removes the minus sign and makes it easier to type into a calculator.

#### WORKED EXAMPLE — TUTORIAL 4, EXAMPLE 2

**Question.** A fair die is rolled once. What is the information content of observing a specific face?

### 1 Find the probability

A fair die has six equally likely faces:

$$p(E) = 1/6$$

### 2 Apply the formula

$$I = -\log_2(1/6) = \log_2 6 = \ln 6 / \ln 2 = 1.7918 / 0.6931$$

**2.5850 bits**

**Sanity check.** 2 bits could label only 4 faces; 3 bits could label 8. Six faces sits between, so the answer must sit between 2 and 3 — and 2.585 does. Information does not have to be a whole number of bits.

## 4.4 The Discrete Memoryless Source

Before entropy, one definition. A **source** is anything producing outcomes at random from a fixed list. That list of possible symbols is the **source alphabet**, and each item in it is a **symbol** or **letter**.

Sources come in two kinds:

- **Source with memory** — the next symbol depends on the ones before. English text is like this: after “Q”, the next letter is almost always “U”.
- **Memoryless source** — every symbol is independent of all the others. Like rolling a die again and again.

A **DMS (Discrete Memoryless Source)** is described by exactly three things, and your slide lists them:

1. The list of symbols it can produce.
2. The probability assigned to each symbol.
3. The rate at which it produces them (symbols per second).

## 4.5 Entropy — the average information

The formula  $I(x_i)$  gives the information in *one particular* symbol. But a source keeps producing symbols. On average, how much information does each one carry? That average is called **entropy**, written **H(X)**.

**FORMULA — ENTROPY OF A SOURCE**

$$H(X) = \sum P(x_i) \log_2(1/P(x_i)) = - \sum P(x_i) \log_2 P(x_i)$$

units: bits per symbol · the sum runs over every symbol in the alphabet

#### IN SIMPLE WORDS

Entropy is just a **weighted average**. For each symbol, multiply *how much information it carries* by *how often it appears*, then add all those products together.

That word “weighted” matters. A very rare symbol carries a lot of information, but it shows up so seldom that it barely moves the average. A common symbol carries little information but appears constantly. Entropy balances the two.

Three ways to say the same thing, all correct in an exam: entropy is **the average information per symbol**; entropy is **the average uncertainty of the source**; entropy is **the smallest average number of bits needed to represent one symbol**. That last one is the bridge to Module 7.

#### EXAM TIP – THE TWO ASSUMPTIONS BEHIND $H(X)$

Your slide states them explicitly, and examiners ask for them:

1. The source is **stationary** — the probabilities do not change over time.
2. Successive symbols are **statistically independent** and arrive at an average rate of  $r$  symbols/sec.

### Three properties of entropy

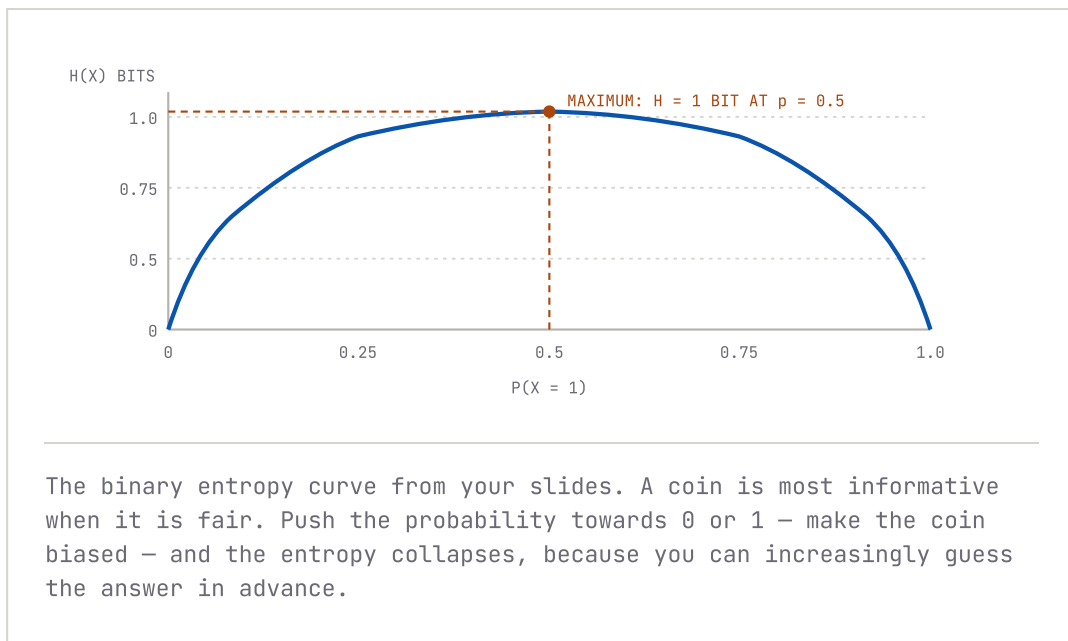
#### FORMULA – BOUNDS ON ENTROPY

$$0 \leq H(X) \leq \log_2 m$$

$m$  = number of symbols in the alphabet

1. **The lowest possible entropy is 0.** This happens when one symbol has probability 1 and all the others have probability 0. The source always says the same thing, so there is no uncertainty and no information.
2. **The highest possible entropy is  $\log_2 m$ .** This happens when all  $m$  symbols are **equally likely**. Maximum unpredictability means maximum average information.

3. **Everything real sits in between.** The more lopsided the probabilities, the lower the entropy — and the more a compressor can squeeze the data.



## 4.6 Information rate

Entropy is measured *per symbol*. To find out how many bits per *second* a source produces, multiply by how many symbols it produces per second.

### FORMULA — INFORMATION RATE

$$R = r \times H(X) \quad \text{bits per second}$$

$r$  = symbol rate (symbols/sec)     $H(X)$  = entropy (bits/symbol)

### IN SIMPLE WORDS

(symbols per second)  $\times$  (information bits per symbol) = (information bits per second).  
The units cancel exactly like buying apples: 5 boxes/hour  $\times$  12 apples/box = 60 apples/hour.

Keep **R** firmly separate from **C**. **R** is how fast your *source* talks; **C** is how fast the *channel* can listen. Module 6 shows that everything works if  $R < C$ , and nothing works if  $R > C$ .

## 4.7 Worked examples — Tutorial 4, Examples 3, 4 and 5

### WORKED EXAMPLE — EXAMPLE 3, THE FAIR COIN

**Question.** A fair coin has two outcomes {H, T}, each with  $p = 0.5$ . Calculate the entropy.

#### 1 Apply the entropy formula to both outcomes

$$H(X) = -[ 0.5 \log_2 0.5 + 0.5 \log_2 0.5 ]$$

#### 2 Evaluate the logs

Since  $\log_2 0.5 = \log_2(1/2) = -1$ :

$$H(X) = -[ 0.5(-1) + 0.5(-1) ] = -[-0.5 - 0.5] = -(-1)$$

$$H(X) = 1.0 \text{ bit}$$

This is the **maximum possible** entropy for a binary source, because  $\log_2 2 = 1$ . A fair coin is the most unpredictable two-outcome source there is. This is also the origin of the word “bit”: one bit is exactly the information in one fair coin toss.

### WORKED EXAMPLE — EXAMPLE 4, THE BIASED COIN

**Question.** A biased coin has  $P(X = H) = 0.8$ . Calculate the entropy.

#### 1 Find both probabilities

Probabilities must add to 1, so:

$$P(H) = 0.8 \quad P(T) = 1 - 0.8 = 0.2$$

#### 2 Substitute into the entropy formula

$$H(X) = -[ 0.8 \log_2 0.8 + 0.2 \log_2 0.2 ]$$

### 3 Evaluate each logarithm

$$\log_2 0.8 = -0.3219 \quad \log_2 0.2 = -2.3219$$

$$H(X) = -[0.8(-0.3219) + 0.2(-2.3219)]$$

$$= 0.2575 + 0.4644$$

$$H(X) \approx 0.7219 \text{ bits}$$

**Read the meaning.** The biased coin gives 0.72 bits, the fair coin gave 1.0 bit. Bias always *lowers* entropy, because the outcome is easier to guess. And this is not abstract: it says a long recording of this coin could in principle be compressed to about 72% of its length, while a fair coin's record cannot be compressed at all.

### WORKED EXAMPLE – EXAMPLE 5, THE TWO-MODE SENSOR

**Question.** A sensor operates in two modes. Mode 1 outputs either A or B, each with probability 0.5. Mode 2 outputs one of {C, D, E, F}, each with probability 0.25. Z is a mode indicator with  $P(Z = 1) = P(Z = 2) = 0.5$ . The output is X. Calculate  $H(X)$ .

#### 1 Understand the two-stage randomness

To produce one output, the sensor does two random things: first it picks a mode, then it picks a symbol *within* that mode. To get the overall probability of a symbol, multiply the two.

#### 2 Find the probability of each of the six symbols

A and B come from Mode 1, which is chosen half the time, and within it each has probability 0.5:

$$P(A) = P(B) = 0.5 \times 0.5 = 0.25$$

C, D, E and F come from Mode 2, chosen half the time, each with probability 0.25 inside it:

$$P(C) = P(D) = P(E) = P(F) = 0.5 \times 0.25 = 0.125$$

#### 3 Check the probabilities add to 1

$$2(0.25) + 4(0.125) = 0.5 + 0.5 = 1 \quad \checkmark$$

Always do this check. If it does not come to 1, you have made an error and everything after it will be wrong.

#### 4 Apply the entropy formula to all six symbols

$$H(X) = -[ 2(0.25 \log_2 0.25) + 4(0.125 \log_2 0.125) ]$$

Since  $\log_2 0.25 = -2$  and  $\log_2 0.125 = -3$ :

$$\begin{aligned} H(X) &= -[ 2(0.25 \times -2) + 4(0.125 \times -3) ] \\ &= -[ -1 + -1.5 ] = 1 + 1.5 \end{aligned}$$

$$H(X) = 2.5 \text{ bits}$$

**Sanity check.** There are 6 symbols, so the maximum possible entropy is  $\log_2 6 = 2.585$  bits. Our answer of 2.5 sits just below that ceiling — correct, because the six symbols are *nearly* but not quite equally likely.

#### EXAM TIP — ENTROPY CHECKLIST

1. List every symbol and its probability. **Confirm they sum to 1.**
2. Watch for probabilities that are neat powers of  $1/2$  — 0.5, 0.25, 0.125, 0.0625. Their  $\log_2$  values are exactly  $-1, -2, -3, -4$ , so you need no calculator.
3. Compute  $-\sum P \log_2 P$ . Keep 4 decimal places until the very end.
4. Check your answer lies between 0 and  $\log_2 m$ .
5. State the unit: **bits per symbol**. Marks are lost for missing units.

06 FROM THIRD.PDF, SLIDES 16-35

## Channels and channel capacity

Module 5 measured the source. Now we measure the pipe. The result is the most famous equation in communications, and it tells you the speed limit no engineer can ever break.

### 5.1 The discrete memoryless channel

A **channel** is the path the symbols travel down. A **DMC (Discrete Memoryless Channel)** is the mathematical model of it: symbols go in as  $X$ , symbols come out as  $Y$ , and because of noise, what comes out is not always what went in.

“Memoryless” means the noise hitting this symbol has no connection to the noise that hit the last one. Each symbol takes its chances independently.

#### IN SIMPLE WORDS

The channel is described entirely by a set of “if I send this, what is the chance you receive that” probabilities. They are written  $P(y_j | x_i)$ , read as “the probability of receiving  $y_j$  **given that**  $x_i$  was sent”. These are called the **channel transition probabilities**.

#### The channel matrix

Collect all those probabilities into a grid, and you have the **channel matrix** — a complete description of the channel.

#### FORMULA - CHANNEL MATRIX AND ITS ROW RULE

$[P(Y|X)]$  is an  $m \times n$  matrix whose entry in row  $i$ , column  $j$  is  $P(y_j | x_i)$

Row rule:  $\sum_j P(y_j | x_i) = 1$  for every row  $i$

Input probabilities as a row matrix:  $[P(X)] = [P(x_1) \ P(x_2) \ \dots \ P(x_m)]$

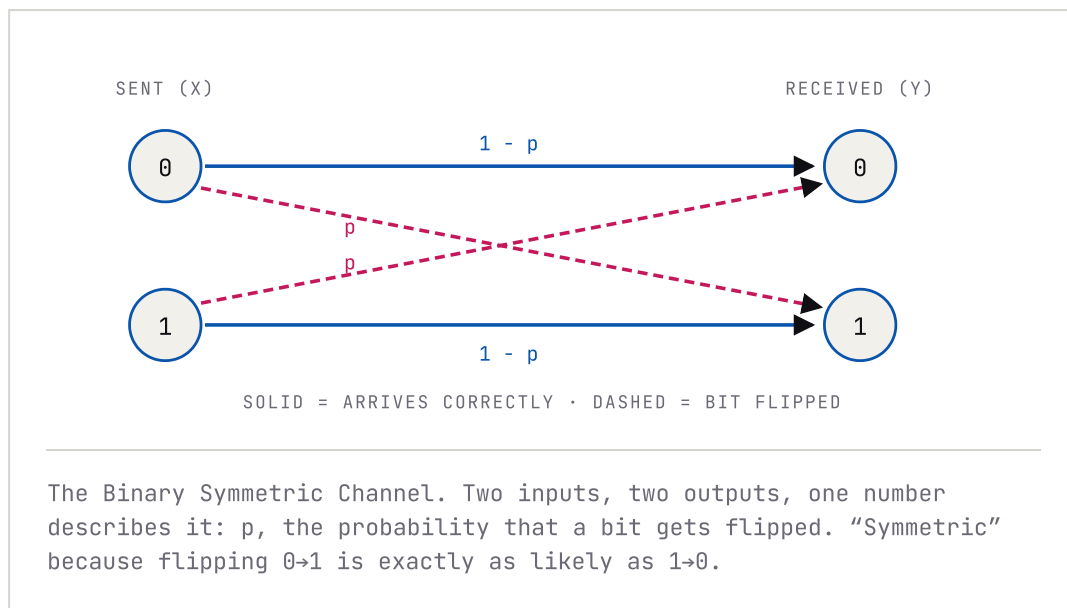
#### IN SIMPLE WORDS - WHY EVERY ROW ADDS TO 1

Each **row** is one transmitted symbol. Reading across that row tells you all the things that symbol could turn into at the far end. Something must come out — the symbol cannot simply vanish — so those possibilities must together account for 100% of the cases. Hence every row sums to exactly 1.

Columns do **not** have to sum to 1. This is the single most common mistake with channel matrices.

## 5.2 The Binary Symmetric Channel

The BSC is the simplest interesting channel, and it appears throughout your slides.



### FORMULA — THE BSC MATRIX

$$[P(Y|X)] = \begin{bmatrix} 1-p & p \\ p & 1-p \end{bmatrix}$$

$$\begin{bmatrix} p & 1-p \end{bmatrix}$$

$p$  = crossover (error) probability · each row sums to  $(1-p) + p = 1$  ✓

### EXAM TIP — THE THREE SPECIAL VALUES OF $P$

- $p = 0$  — a perfect channel. Nothing is ever flipped. Capacity = 1 bit/symbol.
- $p = 0.5$  — the *worst possible* channel. The output is a pure coin toss with no relation to the input. Capacity = 0. You learn nothing.

- **$p = 1$**  — every bit is flipped. This sounds terrible but is actually *perfect*: just invert everything at the receiver. Capacity = 1 bit/symbol again.

That is why the slides restrict  $p$  to  $0 \leq p \leq 1/2$ . Anything past 0.5 is just a relabelled version of something below it.

## 5.3 Channel capacity

Now the central definition.

### FORMULA — CHANNEL CAPACITY

Per symbol:  $C_s = \max_{\{P(x_i)\}} I(X;Y)$  bits/symbol

Per second:  $C = r \times C_s$  bits/second, where  $r =$  symbols/sec

### IN SIMPLE WORDS

Capacity is **the maximum amount of information the channel can carry per symbol**, in the best possible case.

Two pieces of that sentence need unpacking:

**$I(X;Y)$**  is the **mutual information** — how much knowing the output  $Y$  tells you about the input  $X$ . If the channel were perfect, seeing  $Y$  would tell you  $X$  exactly. If the channel were pure noise, seeing  $Y$  would tell you nothing about  $X$ , and  $I(X;Y) = 0$ .

“**max over  $\{P(x_i)\}$** ” means: the answer depends on how often you choose to send each symbol, so we take the *best possible* choice. Capacity is the channel's potential when used as cleverly as possible, not when used carelessly. For the BSC, the best choice is always to send 0 and 1 equally often.

The slides add the practical caveat: to actually reach capacity, **the information must be coded efficiently** — which is what Modules 7 and 8 build.

### Two special cases from the slides

#### FORMULA — CAPACITY OF SPECIAL CHANNELS

Noiseless channel (lossless and deterministic):

$$I(X;Y) = H(X) = H(Y) \quad \text{and} \quad C_s = \log_2 m = \log_2 n$$

Binary Symmetric Channel:

$$I(X;Y) = H(Y) + p \log_2 p + (1-p) \log_2(1-p)$$

$$C_s = 1 + p \log_2 p + (1-p) \log_2(1-p)$$

#### IN SIMPLE WORDS – READING THE BSC CAPACITY FORMULA

Look at the shape of it: **capacity = 1 – (something the noise steals)**.

The **1** is what you would get on a perfect binary channel: one bit per symbol. The two log terms are always negative, so they subtract. They are exactly the **binary entropy of the noise** — the uncertainty the channel injects.

Check it with numbers. If  $p = 0$ : the terms vanish and  $C_s = 1$  bit. If  $p = 0.5$ : the terms give  $0.5(-1) + 0.5(-1) = -1$ , so  $C_s = 1 - 1 = 0$ . Exactly as the table above predicted.

## 5.4 The Shannon–Hartley law

Everything so far has been about discrete symbols. Now we move to a real analog channel with a bandwidth and a noise level. This gives the single most quoted formula in communication engineering.

#### FORMULA – SHANNON-HARTLEY LAW

$$C = B \log_2(1 + S/N) \quad \text{bits per second}$$

$B$  = channel bandwidth in Hz ·  $S$  = signal power ·  $N$  = noise power ·  $S/N$  as a plain ratio, **not** in dB

Per-sample form:  $C_s = \frac{1}{2} \log_2(1 + S/N)$  bits/sample

#### IN SIMPLE WORDS

This one formula says: **your maximum error-free speed depends on only two things** — how much frequency room you have ( $B$ ), and how loud your signal is compared to the noise ( $S/N$ ).

Notice the asymmetry. Bandwidth sits *outside* the log, so doubling B doubles the capacity. Power sits *inside* the log, so doubling S adds only a little. **Bandwidth is far more valuable than power.** That is why 5G went hunting for mmWave spectrum instead of just shouting louder.

## 5.5 Where the formula comes from

Your slides give a neat intuitive derivation. It is a common exam question, so follow it once carefully.

### DERIVATION – SHANNON-HARTLEY IN FOUR STEPS

#### 1 How many voltage levels can you tell apart?

The received signal has power  $S + N$ , so its RMS voltage is  $\sqrt{S+N}$ . The noise alone has RMS voltage  $\sqrt{N}$ . You can only distinguish levels that are further apart than the noise. So the number of distinguishable levels is:

$$M = \sqrt{S+N} / \sqrt{N} = \sqrt{1 + S/N}$$

#### 2 How much information does one pulse carry?

From Module 3, a pulse with M levels carries  $\log_2 M$  bits:

$$I = \log_2 \sqrt{1 + S/N} = \frac{1}{2} \log_2(1 + S/N) \text{ bits}$$

The  $\frac{1}{2}$  appears because the square root is the same as a power of one half, and  $\log(a^{1/2}) = \frac{1}{2} \log a$ .

#### 3 How many pulses per second can the bandwidth carry?

By Nyquist, a channel of bandwidth B Hz can carry at most  $2B$  independent pulses per second. Writing K for the number of pulses per second:

$$C = (K/2) \log_2(1 + S/N) \quad \text{with } K = 2B$$

#### 4 Substitute $K = 2B$

$$C = (2B/2) \log_2(1 + S/N)$$

$$C = B \log_2(1 + S/N) \text{ bits/sec}$$

## 5.6 The bandwidth–power trade-off

Your slide highlights the most useful consequence of the law: for a fixed capacity  $C$ , **you can trade bandwidth against power**.

### IN SIMPLE WORDS

Suppose you must deliver a fixed data rate. You have two ways to buy it:

- **Wide bandwidth, low power** — spread the signal thinly over a lot of spectrum and whisper. This is what spread spectrum and GPS do; a GPS signal arrives weaker than the background noise, yet still works.
- **Narrow bandwidth, high power** — squeeze into a small slice of spectrum and shout. This is what a broadcast tower does.

Both can hit the same  $C$ . Which you choose depends on what is scarce and expensive in your situation: spectrum licences, or battery life.

## 5.7 Putting $R$ and $C$ together

Now the two threads join. Module 5 gave you  $R$ , how fast your source produces information. This module gave you  $C$ , how fast the channel can carry it. Your slide states the condition for a working system:

FORMULA – THE CONDITION FOR A SYSTEM TO WORK

$$H(X) \leq R < C$$

And this is the **Shannon channel coding theorem**, quoted in full on your slides:

### SHANNON'S CHANNEL CODING THEOREM

*“If the information rate of a given source does not exceed the capacity of a given channel, then there exists a coding technique that makes possible transmission through this unreliable channel with an arbitrarily low error rate.”*

### IN SIMPLE WORDS – WHY THIS RESULT IS ASTONISHING

Read it slowly, because it says something that sounds impossible.

You have a **noisy** channel. It corrupts bits. And yet Shannon proved that as long as you send slower than  $C$ , there *exists* a code that gets your data across with an error

rate as close to zero as you like. Not “fewer errors” — **arbitrarily few**. You name the error rate; a code exists that beats it.

Before 1948, every engineer believed that to cut errors you had to slow down towards zero speed. Shannon showed you can keep full speed right up to  $C$  and still get near-perfect reliability. The catch: he proved such codes *exist* but did not say how to build them. Finding them took the next fifty years — Hamming, convolutional, turbo, LDPC, polar. That hunt is Module 8.

And the hard flip side: if  $R > C$ , reliable communication is **impossible**, no matter how clever your coding. It is a wall, not a guideline.

#### EXAM TIP — THE CLASSIC SHANNON-HARTLEY NUMERICAL

A telephone line has  $B = 3$  kHz and  $\text{SNR} = 30$  dB. Find its capacity.

**Step 1 — convert dB to a plain ratio.** This is the step students forget. From  $30 = 10 \log_{10}(S/N)$  we get  $S/N = 10^3 = 1000$ .

**Step 2 — substitute.**  $C = 3000 \times \log_2(1 + 1000) = 3000 \times \log_2(1001) = 3000 \times 9.97 \approx 29\,900$  bits/s.

That number is why old dial-up modems topped out near 33.6 kbit/s. They had run into a law of physics, not an engineering limitation.

**Never** put a dB value straight into the Shannon formula. Convert with  $S/N = 10^{(\text{dB}/10)}$  first.

## 6.8 When the SNR is not handed to you — noise density and dBm

Tutorial 7 is built entirely on one complication: the question gives you a **noise density** and a **signal power in dBm** instead of a ready-made  $S/N$ . Two extra lines of work turn it back into the Shannon problem you already know.

#### FORMULA — NOISE POWER FROM NOISE DENSITY

$$N = N_0 \cdot B \quad \text{watts} \quad N_0 \text{ in W/Hz, } B \text{ in Hz}$$

$$N_0 = k T \quad \text{where } k = 1.380649 \times 10^{-23} \text{ J/K}$$

$$\begin{aligned} \text{At room temperature } T \approx 290 \text{ K: } N_0 &\approx 4.0 \times 10^{-21} \text{ W/Hz} \\ &= -174 \text{ dBm/Hz} \end{aligned}$$

In dBm the multiplication becomes an addition:

$$N_{\text{dBm}} = N_{0, \text{dBm/Hz}} + 10 \log_{10}(B)$$

#### FORMULA - DBM BOTH WAYS

$$P_{\text{dBm}} = 10 \log_{10}(P \text{ in watts}) + 30$$

$$P \text{ in watts} = 10^{(P_{\text{dBm}} - 30)/10}$$

#### IN SIMPLE WORDS - WHAT DBM IS

dBm is decibels **relative to one milliwatt**. 0 dBm is 1 mW, 30 dBm is 1 W, -30 dBm is 1  $\mu$ W. Every 10 dB is a factor of ten in power, and every 3 dB is a factor of two.

The reason engineers live in dBm is in the formula above: once everything is in dB, **multiplying becomes adding and dividing becomes subtracting**. Working out an SNR is then a subtraction you can do in your head.

And -174 dBm/Hz is worth memorising outright. It is the thermal noise floor of the universe at room temperature, and every radio link on Earth is measured against it.

#### WORKED EXAMPLE - TUTORIAL 7, NARROWBAND RF

**Question.**  $B = 1 \times 10^6$  Hz,  $N_0 = -174$  dBm/Hz, received signal power  $S = -100$  dBm. Find the channel capacity.

##### 1 Noise power, staying in dBm

$$N_{\text{dBm}} = -174 + 10 \log_{10}(10^6) = -174 + 60$$

$$N = -114 \text{ dBm}$$

##### 2 SNR — a subtraction in dB

$$\text{SNR}_{\text{dB}} = S_{\text{dBm}} - N_{\text{dBm}} = -100 - (-114) = 14.0 \text{ dB}$$

$$\text{as a plain ratio: } S/N = 10^{14/10} = 25.12$$

### 3 Shannon

$$C = B \log_2(1 + S/N) = 10^6 \times \log_2(26.12) = 10^6 \times 4.707$$

$$C \approx 4.71 \text{ Mbit/s}$$

Notice step 2. Because both powers were already in dBm, the ratio S/N became a *subtraction* and no conversion to watts was needed at all. Converting to watts first gives the same 25.12, but with four more chances to slip a decimal point.

#### WORKED EXAMPLE – TUTORIAL 7, THERMAL NOISE FROM KT

**Question.**  $B = 2 \times 10^6$  Hz,  $T = 290$  K,  $S = -50$  dBm. Find C.

##### 1 Noise density from the temperature

$$N_0 = kT = (1.380649 \times 10^{-23})(290) = 4.0039 \times 10^{-21} \text{ W/Hz}$$

##### 2 Noise power

$$N = N_0 B = (4.0039 \times 10^{-21})(2 \times 10^6) = 8.008 \times 10^{-15} \text{ W}$$

$$\approx -110.97 \text{ dBm}$$

##### 3 Signal power in watts, and the ratio

$$S = 10^{(-50-30)/10} = 10^{-8} \text{ W}$$

$$S/N = 10^{-8} / (8.008 \times 10^{-15}) = 1.2488 \times 10^6 \quad (60.96 \text{ dB})$$

##### 4 Capacity

$$C = 2 \times 10^6 \times \log_2(1 + 1.2488 \times 10^6) = 2 \times 10^6 \times 20.25$$

$$C \approx 40.5 \text{ Mbit/s}$$

#### EXAM TIP – THE THREE-LINE DRILL

Every one of these questions is the same three lines. Write them down before you start and you cannot get lost:

1. **N** — from  $N_0B$ , or in dBm from  $N_0 + 10 \log_{10} B$ . If only T is given,  $N_0 = kT$  first.
2. **S/N** — subtract in dB if both are in dBm, then convert with  $10^{\text{dB}/10}$ .
3. **C** —  $B \log_2(1 + S/N)$ .

The one trap: **never put a dB value straight into the Shannon formula**. It wants the plain ratio.

TUTORIAL 7 IN ONE TABLE — CHECK YOUR OWN ANSWERS AGAINST THESE

| CASE                 | B       | S         | N                     | SNR      | C          |
|----------------------|---------|-----------|-----------------------|----------|------------|
| Narrowband RF        | 1 MHz   | -100 dBm  | -114 dBm              | 14.0 dB  | 4.71 Mb/s  |
| Elevated noise floor | 5 MHz   | -90 dBm   | -103.01 dBm           | 13.01 dB | 21.96 Mb/s |
| mmWave class         | 1 GHz   | -50 dBm   | -84 dBm               | 34.0 dB  | 11.30 Gb/s |
| Narrowband IoT       | 200 kHz | -110 dBm  | -120.99 dBm           | 10.99 dB | 0.752 Mb/s |
| Density in W/Hz      | 10 MHz  | 1 $\mu$ W | $5 \times 10^{-14}$ W | 73.01 dB | 242.5 Mb/s |
| Thermal, from kT     | 2 MHz   | -50 dBm   | -110.97 dBm           | 60.96 dB | 40.5 Mb/s  |

#### READ THE TABLE, NOT JUST THE NUMBERS

Compare the mmWave row with the narrowband IoT row. mmWave has **5000 times** the bandwidth and gets **15000 times** the capacity. Bandwidth sits outside the logarithm, so buying more of it pays almost linearly — which is the whole argument for going to millimetre waves, and it is the same lesson as §6.4.

07 FROM THIRD.PDF SLIDES 35-61, AND FOURTH.PDF

## Source coding — making data smaller

Entropy told you the theoretical floor. Source coding is the craft of actually reaching it. This module ends with Huffman coding, which is genuinely optimal — and which you can do by hand in the exam.

### 6.1 What source coding is

Straight from your slide: **source coding is the conversion of the output of a DMS into a sequence of binary symbols.** The device that does it is the **source encoder**.

Its objective, also from your slide: **minimise the average bit rate needed to represent the source, by reducing its redundancy.**

IN SIMPLE WORDS — THE WHOLE TRICK IN ONE LINE

**Give short codes to common symbols, and long codes to rare symbols.**

Morse code figured this out in 1837. In English, “E” is the most common letter, so Morse gave it the shortest possible code: a single dot. “Q” is rare, so it got four symbols: dash-dash-dot-dash. Because you send E far more often than Q, the *average* message gets much shorter.

Compare that to a fixed-length code, where every symbol costs the same number of bits regardless of how often it appears. That wastes bits on the common cases — and the common cases are most of your data.

### 6.2 The four measurements

These four definitions come straight from your slides, and every source coding numerical uses them.

FORMULA — THE FOUR SOURCE CODING QUANTITIES

1. Code word length:  $n_i$  = number of bits in the code word for symbol  $x_i$

2. Average code word length:  $L = \sum P(x_i) n_i$   
bits/symbol

3. Code efficiency:  $\eta = L_{\min} / L = H(X) / L$

#### 4. Code redundancy: $\gamma = 1 - \eta$

##### IN SIMPLE WORDS

$L$  is a weighted average, exactly like entropy was. For each symbol, multiply *how long its code is* by *how often it appears*, then add everything up. It answers: on average, how many bits does one symbol cost me?

$\eta$  (eta) compares your code to the theoretical best. If entropy says you need at least 2.36 bits and your code uses 2.38, then  $\eta = 2.36/2.38 = 0.99$  — you are 99% of the way to perfect. Efficiency is always between 0 and 1; closer to 1 is better.

$\gamma$  (gamma) is simply the wasted fraction. An efficiency of 0.99 means a redundancy of 0.01, or 1% waste.

## 6.3 The source coding theorem

##### FORMULA — SHANNON'S SOURCE CODING THEOREM

$$L \geq H(X)$$

and  $L$  can be made as close to  $H(X)$  as you like with a suitable code, so  $L_{\min} = H(X)$ .

##### IN SIMPLE WORDS — THE HARD FLOOR

This is the answer to information theory's first great question: *how much can I compress?*

**You can never compress a source below its entropy.** If a source has entropy 2.36 bits/symbol, no coding scheme in existence — not ZIP, not any future invention — can represent it in fewer than 2.36 bits per symbol on average. Entropy is a wall.

But the theorem also gives good news: you can get *arbitrarily close* to that wall. There is no gap you are forced to waste. A good code lands within a fraction of a percent.

This is why entropy is the single most important number in the subject. It is not an abstraction — it is the exact size your file will be after perfect compression.

## 6.4 Classification of codes

Your slides give a table of six codes and use it to define seven properties. Learn this table — it is a favourite exam question.

| THE SIX EXAMPLE CODES FROM THIRD.PDF |        |        |        |        |        |        |
|--------------------------------------|--------|--------|--------|--------|--------|--------|
| SYMBOL                               | CODE 1 | CODE 2 | CODE 3 | CODE 4 | CODE 5 | CODE 6 |
| $x_1$                                | 00     | 00     | 0      | 0      | 0      | 1      |
| $x_2$                                | 01     | 01     | 1      | 10     | 01     | 01     |
| $x_3$                                | 00     | 10     | 00     | 110    | 011    | 001    |
| $x_4$                                | 11     | 11     | 11     | 111    | 0111   | 0001   |

| THE SEVEN PROPERTIES, APPLIED TO THOSE CODES |                                                                               |                         |
|----------------------------------------------|-------------------------------------------------------------------------------|-------------------------|
| PROPERTY                                     | MEANING                                                                       | WHICH CODES             |
| <b>Fixed-length</b>                          | Every code word has the same number of bits.                                  | <b>1, 2</b>             |
| <b>Variable-length</b>                       | Code words have different lengths.                                            | <b>3, 4, 5, 6</b>       |
| <b>Distinct</b>                              | No two symbols share the same code word.                                      | <b>All except 1</b>     |
| <b>Prefix-free</b>                           | No code word is the beginning of another code word.                           | <b>2, 4, 6</b>          |
| <b>Uniquely decodable</b>                    | The original message can always be recovered exactly.                         | <b>2, 4, 5, 6</b>       |
| <b>Instantaneous</b>                         | You know a code word has ended the moment it ends, without peeking ahead.     | <b>2, 4, 6</b>          |
| <b>Optimal</b>                               | Instantaneous <i>and</i> has the smallest possible L for these probabilities. | <b>the best of them</b> |

### IN SIMPLE WORDS – WHY CODE 1 FAILS

In Code 1, both  $x_1$  and  $x_3$  are written 00. If you receive 00, you cannot tell which symbol was sent. The information is destroyed. So Code 1 is **not distinct**, and therefore useless.

### IN SIMPLE WORDS – WHY PREFIX-FREE MATTERS SO MUCH

Look at Code 3:  $x_1 = 0$ ,  $x_2 = 1$ ,  $x_3 = 00$ ,  $x_4 = 11$ . Now receive the bits  $0\ 0$ . Is that  $x_3$ ? Or is it  $x_1$  followed by  $x_1$ ? You cannot tell. The trouble is that  $0$  is the *prefix* (the beginning) of  $00$ .

**Prefix-free** means no code word ever begins another. Then the instant you have received a valid code word, you know it is complete — no other symbol could still be coming. You can decode *instantaneously*, bit by bit, with no waiting and no backtracking.

### EXAM TIP – THE CODE 5 SUBTLETY

Your slides make an important point that catches students out. Code 5 is **not** prefix-free ( $0$  is a prefix of  $01$ ), **yet it is still uniquely decodable**.

Why? Look at its code words:  $0, 01, 011, 0111$ . Every one of them starts with a  $0$ , and no other code word contains a  $0$  anywhere else. So a  $0$  acts as a marker meaning “a new code word begins here”. Decode the whole stream by cutting it just before every  $0$ .

So the exam-safe statement is: **prefix-free is a sufficient condition for unique decodability, but not a necessary one**. Prefix-free codes are always uniquely decodable; some non-prefix-free codes happen to be too. But only prefix-free codes are *instantaneous* — Code 5 forces you to wait for the next  $0$  before you can commit.

## 6.5 The Kraft inequality

Suppose someone says: “I want a code with lengths 1, 2, 3 and 3 bits.” Can such an instantaneous code even exist? The Kraft inequality answers exactly that, without you having to try and build it.

### FORMULA – KRAFT INEQUALITY

$$K = \sum_{i=1}^m 2^{-n_i} \leq 1$$

$n_i$  = length of the  $i$ -th code word    •     $m$  = number of symbols

This is a **necessary and sufficient** condition for an instantaneous binary code with those lengths to exist.

### IN SIMPLE WORDS

Think of it as a budget. You start with a total budget of 1. A code word of length  $n_i$  costs you  $2^{-n_i}$  of that budget: a 1-bit code costs 1/2, a 2-bit code costs 1/4, a 3-bit code costs 1/8, and so on. Short code words are expensive, long ones are cheap.

If your total spend comes to 1 or less, the code is buildable. If it comes to more than 1, you have overspent and no such prefix-free code can exist — the code words would be forced to collide.

**Check the example:** lengths 1, 2, 3, 3 give  $1/2 + 1/4 + 1/8 + 1/8 = 1$ . Exactly 1, so yes, it exists (and it is perfectly efficient).

#### EXAM TIP — WHAT KRAFT DOES NOT TELL YOU

Your slide is careful about this, so an examiner may be too. The Kraft inequality:

- tells you an instantaneous code with those lengths **exists**;
- does **not** show you how to find the actual code words;
- does **not** mean that any arbitrary code satisfying it is automatically uniquely decodable.

It is an existence test, not a construction method. To actually build the code, use Shannon–Fano or Huffman.

## 6.6 Entropy coding, method 1 — Shannon–Fano

**Entropy coding** means designing a variable-length code whose average length  $L$  gets as close as possible to  $H(X)$ . Your slides give two methods. Shannon–Fano is the older and simpler one.

#### THE SHANNON–FANO ALGORITHM — FOUR STEPS

1. List the symbols in order of **decreasing** probability.
2. Split the list into two groups whose total probabilities are **as close to equal as possible**. Assign 0 to the upper group and 1 to the lower group.
3. Repeat that splitting inside each group, again and again, until every group has just one symbol left.
4. Read each symbol's code word by collecting the 0s and 1s it collected, left to right.

#### WORKED EXAMPLE — SHANNON–FANO, FROM THIRD.PDF

**Question.** Build a Shannon–Fano code for a source with probabilities 0.30, 0.25, 0.20, 0.12, 0.08, 0.05, then find  $H(X)$ ,  $L$  and  $\eta$ .

### 1 Already in decreasing order — do the first split

Total is 1.00, so aim for two halves of about 0.50 each.

Upper:  $x_1 + x_2 = 0.30 + 0.25 = 0.55 \rightarrow$  assign **0**

Lower:  $x_3 + x_4 + x_5 + x_6 = 0.20 + 0.12 + 0.08 + 0.05 = 0.45 \rightarrow$   
assign **1**

0.55 against 0.45 is the closest split available.

### 2 Split the upper group $\{x_1, x_2\}$

Only one way to divide two symbols:  $x_1$  gets **0**,  $x_2$  gets **1**. Both are now alone, so they are finished.

$x_1 \rightarrow$  **00**       $x_2 \rightarrow$  **01**

### 3 Split the lower group $\{x_3, x_4, x_5, x_6\}$ (total 0.45)

$x_3$  alone = 0.20  $\rightarrow$  assign **0**      (so  $x_3 \rightarrow$  **10**, finished)

$x_4 + x_5 + x_6 = 0.25 \rightarrow$  assign **1**

### 4 Split $\{x_4, x_5, x_6\}$ (total 0.25)

$x_4$  alone = 0.12  $\rightarrow$  assign **0**      (so  $x_4 \rightarrow$  **110**, finished)

$x_5 + x_6 = 0.13 \rightarrow$  assign **1**

### 5 Split the last pair $\{x_5, x_6\}$

$x_5 \rightarrow$  **0** giving **1110**       $x_6 \rightarrow$  **1** giving **1111**

| THE FINISHED SHANNON-FANO CODE |                    |        |        |        |        |      |                |
|--------------------------------|--------------------|--------|--------|--------|--------|------|----------------|
| SYMBOL                         | P(X <sub>i</sub> ) | STEP 1 | STEP 2 | STEP 3 | STEP 4 | CODE | N <sub>i</sub> |
| x <sub>1</sub>                 | 0.30               | 0      | 0      |        |        | 00   | 2              |
| x <sub>2</sub>                 | 0.25               | 0      | 1      |        |        | 01   | 2              |
| x <sub>3</sub>                 | 0.20               | 1      | 0      |        |        | 10   | 2              |
| x <sub>4</sub>                 | 0.12               | 1      | 1      | 0      |        | 110  | 3              |
| x <sub>5</sub>                 | 0.08               | 1      | 1      | 1      | 0      | 1110 | 4              |
| x <sub>6</sub>                 | 0.05               | 1      | 1      | 1      | 1      | 1111 | 4              |

### 1 Find the entropy H(X)

$$\begin{aligned}
 H(X) &= -[0.30 \log_2 0.30 + 0.25 \log_2 0.25 + 0.20 \log_2 0.20 \\
 &\quad + 0.12 \log_2 0.12 + 0.08 \log_2 0.08 + 0.05 \\
 &\quad \log_2 0.05] \\
 &= 0.5211 + 0.5000 + 0.4644 + 0.3671 + 0.2915 + 0.2161
 \end{aligned}$$

$$H(X) = 2.36 \text{ bits/symbol}$$

### 2 Find the average code word length L

$$\begin{aligned}
 L &= \sum P(x_i) n_i \\
 &= 0.30(2) + 0.25(2) + 0.20(2) + 0.12(3) + 0.08(4) + \\
 &\quad 0.05(4) \\
 &= 0.60 + 0.50 + 0.40 + 0.36 + 0.32 + 0.20
 \end{aligned}$$

$$L = 2.38 \text{ bits/symbol}$$

### 3 Find the efficiency

$$\eta = H(X)/L = 2.36 / 2.38$$

$$\eta = 0.99, \text{ so } 99\% \text{ efficient } (\gamma = 1\%)$$

Notice how the code lengths mirror the probabilities: the three common symbols got 2 bits each, and the two rarest got 4 bits each. That is the Morse

principle, applied automatically.

## 6.7 Entropy coding, method 2 — Huffman

Huffman coding does the same job but builds the tree from the *bottom up* instead of the top down. Crucially, your slide states that it produces an **optimal code** — the lowest possible L for those probabilities. No method can beat Huffman.

### THE HUFFMAN ALGORITHM — SIX STEPS FROM YOUR SLIDE

1. List the source symbols in decreasing order of probability.
2. Take the **two smallest** probabilities and add them together. Put the sum back into the list, reordering so it stays in decreasing order. This is called a **reduction**.
3. Repeat step 2 until only **two** probabilities remain.
4. Start at that last reduction. Assign 0 to the first probability and 1 to the second.
5. Work **backwards** through the reductions. Every time you meet a probability you had created by combining two others, split it again and append 0 to one branch and 1 to the other.
6. Read each symbol's code word by **tracing back from the root** to that symbol.

### WORKED EXAMPLE — HUFFMAN, SAME SOURCE (THIRD.PDF + YOUR HANDWRITTEN FOURTH.PDF)

**Question.** Build a Huffman code for probabilities 0.30, 0.25, 0.20, 0.12, 0.08, 0.05.

**Part A — the reductions (combining upwards).** Each time, add the two smallest and re-sort:

| HUFFMAN REDUCTIONS |             |             |             |             |
|--------------------|-------------|-------------|-------------|-------------|
| ORIGINAL           | REDUCTION 1 | REDUCTION 2 | REDUCTION 3 | REDUCTION 4 |
| 0.30               | 0.30        | 0.30        | 0.45        | 0.55        |
| 0.25               | 0.25        | 0.25        | 0.30        | 0.45        |
| 0.20               | 0.20        | 0.25        | 0.25        |             |
| 0.12               | 0.13        | 0.20        |             |             |
| 0.08               | 0.12        |             |             |             |
| 0.05               |             |             |             |             |

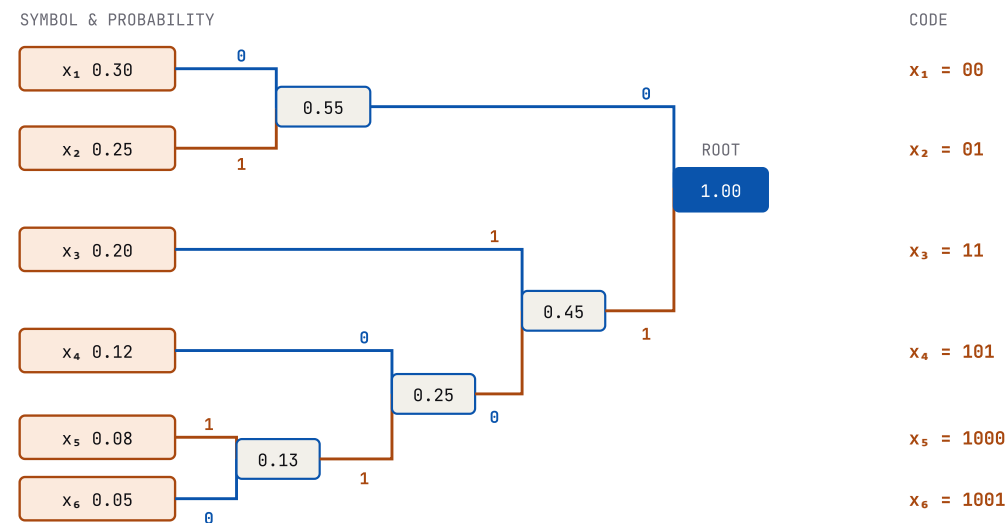
Reduction 1: the two smallest are 0.08 and 0.05, giving 0.13.

Reduction 2: now the two smallest are 0.13 and 0.12, giving 0.25.

Reduction 3: the two smallest are 0.25 and 0.20, giving 0.45.

Reduction 4: the two smallest are 0.30 and 0.25, giving 0.55. Two left — stop.

**Part B — the tree.** Drawn as a tree, the same process looks like this:



The Huffman tree. Blue branches are labelled 0, gold branches are labelled 1. To read a symbol's code, start at the root (1.00) and walk out to the symbol, writing down each branch label as you pass it. This matches the handwritten tree in your fourth.pdf, including its worked codes  $x_1 = 00$  and  $x_5 = 1000$ .

## WORKED EXAMPLE CONTINUED – HUFFMAN RESULTS

| THE FINISHED HUFFMAN CODE |          |      |       |              |
|---------------------------|----------|------|-------|--------------|
| SYMBOL                    | $P(X_i)$ | CODE | $N_i$ | $P \times N$ |
| $x_1$                     | 0.30     | 00   | 2     | 0.60         |
| $x_2$                     | 0.25     | 01   | 2     | 0.50         |
| $x_3$                     | 0.20     | 11   | 2     | 0.40         |
| $x_4$                     | 0.12     | 101  | 3     | 0.36         |
| $x_5$                     | 0.08     | 1000 | 4     | 0.32         |
| $x_6$                     | 0.05     | 1001 | 4     | 0.20         |
| L = total                 |          |      |       | 2.38         |

$$H(X) = 2.36 \text{ bits/symbol} \quad L = 2.38 \text{ bits/symbol}$$

$$\eta = 2.36/2.38$$

$$\eta = 0.99 \quad \cdot \quad \gamma = 0.01$$

#### EXAM TIP – HUFFMAN CODES ARE NOT UNIQUE

Your two answers may differ from a friend's and *both be correct*. Three things are free choices:

- Which branch you call 0 and which you call 1 (swapping them everywhere just flips all the bits).
- How you break ties when two probabilities are equal.
- Where you place a merged probability among equal values when re-sorting.

What is **not** free is **L**. Every correct Huffman code for the same source has the same average length. So if your L matches, your answer is right even if the individual code words differ. Always state L, H(X) and  $\eta$  at the end — that is where the marks are.

### Shannon–Fano versus Huffman

| COMPARING THE TWO METHODS |                         |                                 |
|---------------------------|-------------------------|---------------------------------|
|                           | SHANNON–FANO            | HUFFMAN                         |
| Direction                 | Top-down (split)        | Bottom-up (combine)             |
| Optimal?                  | Not always              | <b>Always optimal</b>           |
| Ease by hand              | Slightly easier         | Slightly longer, but mechanical |
| This example              | L = 2.38, $\eta$ = 0.99 | L = 2.38, $\eta$ = 0.99         |

In this particular source both methods happen to tie. That is common for well-behaved probabilities. On less friendly sources, Shannon–Fano can fall short while Huffman never does — which is exactly why Huffman is the one used inside JPEG, MP3 and ZIP.

## 6.8 Redundancy

Your slides define **redundancy** as the reduction in information content of a message from its maximum value, and give a lovely example.

### IN SIMPLE WORDS – THE ENGLISH ALPHABET EXAMPLE

English has 26 letters. If all 26 were equally likely, each letter would carry  $\log_2 26 = 4.7$  bits.

But they are not equally likely. E and T are everywhere; Z and Q are rare. Because the probabilities are lopsided, the actual information content is *less* than 4.7 bits per letter. That gap is redundancy.

And real English has even more structure than that: after “Q” comes “U”, after “TH” comes “E”. Counting all of it, English carries only about 1 to 1.5 bits per letter. Which is why a text file compresses to roughly a quarter of its size, and why *you can still read this sentence*.

Redundancy is not always bad. It is why you can still understand someone on a bad phone line. Source coding strips it out for efficiency; channel coding (Module 8) then puts a controlled amount back for reliability.

## 6.9 Rate distortion theory

Everything so far assumed **lossless** coding: you get back exactly what you put in. But sometimes that is impossible, or simply not worth it.

### IN SIMPLE WORDS – WHY LOSSY CODING IS UNAVOIDABLE

Your slide gives the clearest case: **a continuous-amplitude source such as speech**. Its true amplitude is a real number with infinite decimal places. To represent it with a finite code word you *must* quantize — and quantizing throws information away, permanently.

So the problem becomes **“source coding with a fidelity criterion”**: not “how do I compress perfectly”, but “how small can I get it if I am allowed to be a little bit wrong?” The branch of information theory that answers this is **Rate Distortion Theory (RDT)**.

Your slide names the two situations where RDT applies:

1. The coding alphabet cannot represent the source exactly, forcing **lossy data compression**.
2. You want to transmit at a rate **greater than the channel capacity** — so something must be sacrificed.

### FORMULA – THE RATE DISTORTION FUNCTION

$$R(D) = \min_{P(y_j|x_i) \in P_D} I(X;Y)$$

subject to  $\sum_j P(y_j|x_i) = 1$  ·  $P_D$  = the set of conditional probabilities giving distortion no worse than D

In words, straight from your slide:  **$R(D)$  is the smallest coding rate possible for which the average distortion is guaranteed not to exceed D.**

#### IN SIMPLE WORDS

$R(D)$  is the **price list** for quality. Feed in how much error you are willing to tolerate (D), and it returns the fewest bits per symbol you need (R).

The relationship is a see-saw, and this is the sentence to remember: **as the allowed distortion D goes up, the required rate  $R(D)$  goes down.** Tolerating more error lets you use fewer bits.

You use this every day. A 320 kbps MP3 sounds excellent; a 64 kbps MP3 sounds tinny but is five times smaller. Netflix drops your video quality on a weak connection. Both are moving along the  $R(D)$  curve, trading fidelity for rate.

## 6.10 Data compression

Finally, your slides close the topic with two definitions worth quoting exactly:

#### TWO DEFINITIONS FROM THIRD.PDF

**Data compression** is a purposeful or unavoidable reduction in the information content of data from a continuous or discrete source.

A **data compressor** is a device that supplies a code with the least number of symbols for the representation of the source output, subject to permissible or acceptable distortion.

Note the two words “purposeful or unavoidable”. Sometimes you compress because you *chose* to save space (purposeful). Sometimes you compress because the source is continuous and finite codes simply cannot hold it (unavoidable). And “subject to permissible distortion” is the RDT idea again: the compressor is only allowed to be as wrong as you have said it may be.

EXAM TIP – LOSSLESS VS LOSSY IN ONE LINE EACH

**Lossless** (Huffman, Shannon–Fano, ZIP): perfect reconstruction, limited by  $L \geq H(X)$ . Used for text, code, spreadsheets — anywhere a single wrong bit matters.

**Lossy** (quantization, JPEG, MP3): imperfect reconstruction, governed by  $R(D)$ . Used for images, audio, video — anywhere human senses will not notice small errors.



## Channel coding — fighting the noise

Module 7 made the data as small as possible. Now we deliberately make it bigger again — but in a very clever way, so the receiver can repair the damage the channel does. This is the largest topic in your materials.

### 7.1 The basic idea

Your slide states it in three words: **introducing redundancy**.

#### FORMULA — WHAT THE CHANNEL ENCODER DOES

$k$  bits in  $\rightarrow$  channel encoder  $\rightarrow$   $n$  bits out, where  $n > k$

the extra  $(n - k)$  bits are called **redundant bits** or **parity bits**

#### IN SIMPLE WORDS

You put in 4 bits and get out 7. The three extra bits carry no new message — they are calculated from the original 4 by a fixed rule. If noise damages the block on the way, the extra bits will no longer agree with the rest, and that disagreement tells the receiver both *that* something broke and often *which bit* broke.

Think of a parcel. The item is your data. The bubble wrap is the redundancy. The bubble wrap is not the thing you wanted to send, it costs you space and postage, but it means the item survives the journey.

Your slide is honest about the price. The **two disadvantages of channel encoding** are:

1. **Bandwidth goes up.** You are sending  $n$  bits where  $k$  would have done, so you need more channel capacity or more time.
2. **Complexity and cost go up.** Encoders and especially decoders need real computation.

Why accept that? Because, as your slide says, of the **unpredictable nature of the wireless channel especially — there is a large probability of bit errors**. Without coding, modern mobile data would be unusable.

**Channel decoding**, then, is simply: undo the effect of channel encoding — strip the parity bits back off, after using them to fix any damage.

#### THE SIMPLEST POSSIBLE EXAMPLE, FROM YOUR SLIDE

Suppose we want to transmit **1011**. A simple scheme adds one parity bit:

**1011** → **10110**

The message **1011** contains three 1s, which is an odd count. The appended **0** keeps the total odd, so this is an **odd parity** scheme. (Had the scheme been even parity, the appended bit would have been a 1, giving **10111**.)

Either way the receiver does the same thing: count the 1s and check against the agreed rule. If the count breaks the rule, an error occurred. **Here redundancy is added purposefully.**

## 7.2 The two families: FEC and ARQ

Your slide splits all channel coding into exactly two mechanisms. This is a guaranteed exam question.

### FEC

#### Forward Error Correction

A digital signal processing technique used to **enhance data reliability**.

The receiver fixes the errors itself, using the parity bits. Nothing is sent back to the transmitter.

Two sub-types:

- Block error codes
- Convolutional error codes

### ARQ

#### Automatic Repeat reQuest

A protocol for **error control** in data transmission, used to **enhance data integrity**.

The receiver only *detects* errors, then asks the transmitter to send that block again.

### IN SIMPLE WORDS – WHEN TO USE WHICH

**FEC is like sending a letter with the same information written three times.** If one copy smudges, the reader still works it out alone. No reply needed. Use FEC when you cannot ask again: live video, satellite links from deep space, broadcast TV, or anywhere the round trip would take too long.

**ARQ is like a phone call where you say “sorry, say that again?”** Simpler and it only costs you when errors actually happen. Use ARQ when a return path exists and delay is acceptable: file downloads, web pages, email. TCP on the internet is ARQ.

Real systems such as 4G and 5G use **both** together (called hybrid ARQ): FEC fixes the small stuff instantly, and ARQ handles the rare cases FEC could not repair.

## 7.3 Error detection versus error correction

Your slide lists the two major purposes of channel coding, with examples of each. Learn both lists.

| THE TWO PURPOSES OF CHANNEL CODING |                                                                                    |                                                               |
|------------------------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------|
| PURPOSE                            | WHAT THE RECEIVER CAN DO                                                           | EXAMPLE CODES                                                 |
| <b>Error detection</b>             | Determines that an error <i>has occurred</i> .                                     | Parity-check codes, Cyclic Redundancy Check (CRC), Checksum   |
| <b>Error correction</b>            | Determines that an error occurred <b>and identifies the likely original data</b> . | Hamming, BCH, Reed-Solomon, Convolutional, Turbo, LDPC, Polar |

### IN SIMPLE WORDS

Detection says “*something is wrong here.*” Correction says “*something is wrong here, and here is what it should have been.*”

Correction is far more useful but needs more redundancy. Detection is cheap, which is why it pairs naturally with ARQ — detect the problem, then just ask again.

## 7.4 Codeword and code rate

### FORMULA – CODE RATE

**Codeword** = the encoded sequence transmitted through the channel

$$R = k / n$$

$k$  = number of information bits ·  $n$  = total transmitted bits

Example: a (7,4) code has  $R = 4/7 \approx 0.571$

#### CORRECTION – A MISPRINT ON SLIDE 8 OF YOUR PDF

Your Channel Coding slide 8 prints the code rate as  $R = n/k$  and says a (7,4) code has  $R = 7/4$ . **This is inverted.** The standard definition used in every textbook, and in the rest of your own slide deck, is:

$$R = k/n, \text{ so a (7,4) code has } R = 4/7 \approx 0.571$$

You can prove the slide is inconsistent using its own next sentence: it says “a lower code rate means more redundancy”. That is only true for  $k/n$ . With  $k/n$ , a rate of  $1/3$  means 1 information bit per 3 sent — lots of redundancy. With  $n/k$ , a “lower” rate would mean less redundancy, contradicting the sentence.

Also note that a code rate is a fraction of useful bits, so it must always be **between 0 and 1**. A rate of  $7/4 = 1.75$  would mean getting out more information than you put in, which is impossible.

**In the exam, write  $R = k/n$ .** If your lecturer insists on the slide version, answer with  $k/n$  and add one line noting the standard convention — that is defensible and shows you understand it.

#### IN SIMPLE WORDS

Code rate answers: **what fraction of what I send is actual message?**

$R = 4/7$  means: of every 7 bits on the wire, 4 are your data and 3 are protection. About 57% useful.

$R = 1/2$  means half your transmission is protection.  $R = 1/3$  means two thirds is protection — very heavy armour, used for deep-space probes where a retransmission would take hours.

**The trade-off:** lower code rate → more redundancy → better error protection → but more bits to send, so lower useful throughput.

## 7.5 Hamming distance

Your slide calls this **one of the most important concepts in coding theory**, and it is right.

### FORMULA - HAMMING DISTANCE

$d(a,b)$  = the number of positions in which two codewords differ

Example from your slide:

101101

100001

↑↑ they differ in 2 positions, so  $H_D = 2$

### IN SIMPLE WORDS

Line the two codewords up and count how many columns disagree. That is the Hamming distance. A quick way to compute it: XOR the two words together and count the 1s.

Now the key idea: the **minimum Hamming distance**  $d_{\min}$  of a code is the smallest distance between *any* two valid codewords in it. It is the code's weakest link, and it decides everything the code can do.

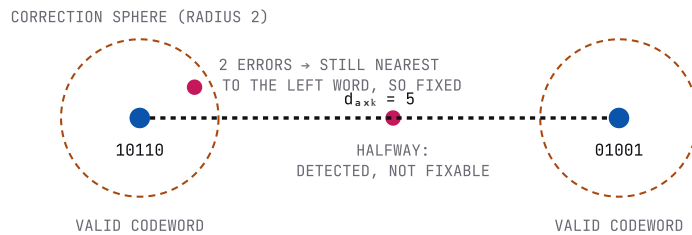
### FORMULA - WHAT $d_{\min}$ BUYS YOU

Can **detect** up to  $d_{\min} - 1$  errors

Can **correct** up to  $\lfloor (d_{\min} - 1)/2 \rfloor$  errors

$\lfloor \rfloor$  means round down to the nearest whole number

Your slide's example: if  $d_{\min} = 5$ , the code can detect up to 4 errors and correct up to 2 errors.



Why the formulas work. Picture every valid codeword as a dot with a sphere around it. Errors push the received word away from its dot. If it stays inside its own sphere, the decoder snaps it back – corrected. If it lands between spheres, the decoder knows it is invalid but cannot say which dot it came from – detected only.

#### IN SIMPLE WORDS – WHERE THE TWO FORMULAS COME FROM

**Detection:** if two valid codewords are 5 apart, then changing up to 4 bits can never turn one valid codeword into another valid one. So any 1-to-4 bit error lands on an invalid word, and the receiver spots it. At 5 errors you could land exactly on a different valid codeword — and then you would never know.

**Correction:** the decoder repairs by choosing the *nearest* valid codeword. With distance 5, if 2 bits flip you are still closer to the original (2 away) than to the other (3 away), so you get corrected. If 3 bits flip, you are now closer to the wrong one, and the decoder confidently corrects to the wrong answer. So 2 is the limit — which is  $\lfloor (5-1)/2 \rfloor = 2$ .

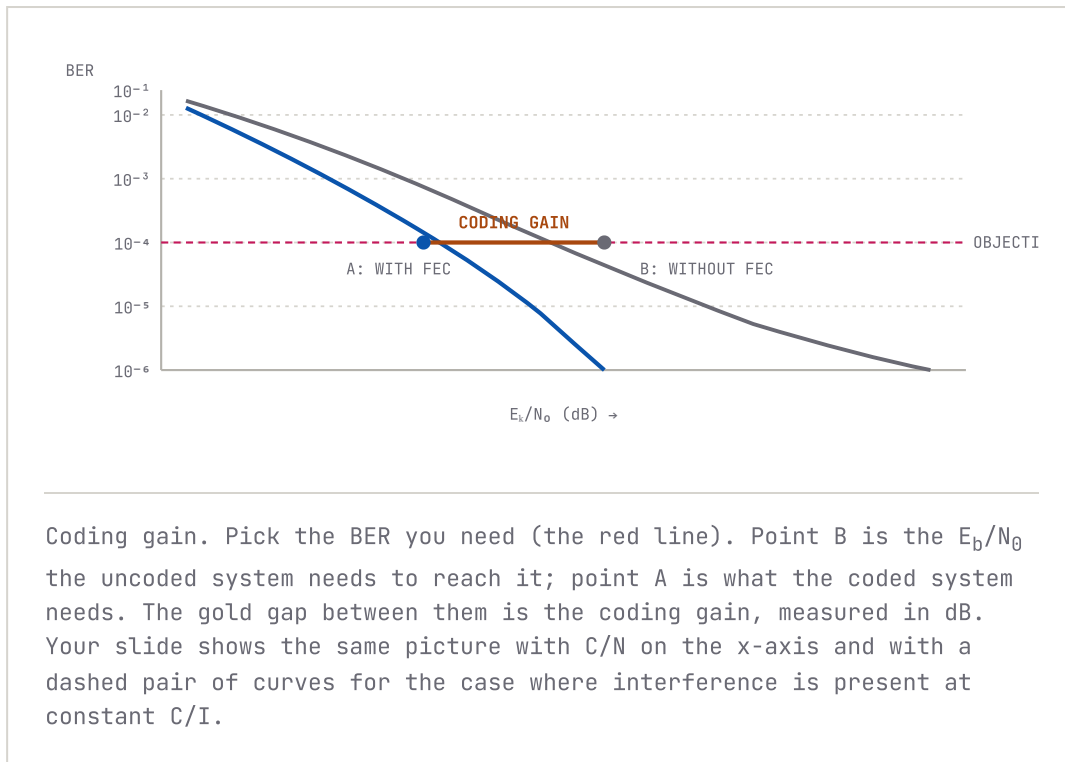
#### EXAM TIP

Quick table worth memorising:

| $D_{\text{MIN}}$ | DETECT | CORRECT | COMMENT              |
|------------------|--------|---------|----------------------|
| 1                | 0      | 0       | Useless              |
| 2                | 1      | 0       | Single parity bit    |
| 3                | 2      | 1       | Hamming codes        |
| 4                | 3      | 1       | Extended Hamming     |
| 5                | 4      | 2       | Your slide's example |

## 7.6 Coding gain

Your slide has two performance plots. They both show the same thing, and it is the reason channel coding is worth its cost.



### IN SIMPLE WORDS

**Coding gain is how much transmit power you are allowed to save by using a code instead of brute force.**

Suppose a code gives 3 dB of coding gain. 3 dB is a factor of 2 in power. So you could either transmit at full power without coding, or **halve your transmit power** and use the code, and get exactly the same BER. For a phone, that is double the battery life. For a satellite, it is a smaller, cheaper spacecraft.

The slide also notes the x-axis can be labelled C/N, S/N, or  $E_b/N_0$  — they all express the same signal-versus-noise idea.

## 7.7 Error detection methods: VRC, LRC, CRC, checksum

Your slide gives a tree of four detection methods.

### VRC and LRC

#### IN SIMPLE WORDS

**VRC = Vertical Redundancy Check.** One parity bit added to each *character* (each row). Also just called a parity check.

**LRC = Longitudinal Redundancy Check.** A whole extra *character* added at the end of a block, where each of its bits is the parity of one *column* across all the characters. Your slide calls this the **block check character**.

Used together, they form a grid: parity on every row *and* every column. A single flipped bit then breaks exactly one row check and exactly one column check — and where they cross is the broken bit. So VRC + LRC together can actually *correct* a single error, not merely detect it.

#### WORKED EXAMPLE – THE VRC/LRC TABLE FROM YOUR SLIDE (EVEN PARITY)

SEVEN DATA BITS PER CHARACTER, PLUS A VRC BIT; THE LRC ROW IS THE COLUMN PARITY

| CHAR | DATA BITS (1-7) | VRC | CHECK                         |
|------|-----------------|-----|-------------------------------|
| A    | 1000001         | 0   | two 1s → already even → VRC 0 |
| B    | 0100001         | 0   | two 1s → even → 0             |
| C    | 1100001         | 1   | three 1s → odd → add 1        |
| D    | 0010001         | 0   | two 1s → even → 0             |
| E    | 1010001         | 1   | three 1s → odd → add 1        |
| F    | 0110001         | 1   | three 1s → odd → add 1        |
| G    | 1110001         | 0   | four 1s → even → 0            |
| LRC  | 0000001         | 1   | parity of each column         |

**How the LRC row was built.** Take column 1 down the seven characters: 1,0,1,0,1,0,1 — that is four 1s, which is even, so the LRC bit for column 1 is 0. Column 7 reads 1,1,1,1,1,1,1 — seven 1s, odd, so its LRC bit is 1. Do that for all seven columns and you get 0000001.

## CRC — Cyclic Redundancy Check

CRC is the workhorse of real systems — Ethernet, Wi-Fi, USB, hard drives all use it. The idea is binary division.

#### IN SIMPLE WORDS – THE CRC RECIPE

1. Agree on a **divisor** (also called the generator polynomial). If it is  $r+1$  bits long, you will produce an  $r$ -bit checksum.
2. Append  $r$  zeros to the end of your message.
3. Divide that by the divisor using **modulo-2 division** — which is ordinary long division except every subtraction is replaced by **XOR** (no borrowing, ever).
4. The **remainder** is your CRC. Replace the appended zeros with it. That is the codeword you transmit.
5. The receiver divides the whole received codeword by the same divisor. Remainder zero means no error detected; any other remainder means the data is corrupt.

**XOR rule reminder:**  $0 \oplus 0 = 0$ ,  $1 \oplus 1 = 0$ ,  $0 \oplus 1 = 1$ ,  $1 \oplus 0 = 1$ . Same bits give 0, different bits give 1.

#### WORKED EXAMPLE – THE CRC FROM YOUR SLIDE

**Given.** Message  $M(x) = 11001011$ , Divisor  $= 1011$ .

##### 1 Append zeros

The divisor is 4 bits, so  $r = 3$ . Append three 0s:

$11001011 \rightarrow 11001011000$

## 2 Do the modulo-2 division (XOR at each stage)

11001011000

1011          ← XOR

\_\_\_\_\_

1111011000

1011

\_\_\_\_\_

100011000

1011

\_\_\_\_\_

1111000

1011

\_\_\_\_\_

100000

1011

\_\_\_\_\_

1100

1011

\_\_\_\_\_

Remainder = **111**

## 3 Build the codeword

Replace the three appended zeros with the remainder **111** :

**Code Word = 11001011111**

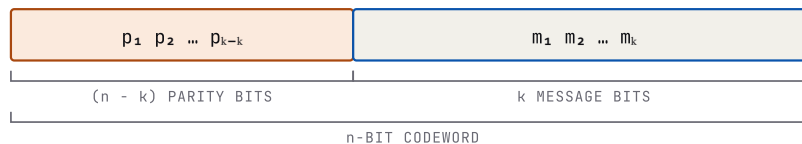
**Checking at the receiver.** Divide `11001011111` by `1011` using the same XOR method. If the remainder is `000`, the data passed. If it is anything else, an error is present.

## 7.8 Linear block codes

Now we move from detection to correction. Your slide describes the block code process in four lines:

1. The information bit stream is divided into blocks of  $k$  bits.
2. Each block is encoded to a larger block of  $n$  bits.
3. The coded bits are modulated and sent over the channel.
4. The reverse procedure is done at the receiver.

FORM 1 - PARITY FIRST (AS ON YOUR SLIDE)



FORM 2 - MESSAGE FIRST (SYSTEMATIC): [  $k$  MESSAGE BITS |  $(n-k)$  PARITY BITS ] - BOTH ARE VALID

The structure of a codeword. Both orderings appear in your slide. What matters is that the message bits survive unchanged inside the codeword – a code with that property is called **systematic**, and it makes decoding much easier because you can simply read the message straight out.

## 7.9 Hamming codes

Hamming codes are the classic single-error-correcting block codes, named after Richard W. Hamming. Your slide gives their full specification:

| BINARY HAMMING CODES – CLASSIFICATION FROM YOUR SLIDE |                                         |
|-------------------------------------------------------|-----------------------------------------|
| PROPERTY                                              | VALUE                                   |
| Type                                                  | Linear block code                       |
| Block length $n$                                      | $2^r - 1$ , where $r \geq 2$            |
| Message length $k$                                    | $2^r - r - 1$                           |
| Rate                                                  | $1 - r/(2^r - 1)$                       |
| Minimum distance                                      | <b>3</b> → detects 2 errors, corrects 1 |
| Alphabet size                                         | 2 (binary)                              |
| Notation                                              | $[2^r - 1, 2^r - r - 1, 3]_2$ -code     |

Put  $r = 3$  into those formulas:  $n = 2^3 - 1 = 7$  and  $k = 2^3 - 3 - 1 = 4$ . That gives the famous **Hamming(7,4)** code — 4 message bits, 3 parity bits, 7 bits total.

## Method 1 — the position method

### IN SIMPLE WORDS – HOW THE POSITION TRICK WORKS

Number the bit positions 1, 2, 3, 4, ... from the right. Put **parity bits at the power-of-two positions**: 1, 2, 4, 8, 16, ... Put **data bits everywhere else**: 3, 5, 6, 7, 9, 10, ...

Now each parity bit checks the positions whose number contains its own bit:

- $r_1$  (position 1) checks positions 1, 3, 5, 7, 9, 11, ...
- $r_2$  (position 2) checks positions 2, 3, 6, 7, 10, 11, ...
- $r_4$  (position 4) checks positions 4, 5, 6, 7, 12, 13, ...
- $r_8$  (position 8) checks positions 8, 9, 10, 11, 12, 13, 14, 15

**Why this is brilliant:** if a single bit gets flipped, add up the position numbers of the parity checks that fail — and that sum is *the position of the broken bit*. The code tells you exactly where the damage is.

### WORKED EXAMPLE – THE HAMMING CONSTRUCTION FROM YOUR SLIDE

**Given.** Data 1001101 (7 data bits). With 7 data bits we need 4 parity bits, giving an 11-bit codeword.

### 1 Lay the data into the non-power-of-two positions

Positions 11, 10, 9, 7, 6, 5, 3 get the data bits; positions 8, 4, 2, 1 are reserved for parity.

pos: 11 10 9 8 7 6 5 4 3 2 1

bit: 1 0 0 \_ 1 1 0 \_ 1 \_ \_

### 2 Compute each parity bit in turn

Your slide adds them one at a time —  $r_1$ , then  $r_2$ , then  $r_4$ , then  $r_8$  — each chosen to make its own group have even parity. The slide's results are:

$$r_1 = 1 \cdot r_2 = 0 \cdot r_4 = 0 \cdot r_8 = 1$$

### 3 Read off the finished codeword

Code = 10011100101

Your slide also shows the extension to longer messages: with positions  $C_1$  to  $C_{31}$ , the parity bits  $p_1 \dots p_5$  sit at  $C_1, C_2, C_4, C_8, C_{16}$  (that is  $2^0$  to  $2^4$ ) and data bits  $d_1 \dots d_{26}$  fill everything else. The pattern never changes — parity always lives at the powers of two.

## Method 2 — the XOR equations (hardware view)

FORMULA — HAMMING(7,4) PARITY EQUATIONS FROM YOUR SLIDE

$$P_1 = D_1 \oplus D_2 \oplus D_3$$

$$P_2 = D_2 \oplus D_3 \oplus D_4$$

$$P_3 = D_1 \oplus D_3 \oplus D_4$$

codeword sent as [  $D_1 D_2 D_3 D_4 P_1 P_2 P_3$  ]

In hardware this is just three XOR gates. Your slide notes: “as per fan-in possible we can develop the hardware, else programming is the better option for Hamming code.” In other words, for small codes build gates; for large ones, use software.

## Method 3 — the generator matrix

### FORMULA — GENERATOR MATRIX ENCODING

$$C = d G$$

$C$  = codeword ·  $d$  = data vector ·  $G$  = generator matrix  
 · all arithmetic is modulo 2

$G = [ I \mid P ]$  — an identity submatrix beside a parity submatrix

### WORKED EXAMPLE — $C = DG$ FROM YOUR SLIDE

**Given.**  $d = [1 \ 0 \ 1]$  and

$$G = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

identity part          parity part

#### 1 Multiply, taking rows of $G$ where $d$ has a 1

$d = [1 \ 0 \ 1]$  selects **row 1** and **row 3** of  $G$ . Add them modulo 2 (XOR):

$$\text{row 1} = 1 \ 0 \ 0 \ 1 \ 0 \ 1$$

$$\text{row 3} = 0 \ 0 \ 1 \ 1 \ 1 \ 0$$

$$\text{XOR} = 1 \ 0 \ 1 \ 0 \ 1 \ 1$$

#### 2 Read the codeword

$$C = [1 \ 0 \ 1 \ 0 \ 1 \ 1]$$

**Notice the systematic structure.** The first three bits of  $C$  are **101** — exactly the message. That is what the identity submatrix does: it copies the message through unchanged. The parity submatrix generates the last three check bits. This is why the code is called systematic.

| ALL 16 CODEWORDS OF THE (7,4) HAMMING CODE — FROM YOUR SLIDE |            |           |            |
|--------------------------------------------------------------|------------|-----------|------------|
| MESSAGE M                                                    | CODEWORD C | MESSAGE M | CODEWORD C |
| 0000                                                         | 0000000    | 1000      | 1000101    |
| 0001                                                         | 0001011    | 1001      | 1001110    |
| 0010                                                         | 0010111    | 1010      | 1010010    |
| 0011                                                         | 0011100    | 1011      | 1011001    |
| 0100                                                         | 0100110    | 1100      | 1100011    |
| 0101                                                         | 0101101    | 1101      | 1101000    |
| 0110                                                         | 0110001    | 1110      | 1110100    |
| 0111                                                         | 0111010    | 1111      | 1111111    |

#### EXAM TIP — USE THIS TABLE TO PROVE $d_{\min} = 3$

Pick any two rows and count the differing positions — you will never find a pair closer than 3 apart. A faster route: for a *linear* code,  $d_{\min}$  equals the smallest number of 1s in any non-zero codeword. Scan the table: 0110001 has three 1s, and nothing non-zero has fewer. Hence  $d_{\min} = 3$ , so it detects 2 errors and corrects 1 — exactly as the classification table stated.

## 7.10 Convolutional codes

Block codes chop the data into separate blocks. **Convolutional codes** do something different: they slide a window along a continuous stream, so each output bit depends on the current input *and several previous ones*. The encoder has memory.

#### FORMULA — CONVOLUTIONAL ENCODER STRUCTURE

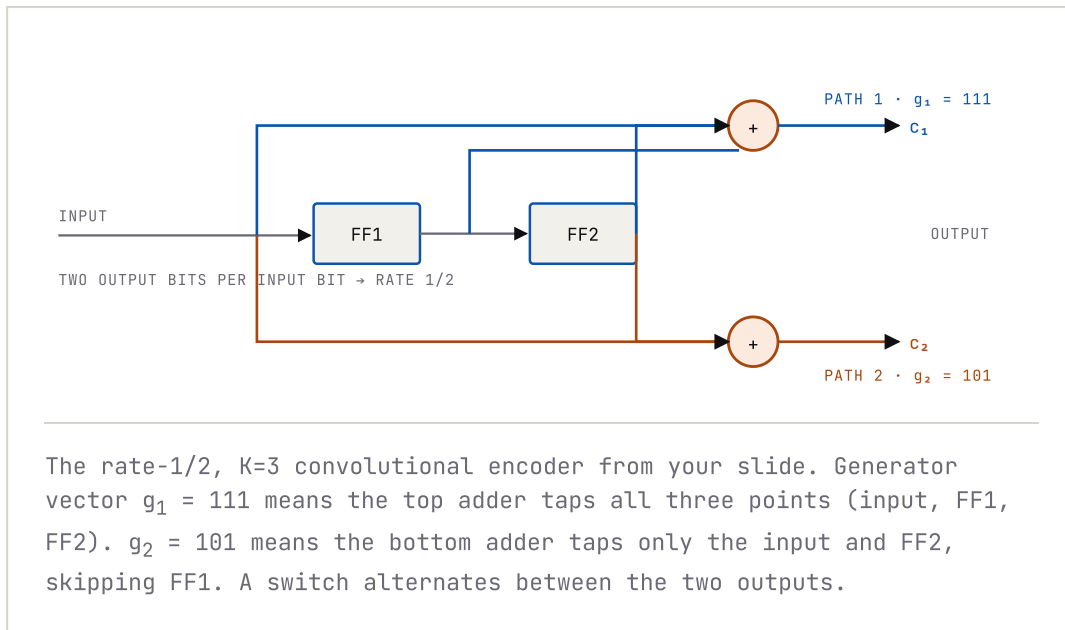
Code rate  $r = k/n$ , built from  $K - 1$  shift registers (flip-flops) and  $n$  modulo-2 adders

$K$  = constraint length · plus an input multiplexer and an output multiplexer

Your slide's example:  $K = 3$ , rate =  $1/2$ , generator vectors  $g_1 = 111$  and  $g_2 = 101$

$$g_1(D) = 1 \cdot D^0 + 1 \cdot D^1 + 1 \cdot D^2 = \mathbf{1 + D + D^2}$$

$$g_2(D) = 1 \cdot D^0 + 0 \cdot D^1 + 1 \cdot D^2 = \mathbf{1 + D^2}$$



#### IN SIMPLE WORDS — READING A GENERATOR VECTOR

A generator vector is simply a **list of which taps are connected**. Read it left to right as [input, FF1, FF2]:

$g_1 = 111 \rightarrow$  connect all three  $\rightarrow c_1 = \text{input} \oplus \text{FF1} \oplus \text{FF2}$

$g_2 = 101 \rightarrow$  connect the first and third only  $\rightarrow c_2 = \text{input} \oplus \text{FF2}$

The **constraint length K** is how many input bits influence any one output bit. Here  $K = 3$ : the current bit plus the two stored in the flip-flops.

The **D** in the polynomial form just means “delay”.  $D^0$  is now,  $D^1$  is one step ago,  $D^2$  is two steps ago. Writing it as a polynomial lets you encode by multiplying polynomials, which is Method 1 below.

### Method 1 — encoding by polynomial multiplication

#### WORKED EXAMPLE — ENCODE 10011, FROM YOUR SLIDE

### 1 Write the message as a D-polynomial

Message (10011), reading position by position:

$$\begin{aligned} m(D) &= 1 \cdot D^0 + 0 \cdot D^1 + 0 \cdot D^2 + 1 \cdot D^3 + 1 \cdot D^4 \\ &= 1 + D^3 + D^4 \end{aligned}$$

### 2 Multiply by $g_1(D)$ to get path 1

$$\begin{aligned} c_1(D) &= (1 + D + D^2)(1 + D^3 + D^4) \\ &= 1 + D + D^2 + D^3 + D^6 \end{aligned}$$

Read the coefficients for  $D^0$  through  $D^6$ : present, present, present, present, absent, absent, present.

**Path 1 output = 1111001**

### 3 Multiply by $g_2(D)$ to get path 2

$$\begin{aligned} c_2(D) &= (1 + D^2)(1 + D^3 + D^4) \\ &= 1 + D^2 + D^3 + D^4 + D^5 + D^6 \end{aligned}$$

**Path 2 output = 1011111**

### 4 Interleave the two outputs (multiplex)

Take one bit from path 1, then one from path 2, alternately:

path 1: 1 1 1 1 0 0 1

path 2: 1 0 1 1 1 1 1

**c = (11, 10, 11, 11, 01, 01, 11)**

**Important note on modulo-2 multiplication.** When multiplying these polynomials, any term that appears an *even* number of times cancels to zero (because  $D^k + D^k = 0$  in modulo 2). This is why the products come out shorter than you might expect.

## Method 2 — encoding with a table

Your slide gives the same result a second way, by tracking the flip-flop contents clock tick by clock tick. This method is slower but almost impossible to get wrong, so it is the safer one in an exam.

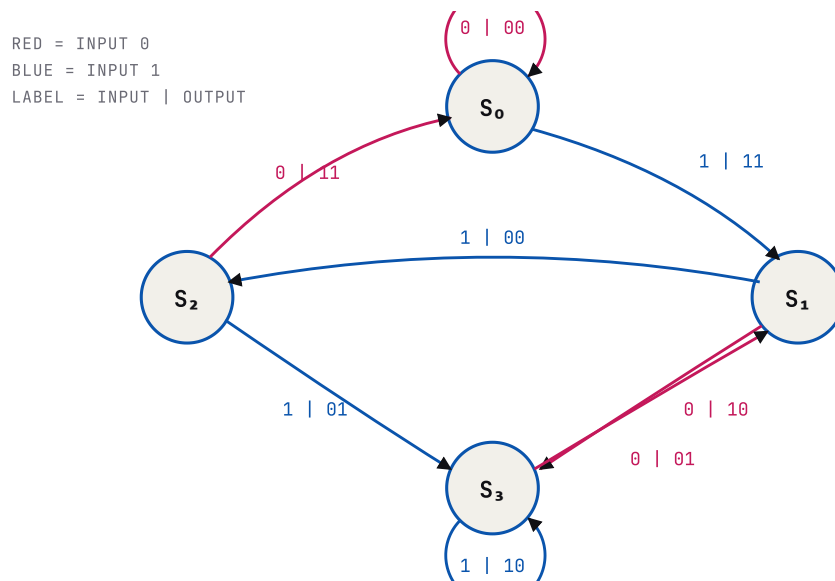
| CIRCUIT ANALYSIS TABLE FOR MESSAGE 10011 — FROM YOUR SLIDE |       |     |     |           |           |              |
|------------------------------------------------------------|-------|-----|-----|-----------|-----------|--------------|
| CLOCK                                                      | INPUT | FF1 | FF2 | $G_1=111$ | $G_2=101$ | CODED OUTPUT |
| t0                                                         | 0     | 0   | 0   | 0         | 0         | 00           |
| t1                                                         | 1     | 0   | 0   | 1         | 1         | 11           |
| t2                                                         | 0     | 1   | 0   | 1         | 0         | 10           |
| t3                                                         | 0     | 0   | 1   | 1         | 1         | 11           |
| t4                                                         | 1     | 0   | 0   | 1         | 1         | 11           |
| t5                                                         | 1     | 1   | 0   | 0         | 1         | 01           |
| t6                                                         | 0     | 1   | 1   | 0         | 1         | 01           |
| t7                                                         | 0     | 0   | 1   | 1         | 1         | 11           |
| t8                                                         | 0     | 0   | 0   | 0         | 0         | 00           |

Reading the output column from t1 to t7 gives 11, 10, 11, 11, 01, 01, 11 — exactly the same answer as the polynomial method. Two independent routes, one answer. That is a good confidence check in the exam.

## 7.11 State diagram, tree and trellis

Because the encoder has memory, its behaviour is best described as a **state machine**. The “state” is simply what is currently sitting in the flip-flops.

| STATE DEFINITIONS — FROM YOUR SLIDE |                                  |
|-------------------------------------|----------------------------------|
| STATE                               | BINARY ( $U_{I-1}$ , $U_{I-2}$ ) |
| $S_0$                               | 0 0                              |
| $S_1$                               | 1 0                              |
| $S_2$                               | 0 1                              |
| $S_3$                               | 1 1                              |



The state diagram from your slide. From every state there are exactly two ways out – one for input 0 (red) and one for input 1 (blue) – and each is labelled with the two output bits it produces. Encoding is just walking this graph.

#### WORKED EXAMPLE – WALKING THE STATE DIAGRAM, FROM YOUR SLIDE

Starting in  $S_0$  and feeding in the bits 1,1,1,0,0,1,0,1:

| TIME $i$     | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     |
|--------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| Input $u_i$  | 1     | 1     | 1     | 0     | 0     | 0     | 1     | 0     | 1     |
| State $s_i$  | $S_0$ | $S_1$ | $S_3$ | $S_3$ | $S_2$ | $S_0$ | $S_0$ | $S_1$ | $S_2$ |
| Output $x_i$ | 11    | 01    | 10    | 01    | 11    | 00    | 11    | 10    | 00    |

Trace the first few by eye on the diagram: from  $S_0$  an input of 1 outputs 11 and moves to  $S_1$ . From  $S_1$  an input of 1 outputs 01 and moves to  $S_3$ . From  $S_3$  an input of 1 outputs 10 and stays in  $S_3$ . The table matches.

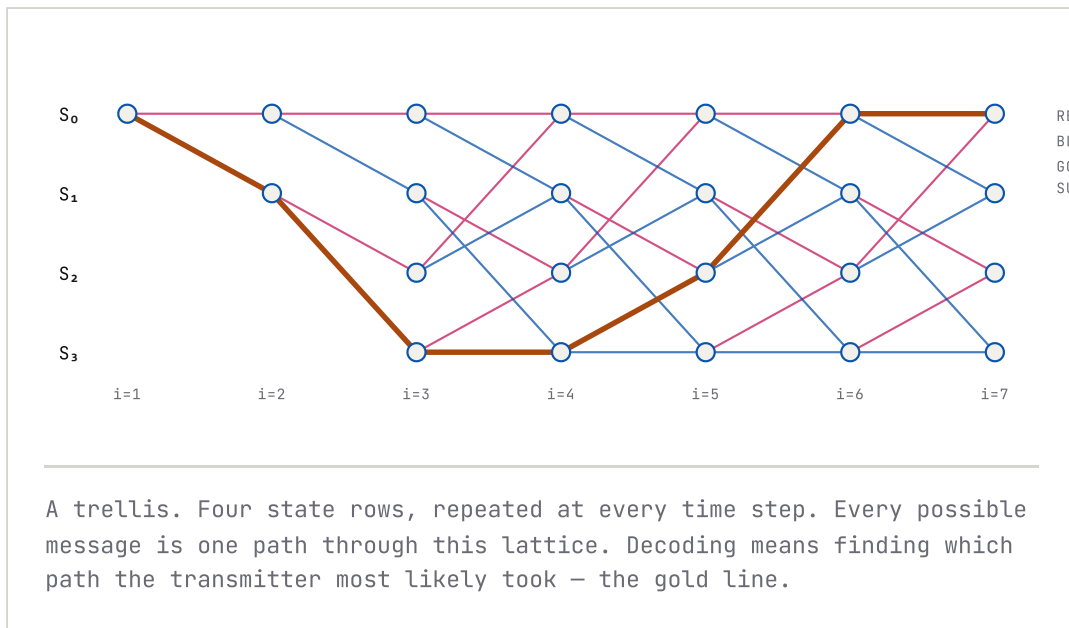
## Tree versus trellis

### IN SIMPLE WORDS

A **tree diagram** draws every possible path: at each step the branch splits in two, up for one input and down for the other. It is easy to understand but it **doubles in size every step** — after 20 bits there are over a million branches. Useless in practice.

A **trellis diagram** fixes that. It notices that the tree keeps arriving back at the *same four states* over and over, so instead of drawing new branches forever, it draws four rows of dots (one per state) repeated across time and joins them up. The trellis stays the same height no matter how long the message is.

**That is why every practical decoder uses the trellis.** It turns an exponentially exploding problem into a fixed-size one.



## 7.12 Viterbi decoding

Finally, the decoder. The **Viterbi algorithm** finds the single most likely path through the trellis.

### IN SIMPLE WORDS — THE CORE IDEA

The receiver gets a noisy bit stream and asks: *which path through the trellis would have produced something closest to what I actually heard?*

The obvious approach — try every possible path and compare — is impossible; there are  $2^N$  of them. Viterbi's insight makes it easy:

**At each state, at each time step, only the best path arriving there can possibly be part of the overall best path. So throw all the others away immediately.**

Two paths arrive at state  $S_2$ . One disagrees with the received bits in 3 places, the other in 5. Whatever happens afterwards is identical for both, so the 5 can never catch up. Discard it forever. The kept one is called the **survivor**.

Because you only ever keep **one survivor per state**, and there are only 4 states, the work per time step is constant no matter how long the message.

#### IN SIMPLE WORDS – BRANCH METRIC AND PATH METRIC

**Branch metric** = the Hamming distance between the two bits a branch says should have been sent, and the two bits actually received. If the branch says 11 and you received 10, the branch metric is 1.

**Path metric** = the running total of branch metrics along a path. This is the number written inside each circle on your slide's trellis (the 0, 2, 3, 3, 0, 2, 3, 0 values).

At the end, the path with the **smallest** total metric wins. Trace it backwards, read off the input bits that produced it, and you have your decoded message.

### The ACS loop and the flowchart

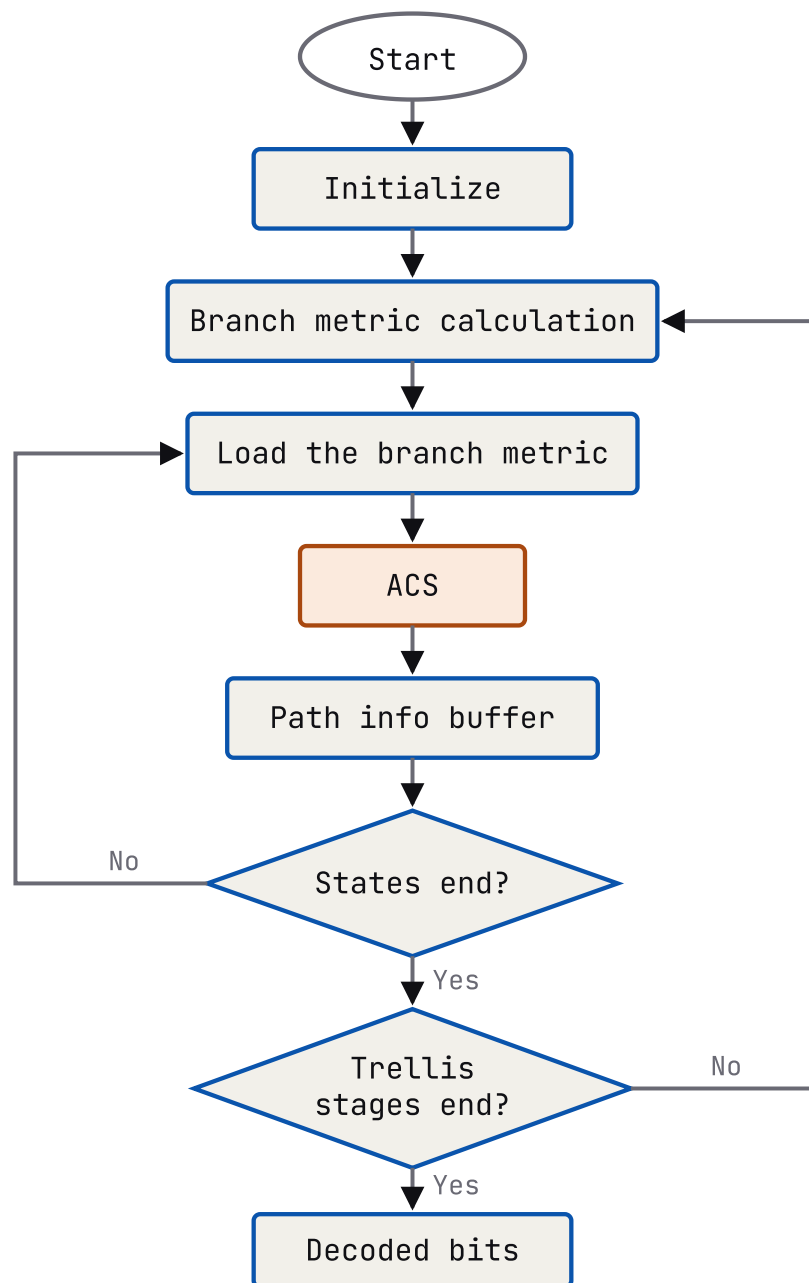
Your slide's flowchart names the central operation **ACS**. It stands for the three things done at every state, at every step:

#### THE ACS OPERATION

**A – Add**      add each incoming branch metric to that branch's previous path metric

**C – Compare**      compare the resulting totals for the paths arriving at this state

**S – Select**      keep the smallest, discard the rest; store it in the path info buffer



The Viterbi decoder flowchart from your slide. Two nested loops: the inner one repeats ACS for every state at the current time step; the outer one moves on to the next trellis stage. When both finish, trace back the surviving path to read out the decoded bits.

#### EXAM TIP – HOW TO ANSWER A VITERBI QUESTION

1. Draw the trellis for the given number of stages. Label every branch with its output bits.
2. Write the received words above the trellis, one per stage.
3. Stage by stage, compute branch metrics (Hamming distance between branch label and received word).

4. At each state, add, compare, select. Write the surviving path metric inside the circle and mark the surviving branch.
5. At the end, pick the state with the smallest metric and **trace backwards** along the marked branches.
6. Read off the input bits (0 for the red-type branch, 1 for the blue-type) to get the decoded message.

Your slide's example uses received words  $y_1=(11)$ ,  $y_2=(01)$ ,  $y_3=(01)$ ,  $y_4=(11)$ ,  $y_5=(11)$ ,  $y_6=(10)$ ,  $y_7=(11)$  with metrics  $\Gamma_0$  through  $\Gamma_7$ . Work through it once with the trellis in front of you and the method will stick.

#### WHERE THIS ENDS UP

Convolutional codes with Viterbi decoding ran GSM, Wi-Fi, and the Voyager probes. Their descendants — turbo codes, LDPC and polar codes, all named on your slide 7 — come within a fraction of a decibel of the Shannon limit from Module 6. 5G uses LDPC for data and polar codes for control channels.

Shannon proved in 1948 that such codes must exist. It took until roughly 1993–2008 to actually find them. That gap is the story of this entire subject.

## 09 SYLLABUS UNIT 3 — ASKED IN THE SEPT 2025 PAPER

## Carrier modulation: ASK, FSK and PSK

Everything so far has been **baseband** — bits turned into voltages that sit near zero frequency. To send those bits through the air they have to be lifted onto a carrier. This module covers the three ways of doing that, and it is written because both of them appeared on your September 2025 paper.

### 9.1 Why a carrier is needed at all

A baseband signal cannot simply be fed to an antenna. Three reasons, and any of the three earns the mark:

- **Antenna size.** An efficient antenna has to be about a quarter of a wavelength long. At 1 kHz the wavelength is 300 km, so the antenna would be 75 km. At 1 GHz it is 7.5 cm. Raising the frequency is what makes radiating possible at all.
- **Sharing the spectrum.** If everybody transmitted at baseband, every signal would sit on top of every other one. Giving each user a different carrier separates them — this is frequency division multiplexing.
- **The channel.** Many channels simply do not pass low frequencies. Anything with a transformer or a capacitor in it blocks DC, and the baseband signal is full of it.

#### IN SIMPLE WORDS

The carrier is a plain sine wave,  $c(t) = A \cos(2\pi f_c t + \varphi)$ . It carries no information by itself. A sine wave has exactly **three** things you can change: its **amplitude**  $A$ , its **frequency**  $f_c$ , and its **phase**  $\varphi$ .

Change the amplitude with the data and you have **ASK**. Change the frequency and you have **FSK**. Change the phase and you have **PSK**. There is no fourth option, which is why the list of digital modulation schemes is so short.

### 9.2 The three schemes, defined

#### FORMULA — THE THREE BINARY SCHEMES

**ASK** (Amplitude Shift Keying), also called On-Off Keying:

$$s(t) = A \cos(2\pi f_c t) \text{ for a } \mathbf{1} \quad \cdot \quad s(t) = 0 \text{ for a } \mathbf{0}$$

**FSK** (Frequency Shift Keying):

$s(t) = A \cos(2\pi f_1 t)$  for a **1**      •       $s(t) = A \cos(2\pi f_2 t)$   
for a **0**

**PSK** (Phase Shift Keying), binary → BPSK:

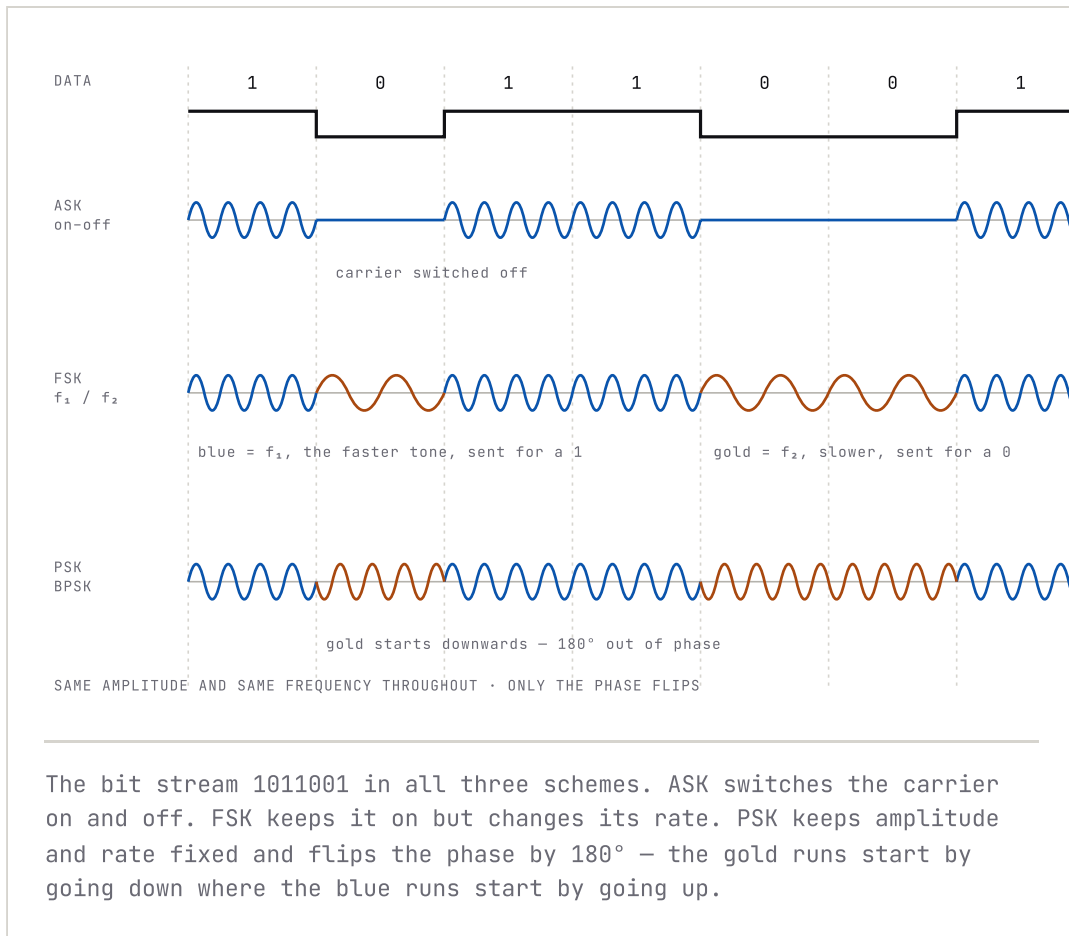
$s(t) = A \cos(2\pi f_c t)$  for a **1**      •       $s(t) = A \cos(2\pi f_c t + \pi) = -A \cos(2\pi f_c t)$  for a **0**

**EXAM TIP – BPSK IS JUST POLAR NRZ TIMES A CARRIER**

Look at that last line again. A **1** gives  $+A \cos$ , a **0** gives  $-A \cos$ . That is exactly polar NRZ from Module 3 multiplied by  $\cos(2\pi f_c t)$ . Every BPSK question becomes easy once you see it that way — and it is why the BPSK modulator in §9.5 is nothing but a level converter followed by a multiplier.

## 9.3 Drawing the waveforms

The September 2025 paper gave the bit stream **1 0 1 1 0 0 1** and asked for the ASK, FSK and PSK waveforms. Here it is, drawn for exactly that sequence.



#### EXAM TIP - HOW TO DRAW THESE WITHOUT LOSING MARKS

1. Rule the bit boundaries first and write the bits above them. Everything else hangs off that grid.
2. Put a **whole number of carrier cycles** in each bit interval — two or three is plenty. Neatness here is most of the mark.
3. ASK: draw the carrier for a 1, a flat line for a 0.
4. FSK: same amplitude everywhere, just squeeze the cycles up for one bit value and stretch them out for the other.
5. PSK: draw the carrier for a 1, and for a 0 draw it upside down. At every change of bit the waveform must visibly jump.

## 9.4 Bandwidth of each scheme

### FORMULA - TRANSMISSION BANDWIDTH

**ASK:**  $B_T = 2 R_b$  (null to null; minimum Nyquist value is  $R_b$ )

**PSK:**  $B_T = 2 R_b$  (same as ASK — same envelope, different sign)

**FSK:**  $B_T = |f_1 - f_2| + 2 R_b$

With raised-cosine shaping instead of rectangles:  $B_T = (1 + \alpha) R_b$

#### IN SIMPLE WORDS — WHERE THE $2R_b$ COMES FROM

In Module 3 the baseband pulse train had a null-to-null bandwidth of  $R_b$ , measured from 0 up to  $R_b$ . Multiplying by a carrier **shifts that whole spectrum up to  $f_c$**  and leaves a mirror image below it. So what was  $R_b$  wide on one side of zero becomes  $R_b$  on each side of  $f_c$  — a total of  $2R_b$ .

FSK is two ASK signals at two different carriers, so you need the gap between them *plus* the width of one of them on either end.

#### WORKED EXAMPLE — BPSK, FROM THE END SEM 2024 PAPER

**Question.** For a BPSK modulator with a carrier frequency of 70 MHz and an input bit rate of 10 Mbps, determine the minimum Nyquist bandwidth and the maximum and minimum upper and lower side frequencies.

##### 1 Find the fundamental of the modulating signal

The fastest pattern the data can make is 1010101..., which is one full square-wave cycle every *two* bits. So the highest fundamental frequency is half the bit rate:

$$f_a = R_b / 2 = 10 / 2 = 5 \text{ MHz}$$

##### 2 Multiply by the carrier — side frequencies

Multiplying two sinusoids gives the sum and the difference:

$$\text{upper side frequency} = f_c + f_a = 70 + 5 = \mathbf{75 \text{ MHz}}$$

$$\text{lower side frequency} = f_c - f_a = 70 - 5 = \mathbf{65 \text{ MHz}}$$

### 3 Minimum Nyquist bandwidth

$$B = 75 - 65$$

$$B = 10 \text{ MHz, which equals } R_b$$

Note the neat result: **the minimum bandwidth of BPSK equals the bit rate.** The  $2R_b$  quoted above is the null-to-null figure for unfiltered rectangular pulses; the Nyquist minimum, with ideal filtering, is  $R_b$ . Say which one you are giving.

#### WORKED EXAMPLE - FSK BANDWIDTH

**Question.** A modulator generates FSK with  $f_{c1} - f_{c2} = 100 \text{ kHz}$ . The data rate is 1 Mbit/s with full-width polar pulses. Find the bandwidth of the modulated output.

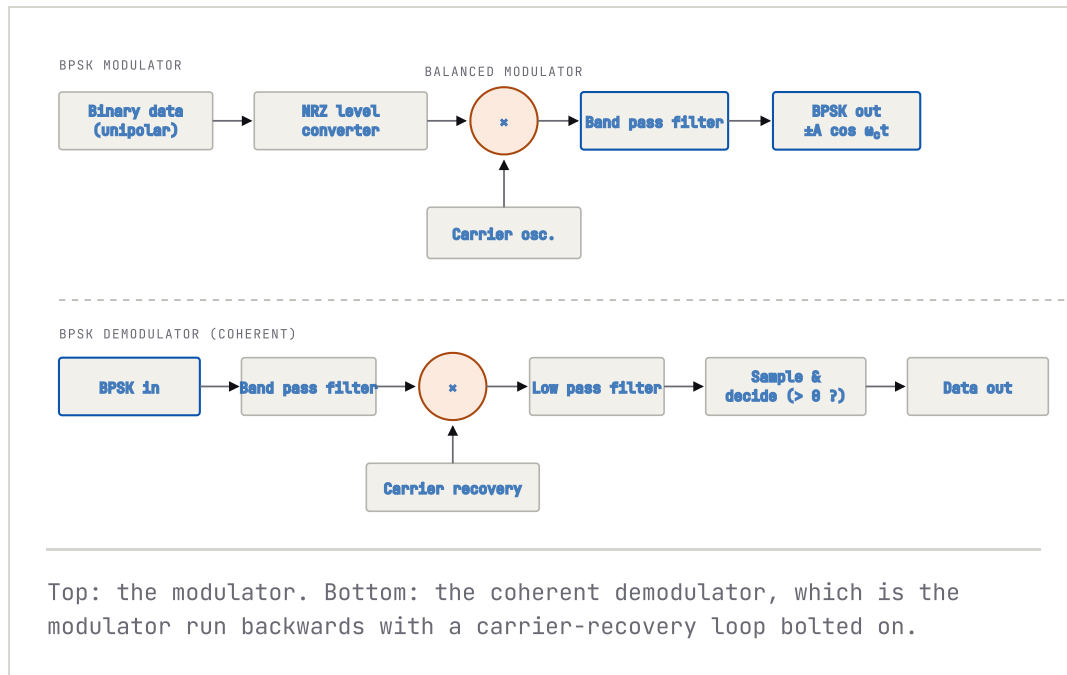
$$B_T = |f_1 - f_2| + 2R_b = 100 \text{ kHz} + 2(1 \text{ MHz}) = 100 \text{ kHz} + 2000 \text{ kHz}$$

$$B_T = 2.1 \text{ MHz}$$

Compare with PSK at the same data rate: 2 MHz. **FSK always needs more bandwidth than PSK**, by exactly the frequency separation. That is FSK's main disadvantage.

## 9.5 The BPSK modulator and demodulator

The 2025 paper asked for both block diagrams with an explanation of each block. Here they are.



## What each block does

| BLOCK                      | ITS JOB                                                                                                                                                                                                                                                                |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NRZ level converter</b> | Turns the unipolar data (0 V and +V) into polar data (-V and +V). This is the step that makes the phase flip possible – a 0 must become a <i>negative</i> number, not zero.                                                                                            |
| <b>Carrier oscillator</b>  | Generates the reference sine wave at $f_c$ .                                                                                                                                                                                                                           |
| <b>Balanced modulator</b>  | A product modulator: it multiplies the polar data by the carrier. Multiplying by +V leaves the carrier alone; multiplying by -V inverts it, which is the $180^\circ$ phase shift. It is called <i>balanced</i> because the carrier itself is suppressed in the output. |
| <b>Band pass filter</b>    | Removes the out-of-band products of the multiplication and limits the transmitted spectrum to its allotted channel.                                                                                                                                                    |
| – receiver –               |                                                                                                                                                                                                                                                                        |
| <b>Band pass filter</b>    | Passes the wanted channel and rejects noise and neighbouring channels before anything else is done.                                                                                                                                                                    |
| <b>Carrier recovery</b>    | Regenerates a local carrier locked to the incoming one in both frequency <i>and</i> phase. BPSK is <b>coherent</b> , so this block is essential – get the phase wrong by $180^\circ$ and every bit comes out inverted.                                                 |
| <b>Balanced modulator</b>  | Multiplies the received signal by the recovered carrier. Since $\cos^2\theta = \frac{1}{2}(1 + \cos 2\theta)$ , this produces the wanted DC-like term plus a component at $2f_c$ .                                                                                     |
| <b>Low pass filter</b>     | Throws away the $2f_c$ component and leaves the baseband data.                                                                                                                                                                                                         |
| <b>Sample and decide</b>   | Samples once per bit at the best instant and compares against a threshold of zero: positive $\rightarrow$ 1, negative $\rightarrow$ 0.                                                                                                                                 |

### EXAM TIP – THE PHASE AMBIGUITY

Because  $+\cos$  and  $-\cos$  look identical to a recovery loop that has just started, a coherent BPSK receiver can lock  $180^\circ$  out and decode every bit backwards. The standard cure is **differential encoding** — send the *change* rather than the value — which gives DPSK. If a question asks "why use DPSK instead of PSK", that is the

answer, and the price is roughly 3 dB worse noise performance because the reference is itself a noisy received symbol.

## 9.6 Comparing the three

| ASK, FSK AND PSK SIDE BY SIDE |                                 |                                  |                                  |
|-------------------------------|---------------------------------|----------------------------------|----------------------------------|
|                               | ASK                             | FSK                              | PSK (BPSK)                       |
| What varies                   | amplitude                       | frequency                        | phase                            |
| Bandwidth                     | $2R_b$                          | $ f_1 - f_2  + 2R_b$<br>– widest | $2R_b$                           |
| Noise immunity                | worst                           | middling                         | best                             |
| Constant envelope?            | no                              | yes                              | yes                              |
| Complexity                    | simplest                        | middling                         | highest (needs coherent carrier) |
| Error probability             | $P_e = Q(\sqrt{E_b/N_0})$       | $P_e = Q(\sqrt{E_b/N_0})$        | $P_e = Q(\sqrt{2E_b/N_0})$       |
| Used in                       | optical fibre, IR remotes, RFID | modems, Bluetooth, LoRa          | satellite, Wi-Fi, 4G/5G          |

### IN SIMPLE WORDS – WHY BPSK WINS

Read the error row. ASK and FSK have  $E_b/N_0$  inside the square root; BPSK has  $2E_b/N_0$ . That factor of two is **3 dB**, and it means BPSK reaches the same error rate on half the power.

The reason is geometric. ASK puts its two symbols at 0 and A — they are A apart. BPSK puts them at  $-A$  and  $+A$  — they are  $2A$  apart. Twice the distance between symbols means the noise has to be twice as large to cause a mistake. **Whenever you can, spend your power moving symbols apart rather than turning them off.**

### EXAM TIP – A ONE-LINE ANSWER FOR "WHICH IS BEST"

**"BPSK**, because for the same bandwidth as ASK it needs 3 dB less power for the same bit error rate, and unlike ASK it has a constant envelope so the transmitter

amplifier can be run in saturation. Its cost is that the receiver must recover the carrier coherently."

#### WHAT THIS MODULE DOES NOT COVER

Unit 3 of your syllabus also contains QPSK and higher M-ary schemes, DPSK detection, non-coherent FSK and the full derivations of  $P_e$ . There is nothing on those in the material you have, so they are not written up here. What is here is what the September 2025 paper actually asked for.

## A APPENDIX

# The complete formula sheet

Every formula in this course, on one page. Print this. If you learn nothing else the night before the exam, learn this.

## Sampling, quantization and PCM

| QUANTITY                 | FORMULA                           | UNITS     |
|--------------------------|-----------------------------------|-----------|
| Nyquist sampling rate    | $f_s \geq 2 f_m$                  | samples/s |
| Bits per sample          | $n = \log_2 L$                    | bits      |
| Number of levels         | $L = 2^n$                         | —         |
| PCM bit rate             | $R_b = n f_s$                     | bits/s    |
| TDM bit rate             | $R_b = N f_s n$                   | bits/s    |
| Quantization step        | $\Delta = V_{pp} / 2^n$           | volts     |
| Max quantization error   | $\epsilon_{\max} = \Delta/2$      | volts     |
| Quantization noise power | $P_q = \Delta^2/12$               | watts     |
| SNR (general)            | $SNR = V_{rms}^2 / (\Delta^2/12)$ | ratio     |
| SQNR (full-scale sine)   | $SQNR_{dB} = 6.02n + 1.76$        | dB        |
| Transmission BW (NRZ)    | $B_T \approx R_b/2$               | Hz        |
| Transmission BW (RZ)     | $B_T \approx R_b$                 | Hz        |

## Line coding and baseband signalling

| QUANTITY                    | FORMULA                                               | UNITS      |
|-----------------------------|-------------------------------------------------------|------------|
| Pulse rate                  | $1 / (\text{pulse duration})$                         | baud       |
| Bit rate from pulse rate    | $\text{Bit rate} = \text{Pulse rate} \times \log_2 L$ | bits/s     |
| Signal rate from data rate  | $S = c \times N \times (1/r)$                         | baud       |
| Average case factor         | $c = \frac{1}{2}$                                     | —          |
| Nyquist max data rate       | $N_{\max} = 2 \times B \times \log_2 L$               | bits/s     |
| NRZ signal rate             | $S_{\text{ave}} = N/2 \quad (r = 1)$                  | baud       |
| RZ / Manchester signal rate | $S_{\text{ave}} = N \quad (r = \frac{1}{2})$          | baud       |
| Minimum bandwidth           | $B_{\min} = S$                                        | Hz         |
| Unipolar NRZ power          | $\frac{1}{2}V^2 + \frac{1}{2}(0)^2 = \frac{1}{2}V^2$  | normalized |
| Scrambler                   | $S = (1 \oplus F)T, \quad F = D^3 \oplus D^5$         | —          |
| Descrambler                 | $T \oplus FT = S$                                     | —          |
| XOR cancellation            | $D^n \oplus D^n = 0$                                  | —          |

## Bandlimited channels, ISI and equalization

| QUANTITY                    | FORMULA                                                           | UNITS     |
|-----------------------------|-------------------------------------------------------------------|-----------|
| Received sample (3 parts)   | $y(t_i) = \mu a_i + \mu \sum_{k \neq i} a_k p[(i-k)T_b] + n(t_i)$ | volts     |
| 1st Nyquist – time          | $p(nT) = 1 \ (n = 0); \ 0 \ (n \neq 0)$                           | –         |
| 1st Nyquist – frequency     | $\sum_m P(f + m/T) = T$                                           | –         |
| Minimum-BW pulse (1/T = 2W) | $p(t) = \text{sinc}(\pi t/T); \ P(f) = T \text{ for }  f  < W$    | –         |
| Raised-cosine bandwidth     | $f_B = (1 + \alpha) / (2T_b)$                                     | Hz        |
| Same, in terms of $R_b$     | $B_T = (R_b/2)(1 + \alpha)$                                       | Hz        |
| Roll-off factor range       | $0 \leq \alpha \leq 1 \ (\alpha = r)$                             | –         |
| Bandwidth efficiency        | $\beta_{rc} = 2/(1 + r)$                                          | bit/s/Hz  |
| Efficiency range            | $1 \leq 2/(1+r) < 2$                                              | bit/s/Hz  |
| Data rate from bandwidth    | $R_b = 2B_T/(1 + r)$                                              | bits/s    |
| 2nd Nyquist                 | $p(t) = 0.5 \text{ at } t = \pm T/2$                              | –         |
| Channel attenuation         | $r(t) = as(t) + n(t), \ 0 \leq a \leq 1$                          | –         |
| Received energy / SNR       | $a^2 E_b \quad \cdot \quad a^2 E_b / N_0$                         | J · ratio |
| Equalizer output            | $p_o[k] = \sum_{n=-N}^N c_n p_r[k - n]$                           | –         |
| Zero-forcing condition      | $p_o[0] = 1; \ p_o[k] = 0 \text{ for } k = \pm 1, \dots, \pm N$   | –         |
| Equalizer taps              | $c = P_r^{-1} P_o \ (2N+1 \text{ taps})$                          | –         |
| Duobinary levels            | $y_n = b_n + b_{n-1} = +2d, \ 0 \text{ or } -2d$                  | volts     |
| Duobinary decoding          | $\hat{b}_n = y_n - \hat{b}_{n-1}$                                 | –         |
| Duobinary efficiency        | $2B \text{ bit/s over bandwidth } B$                              | bit/s/Hz  |

## Information theory

| QUANTITY                   | FORMULA                                           | UNITS       |
|----------------------------|---------------------------------------------------|-------------|
| Information content        | $I(x_i) = -\log_2 P(x_i)$                         | bits        |
| Entropy                    | $H(X) = -\sum P(x_i) \log_2 P(x_i)$               | bits/symbol |
| Entropy bounds             | $0 \leq H(X) \leq \log_2 m$                       | bits/symbol |
| Information rate           | $R = r H(X)$                                      | bits/s      |
| Change of base             | $\log_2 a = \ln a / \ln 2 = \log_{10} a / 0.3010$ | —           |
| Channel matrix row rule    | $\sum_j P(y_j x_i) = 1$                           | —           |
| Capacity per symbol        | $C_s = \max I(X;Y)$                               | bits/symbol |
| Capacity per second        | $C = r C_s$                                       | bits/s      |
| Noiseless channel capacity | $C_s = \log_2 m$                                  | bits/symbol |
| BSC capacity               | $C_s = 1 + p \log_2 p + (1-p) \log_2(1-p)$        | bits/symbol |
| Shannon–Hartley law        | $C = B \log_2(1 + S/N)$                           | bits/s      |
| Per-sample capacity        | $C_s = \frac{1}{2} \log_2(1 + S/N)$               | bits/sample |
| dB to ratio                | $S/N = 10^{(dB/10)}$                              | ratio       |
| Working condition          | $H(X) \leq R < C$                                 | —           |

## Source coding

| QUANTITY                 | FORMULA                                | UNITS       |
|--------------------------|----------------------------------------|-------------|
| Average code length      | $L = \sum P(x_i) n_i$                  | bits/symbol |
| Source coding theorem    | $L \geq H(X)$ , so $L_{\min} = H(X)$   | —           |
| Code efficiency          | $\eta = H(X) / L$                      | —           |
| Code redundancy          | $\gamma = 1 - \eta$                    | —           |
| Kraft inequality         | $K = \sum 2^{-n_i} \leq 1$             | —           |
| Rate distortion function | $R(D) = \min I(X;Y) \text{ over } P_D$ | bits/symbol |

## Channel coding

| QUANTITY                  | FORMULA                                                     | UNITS |
|---------------------------|-------------------------------------------------------------|-------|
| Code rate                 | $R = k / n$                                                 | —     |
| Hamming distance          | number of differing positions                               | —     |
| Errors detectable         | $d_{\min} - 1$                                              | bits  |
| Errors correctable        | $\lfloor (d_{\min} - 1)/2 \rfloor$                          | bits  |
| Hamming block length      | $n = 2^r - 1$                                               | bits  |
| Hamming message length    | $k = 2^r - r - 1$                                           | bits  |
| Hamming rate              | $1 - r/(2^r - 1)$                                           | —     |
| Hamming $d_{\min}$        | 3 (detects 2, corrects 1)                                   | —     |
| Generator matrix encoding | $C = d G$ , with $G = [I \mid P]$                           | —     |
| Convolutional code rate   | $r = k/n$ , using $K-1$ shift registers                     | —     |
| Convolutional encoding    | $c_i(D) = g_i(D) \cdot m(D)$                                | —     |
| Noise power from density  | $N = N_0 B \quad \cdot \quad N_{dBm} = N_0 + 10\log_{10} B$ | watts |
| Thermal noise density     | $N_0 = kT \approx -174 \text{ dBm/Hz at } 290 \text{ K}$    | W/Hz  |
| dBm to watts              | $P = 10^{(P_{dBm}-30)/10}$                                  | W     |
| ASK / PSK bandwidth       | $B_T = 2R_b$ (null to null), $R_b$ (Nyquist min)            | Hz    |
| FSK bandwidth             | $B_T =  f_1 - f_2  + 2R_b$                                  | Hz    |
| BPSK side frequencies     | $f_c \pm R_b/2$                                             | Hz    |
| Error probability         | ASK, FSK: $Q(\sqrt{E_b/N_0})$ · BPSK: $Q(\sqrt{2E_b/N_0})$  | —     |
| Shannon coding theorem    | reliable iff $R < C$                                        | —     |

**B** APPENDIX

## Symbols and glossary

Half the difficulty in this subject is that the same letter means different things in different chapters. Here is every symbol, in one place.

| SYMBOLS USED IN THIS COURSE |                                                                                                                                                             |            |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| SYMBOL                      | MEANS                                                                                                                                                       | MODULE     |
| $f_m$                       | Highest frequency in the message signal                                                                                                                     | 2          |
| $f_s$                       | Sampling frequency                                                                                                                                          | 2          |
| $T_b, T$                    | Bit duration / symbol period ( $= 1/R_b$ )                                                                                                                  | 2, 3, 4    |
| $L$                         | Number of quantization levels, or signal levels                                                                                                             | 2, 3       |
| $n$                         | Bits per sample ( <i>Module 2</i> ); codeword length ( <i>Module 8</i> )                                                                                    | 2, 8       |
| $\Delta$                    | Quantization step size                                                                                                                                      | 2          |
| $R_b, N$                    | Bit rate (data rate)                                                                                                                                        | 2, 3, 4    |
| $S$                         | Signal rate (baud rate) – signal elements per second                                                                                                        | 3          |
| $r$                         | Data elements per signal element ( <i>Module 3</i> ); roll-off factor ( <i>Module 4</i> ); symbol rate ( <i>Module 5</i> ); parity bits ( <i>Module 8</i> ) | 3, 4, 5, 8 |
| $c$                         | Case factor in $S = c \times N \times (1/r)$                                                                                                                | 3          |
| $B_T, f_B, B_t$             | Transmission bandwidth                                                                                                                                      | 2, 3, 4    |
| $\alpha$                    | Roll-off factor of a raised-cosine pulse (same as $r$ in Module 4)                                                                                          | 3, 4       |
| $W$                         | Bandwidth of the pulse spectrum $P(f)$                                                                                                                      | 4          |
| $p(t), P(f)$                | Pulse shaping function and its Fourier transform                                                                                                            | 4          |
| $p_r[k], p_o[k]$            | Received and equalised pulse samples                                                                                                                        | 4          |
| $c_n$                       | Equalizer tap weights                                                                                                                                       | 4          |
| $\beta_{rc}$                | Transmission (bandwidth) efficiency, bit/s per Hz                                                                                                           | 4          |
| $a_n, b_n, y_n$             | Duobinary input bit, precoded level, encoder output                                                                                                         | 4          |
| $I(x_i)$                    | Information content of one symbol                                                                                                                           | 5          |

| SYMBOL                   | MEANS                                                                          | MODULE  |
|--------------------------|--------------------------------------------------------------------------------|---------|
| $H(X)$                   | Entropy – average information per symbol                                       | 5       |
| $m$                      | Number of symbols in the source alphabet                                       | 5       |
| $R$                      | Information rate ( <i>Modules 5-6</i> ); code rate ( <i>Module 8</i> )         | 5, 6, 8 |
| $C, C_s$                 | Channel capacity, per second and per symbol                                    | 6       |
| $p$                      | Crossover (bit flip) probability of the BSC                                    | 6       |
| $I(X;Y)$                 | Mutual information between input and output                                    | 6, 7    |
| $B$                      | Channel bandwidth                                                              | 6       |
| $S/N$                    | Signal-to-noise power ratio (plain ratio, not dB)                              | 6       |
| $n_i$                    | Length of the code word for symbol $x_i$                                       | 7       |
| $L$<br>( <i>italic</i> ) | Average code word length                                                       | 7       |
| $\eta$                   | Code efficiency                                                                | 7       |
| $\gamma$                 | Code redundancy                                                                | 7       |
| $D$                      | Allowed distortion ( <i>Module 7</i> ); delay operator ( <i>Modules 3, 8</i> ) | 3, 7, 8 |
| $k$                      | Number of information bits per block                                           | 8       |
| $d_{\min}$               | Minimum Hamming distance of a code                                             | 8       |
| $K$                      | Constraint length of a convolutional code                                      | 8       |
| $g_1, g_2$               | Generator vectors / polynomials                                                | 8       |
| $G$                      | Generator matrix                                                               | 8       |
| $\oplus$                 | XOR, also written as modulo-2 addition                                         | 3, 4, 8 |

## Abbreviations

| SHORT         | FULL FORM                           | WHAT IT IS                                    |
|---------------|-------------------------------------|-----------------------------------------------|
| AWGN          | Additive White Gaussian Noise       | The standard noise model                      |
| BER / SER     | Bit / Symbol Error Rate             | Fraction received wrongly                     |
| PCM           | Pulse Code Modulation               | Sample, quantize, encode                      |
| SQNR          | Signal to Quantization Noise Ratio  | Quantizer quality                             |
| TDM           | Time Division Multiplexing          | Many signals take turns on one line           |
| NRZ / RZ      | Non-Return to Zero / Return to Zero | Line code families                            |
| AMI           | Alternate Mark Inversion            | Bipolar line code; 1s alternate sign          |
| ISI           | Intersymbol Interference            | Pulses smearing into neighbours               |
| NRZ-L / NRZ-I | NRZ-Level / NRZ-Invert              | Absolute vs differential polar NRZ            |
| PSD           | Power Spectral Density              | How a code's power spreads over frequency     |
| ZFE           | Zero-Forcing Equalizer              | Tapped filter that forces ISI to zero         |
| PAM           | Pulse Amplitude Modulation          | Data carried in the pulse height              |
| MLT-3         | Multi-Level Transmit, 3 levels      | Multitransition line code                     |
| DMS           | Discrete Memoryless Source          | Independent symbols, fixed probabilities      |
| DMC           | Discrete Memoryless Channel         | Independent noise per symbol                  |
| BSC           | Binary Symmetric Channel            | Two inputs, two outputs, flip probability $p$ |
| RDT           | Rate Distortion Theory              | Bits needed for a given allowed error         |
|               |                                     |                                               |

| SHORT     | FULL FORM                                | WHAT IT IS                          |
|-----------|------------------------------------------|-------------------------------------|
| FEC       | Forward Error Correction                 | Receiver repairs errors itself      |
| ARQ       | Automatic Repeat reQuest                 | Receiver asks for a resend          |
| VRC / LRC | Vertical / Longitudinal Redundancy Check | Row parity / column parity          |
| CRC       | Cyclic Redundancy Check                  | Detection by modulo-2 division      |
| ACS       | Add, Compare, Select                     | The core step of Viterbi decoding   |
| LDPC      | Low Density Parity Check                 | Modern near-capacity code (5G data) |

## C APPENDIX

# Test yourself

Try to answer before you open each one. If you can do all forty, you know this material. Answers are written out in full, not just stated.

## Modules 1–3 — system, PCM, line codes

### 1. Why does source coding remove redundancy while channel coding adds it?

Source coding removes **useless, accidental** redundancy that was already in the data, to make it smaller. Channel coding then adds back a small amount of **carefully designed** redundancy whose mathematical structure lets the receiver detect and correct noise errors. You throw out the junk, then add the right tool. No contradiction.

### 2. What do the four letters of AWGN stand for, and what does each mean?

**Additive** — noise adds to the signal,  $y(t) = x(t) + n(t)$ . **White** — equal power at all frequencies. **Gaussian** — the noise voltage follows a bell curve. **Noise** — unwanted random energy, mostly thermal.

### 3. A signal has bandwidth 8 kHz and is quantized into 256 levels. Find $f_s$ , $n$ , $R_b$ and SQNR.

$f_s = 2 \times 8 \text{ kHz} = \mathbf{16 \text{ kSamples/s}}$ .  
 $n = \log_2 256 = \mathbf{8 \text{ bits/sample}}$  (since  $256 = 2^8$ ).  
 $R_b = 8 \times 16\,000 = \mathbf{128 \text{ kb/s}}$ .  
 $\text{SQNR} = 6.02(8) + 1.76 = 48.16 + 1.76 = \mathbf{49.92 \text{ dB}}$ .

### 4. Why does an RZ code need twice the bandwidth of NRZ?

In RZ the pulse only occupies half the bit period, so it is half as wide in time. A narrower pulse changes twice as fast, and faster changes contain higher frequencies. Since bandwidth is roughly the inverse of pulse width, halving the width doubles the bandwidth.

### 5. Draw AMI for the bit stream 1 0 1 1 0 1.

Ignore the zeros and number the 1s: first, second, third, fourth. Assign alternating signs: **+A, -A, +A, -A**. So the waveform is +A, 0, -A, +A, 0, -A. Zeros sit at the 0 level.

**6. A channel of 100 kHz carries data at 125 kb/s with raised-cosine pulses. Find  $\alpha$ .**

Use  $B_T = (R_b/2)(1 + \alpha)$ . So  $100\,000 = (125\,000/2)(1 + \alpha) = 62\,500(1 + \alpha)$ .  
Therefore  $1 + \alpha = 100\,000/62\,500 = 1.6$ , giving  $\alpha = 0.6$ .

**7. What is ISI and how does the raised-cosine pulse defeat it?**

ISI is Intersymbol Interference: a bandwidth-limited channel smears each pulse in time so its tails spill into neighbouring symbol slots and corrupt them. The raised-cosine pulse is shaped so that it is exactly **zero at every other sampling instant**. Its tails still overlap in time, but they contribute nothing at the precise moments when the neighbours are sampled, so there is no interference.

**8. A signal has  $r = 1$  and a bit rate of 200 kbps. Find the average baud rate.**

Use  $S = c \times N \times (1/r)$  with the average case factor  $c = 1/2$ :  
 $S = 1/2 \times 200\,000 \times (1/1) = 100\text{ kbaud}$ .

**9. Name the four reasons we need line coding.**

- 1. Baseline wandering** — the receiver's running average drifts on long runs of the same bit, moving the decision threshold.
- 2. Lack of synchronization** — if the receiver clock is fast or slow it samples in the wrong places.
- 3. DC components** — constant voltages create very low frequencies, which transformers and telephone lines (below 200 Hz) cannot pass.
- 4. Self-synchronization** — the signal should carry its own timing, via transitions marking the start, middle or end of each pulse.

**10. A receiver clock is 0.2% fast on a 2 Mbps link. How many extra bits per second?**

0.2% of  $2\,000\,000 = 4000$  **extra bits per second**. The same 0.2% error on a 1 kbps link would give only 2 extra bits — the damage scales with speed, which is why fast links need self-synchronising codes.

**11. What is the difference between NRZ-L and NRZ-I, and why does it matter?**

In **NRZ-L** the voltage *level* gives the bit directly — it is absolute. In **NRZ-I** the bit is carried by *whether the level inverted* at the start of the bit — it is differential.

It matters because if the two wires are accidentally swapped, NRZ-L inverts every single bit and the message is destroyed, whereas NRZ-I still decodes correctly

because the *changes* are still in the same places.

### 12. Why does Manchester need twice the bandwidth of NRZ — and what do you get for it?

Manchester has  $r = 1/2$ , so  $S_{\text{ave}} = N$  instead of  $N/2$  — twice as many signal elements per second, hence twice the bandwidth.

In return you get a guaranteed transition in the middle of **every** bit, so the clock can never drift, and zero DC component. Perfect synchronization and no DC, paid for in spectrum.

### 13. How does scrambling differ from line coding and channel coding?

**Line coding** chooses the waveform shape. **Channel coding** adds extra redundant bits for error control. **Scrambling** does neither — it randomises the data pattern so long runs of zeros disappear, **without adding a single extra bit**.

It works because XOR undoes itself, so the identical circuit at the far end recovers the original stream exactly.

### 14. Why did serial transmission replace parallel over long distances?

Two reasons. **Cost** — eight wires cost eight times one. **Skew** — no two wires have identical delay, so bits that left together arrive at different times, and it worsens with distance and speed. A single wire has nothing to skew against. Parallel survives only over very short distances, inside chips and across boards.

## Module 4 — bandlimited channels, ISI, equalization

### 15. State Nyquist's first criterion in both the time and the frequency domain.

**Time:**  $p(nT) = 1$  when  $n = 0$ , and  $p(nT) = 0$  for all  $n \neq 0$ . The pulse is tall at its own sampling instant and zero at every other one.

**Frequency:**  $\sum_m P(f + m/T) = T$ . Shift copies of the pulse spectrum by multiples of the symbol rate, add them all up, and the total must be a flat constant.

### 16. Why can't we use the ideal $r = 0$ raised-cosine pulse in practice?

Two reasons. It needs a **brick-wall filter** — flat to  $W$  then instantly zero — which cannot be built from real components. And its tails decay only as  $1/t$ , so pulses from many symbols ago still contribute; a tiny timing error stops them cancelling and the

ISI becomes catastrophic. The eye pattern for  $r \rightarrow 0$  shows this: the eye is almost closed and  $t_h/T$  falls to 0.32.

### 17. A channel of 120 kHz carries data at 150 kb/s with raised-cosine pulses. Find $\alpha$ and the bandwidth efficiency.

From  $B_T = (R_b/2)(1 + \alpha)$ :

$$120\,000 = (150\,000/2)(1 + \alpha) = 75\,000(1 + \alpha)$$

$$1 + \alpha = 1.6, \text{ so } \alpha = 0.6.$$

Efficiency  $\beta_{rc} = 2/(1 + r) = 2/1.6 = 1.25 \text{ bit/s per Hz}$ . Check:  $150 \text{ kb/s} \div 120 \text{ kHz} = 1.25$ . ✓

### 18. What are the three cases of sample rate versus bandwidth?

**$1/T > 2W$**  — no function can satisfy the Nyquist condition; zero-ISI signalling is impossible.

**$1/T = 2W$**  — exactly one solution, the rectangular spectrum (sinc pulse). Minimum bandwidth.

**$1/T < 2W$**  — many solutions exist; the raised cosine is the typical choice.

### 19. Name the six things you can read off an eye diagram.

1. The **optimum sampling instant**, at maximum eye opening.
2. The **noise margin** — the vertical opening.
3. The **slope** — sensitivity to sampling time.
4. **Level-crossing ambiguity** — the timing jitter.
5. **Distortion** at the best sampling point — the ISI.
6. The **error-free sampling region** — the horizontal width.

### 20. Why does a regenerative repeater beat an analog amplifier?

An amplifier boosts signal *and* noise, so noise accumulates at every hop and the link degrades without limit — noise propagation. A regenerative repeater **decides** what the bit was, discards the noisy waveform, and builds a brand new perfect pulse, so noise does not accumulate at all.

Its one weakness: if the detector decides wrongly, that error is regenerated perfectly and **propagated forward**, and nothing downstream can detect it.

### 21. What are the three functions of a regenerative repeater?

1. **Reshaping** incoming pulses by means of an equalizer.
2. **Extracting the timing** needed to sample at the optimum instants.
3. **Making symbol detection decisions** based on the pulse samples.

## 22. Why is it called a “zero-forcing” equalizer, and what is its limitation?

Applying the Nyquist condition to a  $(2N+1)$ -tap filter gives an **infinite** set of simultaneous equations in only  $2N+1$  unknowns, which cannot all be solved. So we **force zeros** at just the  $2N$  nearest sampling instants and accept whatever ISI remains further out. Hence the name.

The limitation follows directly: a 3-tap equalizer only cancels the ISI from one symbol either side. Anything beyond that survives. More taps means better cancellation and more cost.

## 23. Write the matrix equation for a 3-tap equalizer when $p_r[0] = 1$ , $p_r[1] = -0.3$ , $p_r[-1] = -0.2$ , $p_r[2] = 0.1$ , $p_r[-2] = 0.05$ .

Each row shifts the index by one, and the target vector is  $[0, 1, 0]^T$ :

$$[0] \quad [ \quad 1.0 \quad -0.2 \quad 0.05 ] [c_{-1}]$$

$$[1] = [ -0.3 \quad 1.0 \quad -0.20 ] [c_0 \quad ]$$

$$[0] \quad [ \quad 0.1 \quad -0.3 \quad 1.00 ] [c_1 \quad ]$$

Solving gives  $c_{-1} = 0.210$ ,  $c_0 = 1.13$ ,  $c_1 = 0.318$ .

## 24. Why must the timing extraction circuit rectify the signal first?

A bipolar signal swings equally positive and negative, so its symbol-rate rhythm largely cancels out and is too weak to filter. Rectifying flips all the negative parts positive, so nothing cancels and the symbol-rate component becomes strong enough for the resonant timing extractor to ring at and pick out.

## 25. What does duobinary signalling do differently, and what does it gain?

Instead of eliminating ISI, it **deliberately introduces a controlled, known amount** of it — each transmitted value is the sum of the current and the previous symbol, giving three levels ( $+2d$ ,  $0$ ,  $-2d$ ). Because that interference is deterministic, the receiver can subtract it back out.

The gain: it achieves **2B bit/s over bandwidth B** — the ideal Nyquist efficiency of 2 bits/s/Hz — using a *realizable* filter, which the  $r = 0$  raised cosine could never do.

## 26. In duobinary, what do the three output levels tell you?

**+2d** → this bit and the previous one were both 1.

**−2d** → both were 0.

**0** → the two bits were different.

So the magnitude carries the information:  $|y_n|$  near  $2d$  means “same as before”, near  $0$  means “different from before”.

### 27. What is the precoder in a duobinary system for?

Plain duobinary decoding uses  $\hat{b}_n = y_n - \hat{b}_{n-1}$ , which depends on the *previous decision*. One wrong decision would then poison every decision after it — error propagation. The precoder’s XOR-and-delay at the transmitter arranges things so the receiver can decide each bit independently, breaking that chain.

## Modules 5–6 — information and capacity

### 28. A source has four symbols with probabilities 0.5, 0.25, 0.125, 0.125. Find $H(X)$ .

The logs are exact here:  $\log_2 0.5 = -1$ ,  $\log_2 0.25 = -2$ ,  $\log_2 0.125 = -3$ .

$$H(X) = -[0.5(-1) + 0.25(-2) + 0.125(-3) + 0.125(-3)]$$

$$= 0.5 + 0.5 + 0.375 + 0.375 = \mathbf{1.75 \text{ bits/symbol.}}$$

Check: maximum would be  $\log_2 4 = 2$ , and  $1.75 < 2$ . ✓

### 29. Why is entropy maximum when all symbols are equally likely?

Entropy measures average uncertainty. When probabilities are lopsided, you can partly guess what is coming, so on average each symbol surprises you less. When all symbols are equally likely, you have no basis for a guess at all, so every symbol delivers the maximum possible surprise. That maximum equals  $\log_2 m$ .

### 30. A source with $H(X) = 2.5$ bits/symbol emits 1000 symbols/sec. What is $R$ ?

$$R = r H(X) = 1000 \times 2.5 = \mathbf{2500 \text{ bits/sec.}}$$

### 31. Why must every row of a channel matrix sum to 1, but not every column?

Each row corresponds to one *transmitted* symbol, and lists the probabilities of everything it might turn into. Something must come out, so those probabilities must account for 100% of cases — hence each row sums to 1. A column corresponds to

one *received* symbol and collects contributions from different inputs; there is no rule forcing those to total 1.

### 32. A BSC has $p = 0.1$ . Find its capacity per symbol.

$$\begin{aligned} C_s &= 1 + p \log_2 p + (1-p) \log_2 (1-p) \\ &= 1 + 0.1 \log_2 (0.1) + 0.9 \log_2 (0.9) \\ &= 1 + 0.1(-3.322) + 0.9(-0.152) \\ &= 1 - 0.3322 - 0.1368 = \mathbf{0.531 \text{ bits/symbol}}. \end{aligned}$$

So a 10% error rate costs you nearly half your capacity.

### 33. A channel has $B = 4 \text{ kHz}$ and $\text{SNR} = 20 \text{ dB}$ . Find its capacity.

**First convert dB to a ratio:**  $S/N = 10^{20/10} = 10^2 = 100$ .

$$C = B \log_2 (1 + S/N) = 4000 \times \log_2 (101) = 4000 \times 6.658 = \mathbf{26\,632 \text{ bits/s}}.$$

### 34. State Shannon's channel coding theorem and say why it is surprising.

If the information rate of a source does not exceed the channel capacity, then a coding technique exists that allows transmission over that unreliable channel with an **arbitrarily low** error rate.

It is surprising because before Shannon everyone assumed that cutting errors meant slowing down towards zero. Shannon showed you can run at full speed right up to  $C$  and still get near-perfect reliability. The catch is that he proved such codes exist without showing how to build them.

## Modules 7–8 — source and channel coding

### 35. A code has $H(X) = 1.75$ and $L = 1.80$ . Find $\eta$ and $\gamma$ .

$$\eta = H(X)/L = 1.75/1.80 = \mathbf{0.972}, \text{ so } 97.2\% \text{ efficient.}$$

$$\gamma = 1 - \eta = \mathbf{0.028}, \text{ so } 2.8\% \text{ redundancy.}$$

### 36. Can an instantaneous code exist with lengths 1, 2, 2, 3?

$$\text{Apply the Kraft inequality: } K = 2^{-1} + 2^{-2} + 2^{-2} + 2^{-3} = 0.5 + 0.25 + 0.25 + 0.125 = \mathbf{1.125}.$$

Since  $1.125 > 1$ , the budget is overspent, so **no** such instantaneous code can exist.

### 37. Two students produce different Huffman codes for the same source. Can both be right?

**Yes.** Which branch you label 0 or 1, and how you break ties between equal probabilities, are free choices — so the individual code words can legitimately differ. What cannot differ is the average length  $L$ . If both students get the same  $L$ , both answers are correct.

### 38. A code has $d_{\min} = 7$ . How many errors can it detect and correct?

Detect:  $d_{\min} - 1 = 6$  errors.

Correct:  $\lfloor (7 - 1)/2 \rfloor = \lfloor 3 \rfloor = 3$  errors.

### 39. What is the code rate of a (15,11) Hamming code, and what is its $r$ ?

$R = k/n = 11/15 \approx 0.733$ .

Since  $n = 2^r - 1 = 15$ , we get  $2^r = 16$ , so  $r = 4$  parity bits. Check:  $k = 2^4 - 4 - 1 = 11$ . ✓

### 40. Why does Viterbi decoding use a trellis rather than a tree?

A tree doubles in size at every step, so after  $N$  bits it has  $2^N$  branches — computationally impossible. The trellis notices that the encoder keeps returning to the **same small set of states**, so it folds those repeated branches back together. The trellis stays a fixed height regardless of message length, which turns an exponential problem into one whose work per step is constant.

## WHERE TO GO NEXT

You have now covered everything in the materials you gave me. Here is exactly where your syllabus stands.

### Fully covered by this course:

- **Unit 2 — complete.** Digital transmission through bandlimited channels, signal design for bandlimited channels, and system design in the presence of channel distortion. That is Module 4, plus the line-coding half of Module 3.
- All the background the other units rest on: PCM and quantization, line codes, information theory, channel capacity, source coding and channel coding.

### Still uncovered, because no material exists for them yet:

- **Unit 1:** geometric representation of signal waveforms, the optimum receiver for binary signals in AWGN, M-ary pulse modulation and its error probability, symbol synchronisation.
- **Unit 2, one remaining piece:** the full derivation of  $P_e$  (probability of error) for digital PAM.
- **Unit 3:** carrier modulation — ASK, PSK, DPSK and FSK demodulation and detection, and their error probabilities.

- **Unit 4:** spread spectrum — DSSS, FHSS, PN sequence generation, CDMA.
- **Unit 5:** multicarrier modulation and OFDM — cyclic prefix, PAPR, subcarrier and power allocation, channel estimation.

Send me the lecture slides or tutorials for any of those and I will extend this course with the same treatment — simple English, every formula explained, every tutorial problem worked, and the figures lifted straight from your slides.

---

Built from syllabus.pdf, first.pdf, second.pdf, third.pdf, fourth.pdf, fifth.pdf, sixth.pdf, seventh.pdf, eighth.pdf and ninth.pdf — 171 pages in total. Source decks by Prof. U. D. Dalal and the Department of Electronics Engineering, SVNIT Surat; textbook figures reproduced from those decks, originally from B. P. Lathi and from Forouzan. Tutorial and example numbers are kept as printed so you can check every worked solution against your own PDFs.

**Three deliberate departures from the source, each flagged in place:** the code rate on Channel Coding slide 8 is taught as  $R = k/n$  rather than the printed  $n/k$  (Module 8); ninth.pdf Example 4 states 10 Mbps but prints the answer for 1 Mbps, so the method is taught and the arithmetic corrected (Module 3); and the duobinary postcoder rule on the last slide of ninth.pdf is taught as a test on  $|y_n|$ , which is the version that actually works (Module 4).